

PROFEPI Programa de Fortalecimento da Epidemiologia nos Serviços de Saúde

Introdução à

Inteligência Artificial para Predições

em Vigilância em Saúde e Ambiente

E-book

completo

OPAS Organização Pan-Americana da Saúde / Organização Mundial da Saúde Américas

SUS

MINISTÉRIO DA SAÚDE

GOVERNO DO BRASIL
DO LADO DO POVO BRASILEIRO

MINISTÉRIO DA SAÚDE
SECRETARIA DE VIGILÂNCIA EM SAÚDE E AMBIENTE (SVSA)

Coordenação-Geral

Mariângela Simão

Secretária de Vigilância em Saúde e Ambiente/SVSA/MS

Guilherme Loureiro Werneck

Diretor do Departamento de Ações Estratégicas de Epidemiologia e Vigilância em Saúde e Ambiente/Daevs/SVSA/MS

Vivian Siqueira Santos Gonçalves

Coordenadora-Geral de Desenvolvimento da Epidemiologia em Serviços – CGDEP/Daevs/SVSA/MS

Equipe Técnica

Amanda Maciel de Quadros - CGDEP/Daevs/SVSA/MS

Antonio Ygor Modesto de Oliveira - CGDEP/Daevs/SVSA/MS

Cíntia Pereira Donateli - CGDEP/Daevs/SVSA/MS

Criação Digital

Matheus Nunes Oliveira Alves - CGDEP/Daevs/SVSA/MS

Otávio Francisco Batista Martins - CGDEP/Daevs/SVSA/MS

Professores Conteudistas

Alexandre Dias Porto Chiavegatto Filho - Universidade de São Paulo (USP)

Keisyanne de Araujo Moura - Universidade de São Paulo (USP)

Tutores

André Henrique Alves Carneiro - Universidade de São Paulo (USP)

Marcela Quaresma Soares - Universidade de São Paulo (USP)

Vinícius de Camargo - Universidade de São Paulo (USP)

ORGANIZAÇÃO PAN-AMERICANA DE SAÚDE ORGANIZAÇÃO MUNDIAL DA SAÚDE (OPAS/OMS)

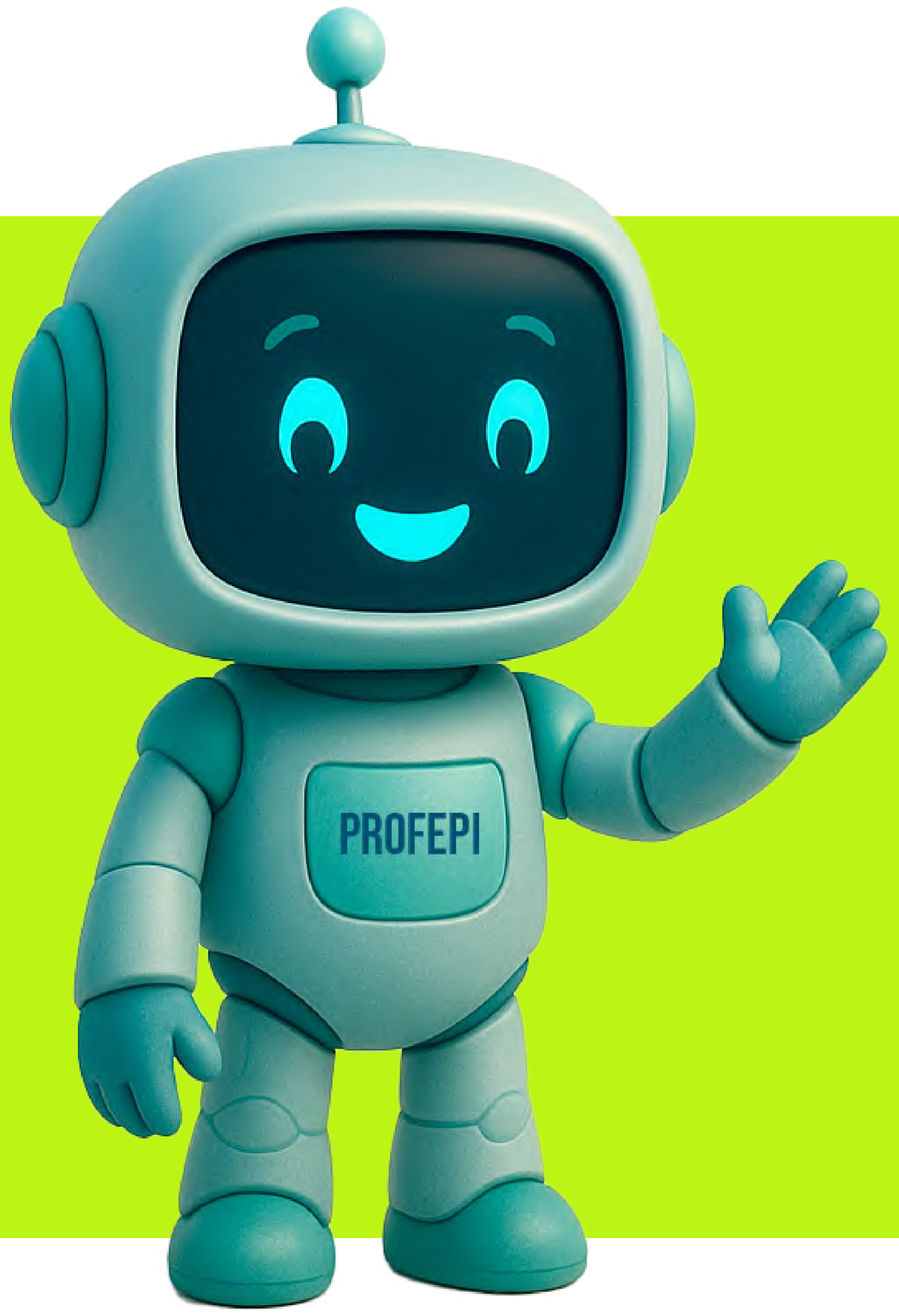
Coordenação-Geral

Cristian Morales Fuhrimann - Representante da OPAS/OMS no Brasil

Alexander Rosewell - Coordenador de Emergências, Evidência e Inteligência em Saúde no Brasil

Equipe Técnica

Josy Maria de Pinho da Silva - OPAS/OMS



Boas-vindas ao Curso

Seja bem-vindo(a) ao curso de "Introdução à Inteligência Artificial para Predições em Vigilância em Saúde e Ambiente". Esta jornada foi preparada para oferecer a você uma base sólida sobre como as tecnologias de IA, e mais especificamente a área de *machine learning*, estão revolucionando a forma como monitoramos, predizemos e agimos sobre questões de saúde pública e ambientais.

O material desta aula foi desenvolvido pela equipe de pesquisadores do **Laboratório de Big Data e Análise Preditiva em Saúde (LABDAPS-USP)** da Faculdade de Saúde Pública da USP.

Por que este assunto é tão relevante hoje?

Você pode estar se perguntando: por que tanto se fala em Inteligência Artificial ultimamente? A resposta está na confluência de três fatores principais que têm impulsionado avanços exponenciais na área:

1. **Aumento Exponencial de Dados (Big Data):** nunca na história coletamos tantos dados em saúde. Desde prontuários eletrônicos e dados de sensores ambientais até informações genômicas, temos um volume massivo de informações que podem ser analisadas para extrair decisões inteligentes.

2. **Avanços em Capacidade Computacional:** o poder de processamento dos computadores cresceu de forma vertiginosa. Empresas de tecnologia têm desenvolvido *chips* e processadores especializados para tarefas de IA, tornando possível treinar modelos complexos cada vez mais rapidamente.

3. **Desenvolvimento de Novos Algoritmos:** a comunidade científica não parou de inovar. Novos algoritmos, como as redes neurais do tipo *transformers* e modelos como o *TabPFN*, foram criados para resolver problemas cada vez mais complexos e com maior precisão.

A importância da área foi recentemente destacada pelo Prêmio Nobel de 2024. O prêmio de Física foi concedido a John J. Hopfield e Geoffrey E. Hinton, "por descobertas e invenções fundamentais que permitem o aprendizado de máquina com redes neurais artificiais". Na Química, Demis Hassabis e John M. Jumper foram laureados pelo desenvolvimento do algoritmo de *machine learning* chamado AlphaFold, que permite predizer a estrutura de proteínas a partir da sua sequência de aminoácidos.

capítulo 1

MÓDULO 1



O que é *Machine Learning*?

1.1 Diferenciando IA Clássica de *Machine Learning*

A Inteligência Artificial (IA) é um campo vasto, mas, quando falamos de predições, *Machine Learning* (ML), ou Aprendizado de Máquina, é a área protagonista. Para entender o que torna ML tão especial, vamos compará-lo com a IA clássica.

Inteligência Artificial Clássica:

neste paradigma, as **regras para a tomada de decisão são preestabelecidas por humanos**. Aqui, programadores e especialistas dizem ao computador exatamente o que fazer.

- **Exemplo 1**: para identificar um e-mail como *spam*, criaríamos uma lista de palavras-chave (como “oferta”, “grátis”, “promoção”), de forma que o e-mail que contivesse essas palavras seria classificado como *spam*.
- **Exemplo 2**: para identificar um rosto humano em uma foto, programaríamos o sistema para procurar formas específicas que se assemelham a nariz, olhos e boca.

Inteligência Artificial com *Machine Learning*:

aqui é onde a mágica acontece. Em vez de darmos as regras, nós permitimos que as **máquinas aprendam as regras de decisão por conta própria!** O aprendizado ocorre por meio da identificação de padrões complexos diretamente dos dados. É um processo muito semelhante ao de como uma criança aprende: por meio de exemplos e da experiência.

FIQUE ATENTO!

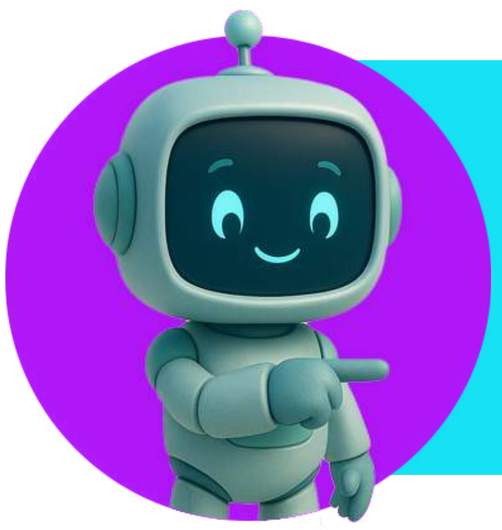
O foco de *machine learning* é resolver problemas práticos de predição para auxiliar na tomada de decisão. Na maioria das suas aplicações, o interesse principal não é entender *como* o modelo chegou a uma conclusão (interpretação), mas sim a sua performance preditiva. Isso nos dá liberdade para usar modelos matematicamente complexos, como **Redes Neurais Profundas**, **Xgboost**, e **Random Forests**, que são capazes de capturar a complexidade do mundo real.

1.2 As Quatro Grandes Categorias de *Machine Learning*

A área de *machine learning* pode ser dividida em quatro categorias principais. Entender cada uma delas é fundamental para saber qual abordagem usar em diferentes problemas.

1. Aprendizado Supervisionado

Esta é a categoria mais comum e intuitiva. No aprendizado supervisionado, o objetivo é **acertar um resultado que já existe e que conhecemos**. Para isso, treinamos o algoritmo com um conjunto de dados que inclui a "solução" ou o "rótulo" (*label*) desejado.



Imagine que você tem dados de pacientes (idade, IMC, pressão arterial) e sabe quais deles vão desenvolver uma doença cardíaca ao longo dos próximos cinco anos (o "rótulo"). O algoritmo aprende a relação entre os dados do paciente e o resultado (ter ou não a doença), para, no futuro, predizer o risco de ocorrência desse desfecho para novos pacientes.

O aprendizado supervisionado se divide em dois tipos de problemas:

Classificação: a variável que queremos predizer é **categórica**.

Exemplos: predizer se um paciente irá a óbito em cinco anos (Sim/Não); se um tumor é benigno ou maligno; ou qual a espécie de uma flor.

Regressão: a variável que queremos predizer é **quantitativa** ou contínua.

Exemplos: predizer quantos meses de sobrevida um paciente terá; qual será o seu IMC no próximo ano; ou o nível de poluição do ar amanhã.

2. Aprendizado Não Supervisionado

Nesta categoria, os dados **não possuem um rótulo ou uma resposta certa**. O objetivo não é predizer um resultado específico, mas sim **encontrar padrões e estruturas ocultas nos próprios dados**. Os tipos mais comuns de aprendizado não supervisionado são:

***Clustering* (Agrupamento):** o algoritmo agrupa os dados em "*clusters*" ou grupos com base em suas similaridades. Por exemplo, poderíamos usar o *clustering* para identificar diferentes perfis de pacientes com base em seus estilos de vida e dados clínicos, sem saber previamente quais seriam esses perfis.

Redução de Dimensionalidade: usada para simplificar dados complexos com muitas variáveis, reduzindo-as a um número menor de componentes principais que ainda retêm a maior parte da informação relevante.

3. Aprendizado por Reforço (*Reinforcement Learning*)

Esta é uma área fascinante e promissora. No aprendizado por reforço, um "**agente**" **(o modelo) interage com um "ambiente" dinâmico**. O agente aprende a tomar uma sequência de ações para maximizar uma recompensa ao longo do tempo. Ele recebe *feedback* constante na forma de "prêmios" (por ações corretas) e "punições" (por ações incorretas).

- **Exemplos clássicos:** treinar uma IA para jogar xadrez, Go (como o famoso algoritmo AlphaGo, que ganhou do campeão mundial) ou videogames.
- **Aplicações em saúde:** esta área é muito promissora para otimizar tratamentos médicos. Por exemplo, um agente poderia aprender a sequência ótima de intervenções (ventilação mecânica, sedação, uso de vasopressores) para um paciente com sepse em uma UTI, com o objetivo de maximizar a chance de recuperação.

4. Aprendizado Autossupervisionado (*Self-supervised Learning*)

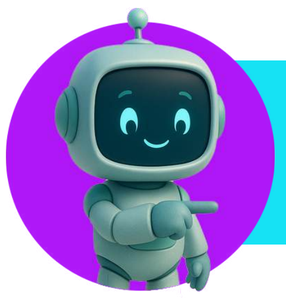
Esta é uma técnica mais recente e poderosa, que está por trás dos grandes modelos de linguagem, como o ChatGPT. Essencialmente, é uma forma de **transformar um problema não supervisionado em um problema supervisionado, gerando os rótulos automaticamente a partir dos próprios dados**.

O processo geralmente consiste em "esconder" partes dos dados e treinar o modelo para predizer essas partes ocultas. Por exemplo:

- **Em texto:** predizer a próxima palavra em uma frase.
- **Em imagens:** remover um pedaço de uma imagem e treinar o modelo para preenchê-lo.
- **Em dados sequenciais:** predizer o futuro a partir de dados do passado.

Essa abordagem permite treinar modelos gigantescos em enormes volumes de dados não rotulados (como toda a internet), um processo chamado de **pré-treinamento (*pre-training*)**. Depois, esses modelos podem ser ajustados (*fine-tuning*) com conjuntos de dados menores e específicos para tarefas particulares, como seguir instruções, usando técnicas como *Reinforcement Learning from Human Feedback* (RLHF).

Predição *vs.* Inferência – Qual o seu objetivo?



Antes de iniciar qualquer projeto de *machine learning*, é necessário fazer uma pergunta fundamental: **o seu objetivo é predição ou inferência?**

- **Inferência:** o interesse está em **entender a relação entre as variáveis**. Queremos saber *como* uma variável de resultado (Y) muda quando alteramos um de seus determinantes (X). Para isso, usamos modelos mais simples e interpretáveis, como regressões linear e logística.
- **Predição:** o interesse está na **performance preditiva**. Queremos o modelo mais acurado possível para predizer um resultado, mesmo que não entendamos completamente suas regras internas.

FIQUE ATENTO!

Existe um *trade-off* (uma troca) bem conhecido entre predição e interpretação. Modelos que são fáceis de interpretar geralmente têm uma performance preditiva inferior, enquanto os modelos mais precisos (como redes neurais profundas) não têm uma interpretação tão direta. Saber o seu objetivo de pesquisa desde o início guiará toda a sua abordagem.

Um Guia Prático para Evitar Erros em *Machine Learning*

Baseado no artigo "How to avoid machine learning pitfalls: a guide for academic researchers", de Michael A. Lones, apresentamos este guia prático para estudantes e pesquisadores. O objetivo é elencar erros comuns e mostrar como evitá-los, garantindo que seus modelos sejam robustos, confiáveis e úteis. **Todos os conceitos apresentados nesta parte serão revistos com maior profundidade ao longo do curso**.

Etapa 1: o que fazer antes de construir um modelo

A preparação é boa parte do sucesso do desenvolvimento de uma análise preditiva. Seguir estes passos antes de escrever uma única linha de código de modelo pode salvar o seu projeto de muitos erros futuros. Ignorar esta etapa é como construir uma casa sem uma fundação sólida: a estrutura pode até subir, mas estará perpetuamente em risco de colapso. Ampliar a compreensão sobre esta fase inicial revela um processo metodológico que transforma um problema em uma questão que pode ser resolvida por um modelo preditivo. Seguir estes passos não apenas salva o projeto de erros futuros, mas também garante que o modelo final seja relevante, preciso e gere valor real para a tomada de decisão em saúde.

1

Entenda os seus dados: saiba de onde eles vêm.

- Como os dados foram coletados?
- Existem artigos que descrevem esse banco de dados?
- As variáveis disponíveis fazem sentido para o problema de predição que você quer resolver?
- **Sempre comece com uma análise exploratória dos dados** para entender suas distribuições, valores faltantes e relações preliminares.

2

Não olhe para todos os seus dados (ainda!): um dos erros mais graves é analisar o conjunto de teste durante a fase exploratória. Isso pode levar a um "vazamento de dados" (*data leakage*), em que, consciente ou inconscientemente, você toma decisões de modelagem baseadas em informações que não estariam disponíveis no mundo real, inflando artificialmente o desempenho do seu modelo.

3

Certifique-se de que há dados suficientes: sem dados suficientes, é impossível treinar um modelo que generalize bem para novos dados. A quantidade necessária depende da complexidade do problema: um "sinal" forte nos dados (uma alta importância preditiva das variáveis) exige menos dados; um sinal fraco exige mais. Se os dados forem escassos, técnicas como a **validação cruzada** serão necessárias. Para dados desbalanceados (por exemplo, poucos casos de uma doença rara), técnicas como *data augmentation* podem, em alguns casos, ser úteis para aumentar o número de observações.

4

Converse com especialistas no assunto: um cientista de dados não trabalha sozinho. Especialistas da área (médicos, enfermeiros, biólogos) podem:

- ajudar a entender o problema e definir quais variáveis são realmente úteis;
- garantir que o modelo seja útil na prática;
- validar se a importância que o modelo dá a certas variáveis faz sentido e ajudar a identificar possíveis vazamentos de dados.

5

Faça um levantamento da literatura: o progresso científico é um processo iterativo. Veja o que outros pesquisadores já fizeram para o mesmo problema. Isso permite aproveitar o conhecimento existente e construir sobre ele, em vez de reinventar a roda.

6

Pense em como o modelo será utilizado: faça a si mesmo a pergunta: "Por que estou construindo este modelo?". Pense na sua implementação no mundo real. Ele será integrado a um prontuário eletrônico? Ou a um aplicativo para pacientes? A resposta a essas perguntas influenciará o tipo de modelo, o momento da coleta dos dados, as variáveis necessárias e as métricas de avaliação.

Etapa 2: como construir modelos confiáveis

Após uma preparação cuidadosa dos dados, a jornada de análise preditiva entra na sua fase mais emblemática: a construção do modelo. Esta etapa é onde a teoria se encontra com a prática e onde as decisões técnicas definem a robustez e a confiabilidade das previsões futuras. Construir um modelo confiável não é apenas uma questão de alimentar dados em um algoritmo. É um processo disciplinado que exige rigor metodológico para garantir que os resultados sejam válidos, generalizáveis e, acima de tudo, úteis para a tomada de decisão em saúde. A disciplina aqui é a chave para evitar armadilhas comuns que podem invalidar todo o trabalho realizado até agora, transformando um modelo promissor em uma ferramenta de predição ineficaz e enganosa.

1

Impeça o vazamento de dados para o teste: este é o mandamento número um.

- **Separe seu conjunto de teste no início e NUNCA toque nele** até a avaliação final do modelo. Qualquer vazamento torna os resultados inúteis e não confiáveis.
- **Exemplo de vazamento:** realizar o pré-processamento (como normalização ou imputação de dados faltantes) usando informações do conjunto de dados completo antes de dividi-lo. A seleção de variáveis e a otimização de hiperparâmetros devem ser feitas usando apenas os dados de treino.

2

Teste vários modelos: não existe um algoritmo que seja o melhor para todos os problemas. Esse conceito é conhecido como o Teorema do Almoço Grátis (*No Free Lunch Theorem*). Portanto, experimente diferentes tipos de algoritmos (tais como redes neurais, random forest, xgboost) para encontrar aquele que gera a melhor predição para o seu problema específico. Ferramentas como a biblioteca Scikit-learn, em Python, facilitam muito esse processo.

3

Não use modelos inapropriados ou desnecessariamente complexos:

- Não use modelos que são inadequados para seus dados (por exemplo, um modelo que só aceita variáveis numéricas para um banco de dados com muitas variáveis categóricas);
- Evite a complexidade desnecessária. Usar *deep learning* para um problema com dados tabulares simples e poucos dados é, na maioria das vezes, um exagero que pode levar a um desempenho ruim;
- Cuidado com o *hype*: modelos mais recentes nem sempre são melhores. Seja cético com artigos de *benchmark* em que os autores têm um interesse claro em que seu novo algoritmo vença os demais.

4

Otimize os hiperparâmetros: hiperparâmetros são "configurações" do modelo que não são aprendidas durante o treinamento, mas que afetam seu comportamento e desempenho (como o número de árvores em um algoritmo *Random Forest*). Use estratégias de otimização como *Grid Search* ou *Random Search* para encontrar a melhor combinação de hiperparâmetros.

- **Lembre-se:** essa otimização deve ser feita usando apenas os dados de treino, idealmente dentro de um esquema de validação cruzada.

Etapa 3: como avaliar modelos

Com um modelo treinado e otimizado em mãos, a próxima etapa do ciclo de desenvolvimento de um algoritmo preditivo é a sua avaliação. Esse é o momento em que se verifica se todo o esforço resultou em uma ferramenta que realmente funciona no mundo real. Uma avaliação honesta e objetiva depende de duas coisas: testar o modelo em um conjunto de dados intocado; e selecionar métricas de performance que façam sentido para o modelo preditivo escolhido, evitando armadilhas comuns que podem levar a uma falsa sensação de sucesso.

1. **Use um conjunto de teste adequado:** a performance final do seu modelo deve ser **sempre** avaliada em um conjunto de teste que não foi usado em nenhuma etapa do treinamento ou da otimização. Esse conjunto de dados deve ser representativo da população onde o modelo será aplicado.

2. **Não use acurácia com dados desbalanceados:** a acurácia (percentual de acertos) é uma métrica enganosa quando as classes são desbalanceadas. Imagine um teste para uma doença rara que afeta 1% da população. Um modelo que sempre prevê "não tem a doença" terá 99% de acurácia, mas será completamente inútil. Em saúde, em que os dados são frequentemente desbalanceados, use métricas mais robustas, como a **Área sob a Curva ROC (AUC), Precisão, Recall ou F1-Score**.

Etapa 4: como comparar modelos

Após ter avaliado o desempenho de diferentes modelos, a etapa seguinte consiste em compará-los para selecionar a melhor opção. Essa decisão, contudo, é mais sofisticada do que apenas escolher o modelo com maior valor em uma métrica. É importante garantir que a comparação seja justa, especialmente ao comparar seus resultados com *benchmarks* externos, e considerar que a solução mais poderosa pode não ser um modelo único, mas sim uma combinação de vários.

1. **Não presuma que valores maiores significam modelos melhores automaticamente:** ao comparar seu resultado com o de outros estudos, seja cauteloso. As comparações devem ser justas. O desempenho de um modelo depende de inúmeros fatores, como o banco de dados usado, a forma como os dados foram divididos e pré-processados, a otimização de hiperparâmetros, se não houve vazamento de dados etc.

2. **Considere combinações de modelos (*ensembles*):** frequentemente, combinar as predições de vários modelos diferentes pode gerar um resultado mais robusto e preciso do que qualquer modelo individual. Técnicas como ***Bagging*** (ex.: *Random Forest*) e ***Boosting*** (ex.: *XGBoost*) são exemplos poderosos de métodos de *ensemble*.

Etapa 5: como relatar resultados

Um modelo, por mais preciso que seja, só gera valor real quando seus resultados são compreendidos, confiáveis e comunicados de forma eficaz. A etapa final de relatar os resultados é na qual o rigor técnico se transforma em impacto prático. Não se trata apenas de anunciar a performance final, mas de construir confiança por meio da transparência do processo, definindo claramente os limites de aplicação do modelo e explicando como ele chega às suas conclusões.

1. **Seja transparente:** o objetivo da pesquisa é contribuir com o conhecimento. Descreva todas as suas decisões e os *trade-offs* que você fez. Se possível, **compartilhe seu código e o modelo treinado** de forma acessível (como em um repositório GitHub). Um código limpo e bem documentado facilita a reprodutibilidade, que é um pilar da ciência.

2. **Não generalize para além do seu conjunto de dados:** seja humilde em suas conclusões. Um modelo treinado com dados de um hospital específico em São Paulo pode não funcionar bem em um hospital em outra região do país. Esteja ciente das limitações do seu estudo e da distribuição dos seus dados.

3. **Olhe seus modelos:** tente entender o que o modelo aprendeu. Use técnicas de **explicabilidade (XAI – *Explainable* AI)**, como os valores de SHAP (SHapley Additive exPlanations), para mostrar como o modelo tomou suas decisões. Isso não só gera conhecimento, mas também aumenta a confiança no seu modelo.



Conclusão e Próximos Passos

Neste capítulo introdutório, cobrimos os conceitos fundamentais da Inteligência Artificial e do *Machine Learning*, diferenciando as abordagens, explorando as principais categorias de aprendizado e, mais importante, estabelecendo um roteiro de boas práticas para a construção e a avaliação de modelos preditivos.

A chave para o sucesso em *machine learning*, assim como em qualquer área da ciência, é manter a mente aberta, acompanhar o desenvolvimento constante da área e, acima de tudo, aceitar que sempre há mais a aprender.

No próximo capítulo, vamos colocar a mão na massa e explorar implementações práticas usando Python.

Referência Principal:

LONES, Michael A. **How to avoid machine learning pitfalls**: a guide for academic researchers. arXiv preprint arXiv:2108.02497, 2021.

capítulo 2

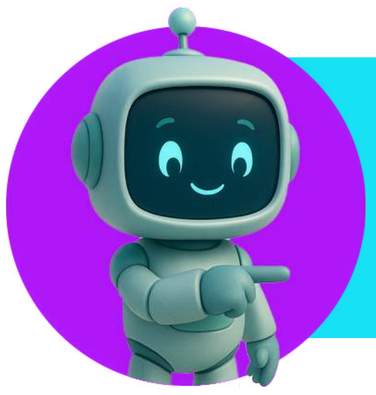
MÓDULO 1



Introdução ao Python para análise de dados em saúde

Por que Python?

Python é uma linguagem de programação acessível, intuitiva e cada vez mais utilizada na área da saúde. Seja para automatizar rotinas, realizar análises estatísticas ou construir *dashboards* (painéis de indicadores), aprender Python abre portas para o uso de ferramentas de ponta na Vigilância em Saúde.



Além disso, Python possui uma comunidade ativa e vasta documentação oficial. Nesta aula, vamos trabalhar os primeiros comandos da linguagem e entender como armazenar, manipular e comparar dados.

Na área da saúde, lidar com grandes volumes de dados é parte do trabalho cotidiano dos profissionais de Vigilância em Saúde. Isso inclui bases de dados epidemiológicos, registros hospitalares, dados de vacinação, entre outros. Por isso, ferramentas computacionais que ajudam nesse processo têm ganhado cada vez mais espaço. Duas linguagens de programação destacam-se nesse contexto:

- **R:** muito utilizada em **estatística**, com forte presença na saúde pública por suas bibliotecas voltadas à análise de dados e bioestatística;
- **Python:** vem se destacando pela **flexibilidade** e capacidade de **integração** com diversas áreas da tecnologia. É frequentemente adotado em projetos de ciência de dados, **inteligência artificial** e automação em saúde.

Para quem já conhece o R, a boa notícia é que grande parte do raciocínio lógico e dos conceitos de análise de dados podem ser aproveitados ao aprender Python. Ter familiaridade com as duas linguagens **amplia as possibilidades** de análise e inovação em saúde coletiva.

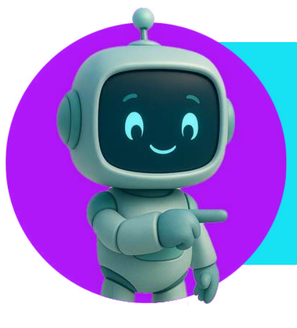
Ambiente de prática: Google Colaboratory (Colab)

Uma das maneiras mais acessíveis de começar a programar em Python é utilizando o **Google Colab** – uma plataforma online que permite executar código Python diretamente do navegador, sem necessidade de instalar nenhum *software*. O Colab é especialmente útil para profissionais de saúde que talvez não tenham um ambiente de programação configurado em seus computadores.

No Colab, você pode criar documentos interativos chamados **notebooks** (algo como "cadernos de anotações" em português), que combinam código executável com textos explicativos, imagens e gráficos. O notebook é composto por **células**, que podem ser de dois tipos principais:

- **Células de código** – nas quais escrevemos comandos em Python. Ao executar a célula, o código é interpretado e o resultado é exibido logo abaixo. Dentro dessas células, podemos usar o símbolo #, conhecido como cerquilha, *hashtag* ou jogo da velha, para adicionar **comentários** – texto que o Python ignora, útil para documentar o que o código faz.

- **Células de texto (Markdown)** – nelas escrevemos explicações, títulos, listas etc., usando a sintaxe Markdown. Essas células não executam código; servem para contextualizar e documentar.



A sintaxe Markdown é uma forma simples de formatar textos, usando símbolos como # para títulos, * para listas e ** para negrito, por exemplo. Ela permite organizar o conteúdo de forma clara e legível, sem complicações.

FIQUE ATENTO!

Registrar anotações e explicações junto aos códigos, seja como comentários nas células de código ou células de Markdown é considerado uma boa prática, pois facilita a compreensão do que está sendo feito, tanto para outras pessoas quanto para você mesmo no futuro.

No Colab, é possível **alternar** entre editar código e texto facilmente, tornando a experiência de aprendizado mais rica: você pode ver um exemplo de código, executá-lo e logo abaixo ler uma explicação ou comentário sobre aquele código.

Conceitos básicos: variáveis e operações

Assim como em outras linguagens, no Python utilizamos **variáveis** para armazenar valores que serão usados durante a execução do programa. Podemos pensar em uma variável como uma "**etiqueta**" que identifica um dado – seja um número, um texto ou outro tipo de informação – para que possamos reutilizá-lo quando necessário. A seguir, vamos revisar os principais tipos de dados que podem ser armazenados em variáveis no Python.

TIPO	DESCRIÇÃO	EXEMPLO
int (integer)	Números **inteiros** (sem casas decimais)	`0, 5, -3`
float	Números de ponto flutuante (**com casas decimais**)	`2.5, 0.125, -1.0`
str (string)	Texto (sequência de **caracteres**)	`"Brasil", 'Saúde'`
bool	Booleano (**lógico**) – valores verdadeiro ou falso	`True, False`

Observe que, em Python, os valores lógicos **True** e **False** iniciam com letra maiúscula. Além disso, texto (string) deve sempre estar entre aspas simples ('texto') ou duplas ("texto").

FIQUE ATENTO!

Atenção: Não misture os dois tipos de aspas no mesmo texto:

```
nome = “Carlos’
```

Aspas de tipos diferentes: começou com aspas duplas (“) e terminou com aspas simples (‘).

Criando e Usando Variáveis

Para criar uma variável em Python, basta escolher um nome e usar = para atribuir um valor. Por exemplo, vamos guardar um texto em uma variável e depois usar essa variável em outras operações:

Python - input

```
idade = 35
nome = “Maria”
total_pacientes = 1500

print() #exibe o conteúdo da variável na tela.
```

Note que atribuímos valores às variáveis `idade`, `nome` e `total_pacientes`. Para visualizar o conteúdo de variável, podemos utilizar a função `print()`, com o nome da variável dentro dos parênteses .

FIQUE ATENTO!

Boas práticas ao nomear variáveis:

- comece com letra;
- use underline para espaços: idade_paciente;
- não comece com números;
- evite caracteres especiais (@, #, %);
- Python diferencia maiúsculas de minúsculas: Idade ≠ idade

Operações Matemáticas no Python

Podemos usar Python como uma calculadora para realizar **operações aritméticas** básicas. Os principais operadores aritméticos em Python são semelhantes aos da matemática tradicional:

- + (adição);
- - (subtração);
- * (multiplicação);
- / (divisão comum, resultado em ponto flutuante);
- // (divisão inteira, sem as casas decimais);
- % (módulo, resto da divisão);
- ** (exponenciação, potência).

Veja alguns exemplos de uso desses operadores:

Python - input

```
print("Soma: ", 1 + 1)                    # Soma 1 + 1
print("Subtração: ", 10 - 5)              # Subtrai 5 de 10
print("Multiplicação: ", 4 * 3)           # Multiplica 4 por 3
print("Exponenciação (potência): ", 2 ** 3)  # 2 elevado a 3
print("Divisão comum: ", 20 / 3)          # Divide 20 por 3 (resultado float)
print("Divisão inteira: ", 10 // 3) # Divide 10 por 3 truncando o decimal
```

Executando essas linhas, o Python calcula cada expressão e o comando `print()` exibe os resultados:

Python - output

```
Soma:  2
Subtração:  5
Multiplicação:  12
Exponenciação (potência):  8
Divisão comum:  6.666666666666667
Divisão inteira:  3
```

Observe que, na divisão comum `20 / 3`, o resultado é um número de ponto flutuante (`6.666...`). Já `10 // 3` realiza a divisão inteira, resultando em `3` (o valor fracionário é descartado). Caso precisemos arredondar um número de ponto flutuante a uma certa quantidade de casas decimais, podemos usar a função de arredondamento `round(operação, n° de casas decimais desejadas)`.

Exemplo:

Python - input

```
round(20/3, 1) #arredonda o resultado da operação 20/3 de 6.666... para
uma casa decimal, resultando em 6.7.
```

Cálculo do IMC

Para contextualizar, vamos aplicar as operações em um exemplo de saúde pública. Suponha que queremos calcular o **Índice de Massa Corporal (IMC)** de uma pessoa. A fórmula do IMC é:

$$IMC = \frac{peso\ (kg)}{altura\ (m)^2}$$

Podemos facilmente calcular isso em Python:

Python - input

```
peso = 85       # peso em kg (exemplo)
altura = 1.72  # altura em metros (exemplo)

IMC = peso / (altura ** 2)
IMC = round(IMC, 1)  # arredonda o resultado para 1 casa decimal
print(IMC)
```

No código apresentado, definimos duas variáveis `peso` e `altura` com valores de exemplo. Em seguida, calculamos o `IMC` dividindo o peso pela altura ao quadrado (`altura ** 2`). Usamos `round(IMC, 1)` para arredondar o resultado para uma casa decimal, pois é usual apresentar o IMC com uma decimal. Por fim, imprimimos o IMC calculado.

Se `peso = 85` kg e `altura = 1.72` m, a saída (`print`) será:

Python - output

```
28.7
```

Esse valor indica o IMC daquela pessoa.

Operadores de Comparação (Relacionais) e Lógicos

Em muitos programas, precisamos **comparar valores** e tomar decisões com base nessas comparações – por exemplo, verificar se um resultado está acima de um limite, ou se duas variáveis representam o mesmo valor. Em Python, em vez de funções específicas para cada comparação, utilizamos **operadores** para essas verificações. Esses operadores são símbolos colocados entre dois valores e produzem um resultado do tipo booleano (`True` ou `False`).

Principais **operadores de comparação (relacionais)**:

- `==` igualdade (verifica se os valores dos dois lados são iguais);
- `!=` diferente (verifica se os valores são distintos);
- `>` maior que;
- `<` menor que;
- `>=` maior ou igual a;
- `<=` menor ou igual a .

Principais **operadores lógicos** (para combinar condições booleanas):

- `and` **E lógico** (`True` se **ambas** as condições forem verdadeiras);
- `or` **OU lógico** (`True` se **pelo menos uma** das condições for verdadeira);
- `not` **Negação** (inverte o valor lógico: `True` vira `False` e vice-versa).

Vamos ver uma tabela que resume o comportamento desses operadores com alguns valores de exemplo. Suponha que temos duas variáveis: `a = 1` e `b = 2`. Então:

Operador	Significado	Exemplo	Resultado
`==`	Igual a	`a == b`	`False`
`!=`	Diferente de	`a != b`	`True`
`>`	Maior que	`a > b`	`False`
`<`	Menor que	`a < b`	`True`
`>=`	Maior ou igual a	`a >= b`	`False`
`<=`	Menor ou igual a	`a <= b`	`True`
`and`	**E** lógico	`a > 0 and b > 0`	`True`
`or`	**OU** lógico	`a > 0 or b < 0`	`True`
`not`	**Negação**	`not (a == b)`	`True`

Como podemos observar, cada comparação gera um resultado **booleano** (`True` ou `False`).

Por exemplo, com `a=1` e `b=2`:

- `a == b` resulta em `False` (1 não é igual a 2);
- `a != b` resulta em `True` (1 é diferente de 2);
- `a < b` resulta em `True` (1 é menor que 2); e assim por diante.

Os operadores lógicos permitem combinar comparações: em `a > 0 and b > 0`, ambas as comparações são verdadeiras, então o resultado do **and** é `True`. Já `a > 0 or b < 0` também é `True`, porque **pelo menos uma** das condições (a primeira) é verdadeira. O operador `not` inverte o resultado lógico: `not (a == b)` era `not (False)`, portanto retorna `True`.

Esses operadores serão muito úteis quando formos introduzir **estruturas condicionais**, permitindo que o programa tome decisões com base em condições, como veremos a seguir.

Listas em Python

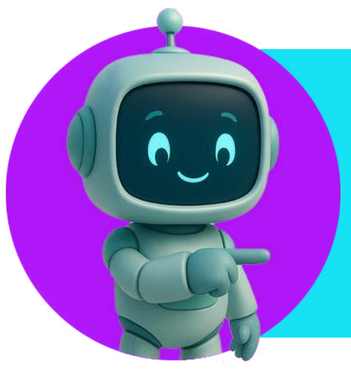
Em Python, uma **lista** é uma estrutura de dados que armazena **múltiplos valores em uma única variável**. Podemos imaginar uma lista como uma **coleção ordenada** de itens, semelhante a uma coluna em uma planilha ou uma lista de nomes em papel.

Algumas características importantes das listas:

- os itens de uma lista ficam **separados por vírgulas**, e a lista em si é delimitada por **colchetes** `[ ]`;
- uma lista pode armazenar elementos de qualquer tipo: números, strings, booleanos, ou mesmo outras estruturas. Podemos ter valores repetidos na lista, e a ordem de inserção é mantida (ou seja, o primeiro item inserido fica no índice 0, o segundo no índice 1 etc.).

Anota essa informação!

Em Python, a contagem dos elementos de uma lista começa em 0 – ou seja, o primeiro item está no índice 0, o segundo no índice 1 e assim por diante.



Para conectar esses conceitos de programação com uma aplicação em saúde pública, vamos trabalhar com um exemplo prático. Imagine um **surto de uma doença transmitida por alimentos (DTA)** ocorrido após um evento, e queremos analisar os dados dos expostos e dos doentes usando Python.

Cenário: Em uma confraternização, foi servida uma maionese caseira. Suspeita-se que a maionese pode ter causado um surto de gastroenterite. Temos uma lista de convidados do evento, quem consumiu a maionese e quem apresentou sintomas posteriormente. Nosso objetivo é cruzar essas informações para identificar possíveis relações.

Vamos criar três listas para o nosso cenário do surto alimentar:

Python

```
# Lista com os nomes de todos os convidados
convidados = [“João”, “Maria”, “Carlos”, “Tatiane”, “Ana”]

# Lista com os nomes de quem comeu a maionese
comeu_maionese = [“João”, “Carlos”, “Ana”]

# Lista com os nomes de quem apresentou sintomas após o evento
teve_sintomas = [“João”, “Ana”]
```

Dessa forma, definimos:

- `convidados`: contém cinco nomes de participantes do evento;
- `comeu_maionese`: contém os nomes dos convidados que consumiram a maionese;
- `teve_sintomas`: contém os nomes dos convidados que tiveram sintomas (por exemplo, diarreia, náusea etc.).

Assim, temos nossos **dados** estruturados em listas. Agora, vamos explorar como extrair informações úteis deste cenário com programação.

Estrutura Sequencial

Uma **estrutura sequencial** é o tipo mais básico de organização de um algoritmo: as instruções são executadas **linha a linha, na ordem em que aparecem**, de cima para baixo, sem desvio de fluxo.

No contexto do nosso exemplo, já vimos um caso de execução sequencial – a criação das listas acima. Cada comando foi executado na ordem e as listas foram preenchidas.

Podemos adicionar mais comandos sequenciais para mostrar, por exemplo, um pequeno relatório inicial dos dados coletados:

Python – input

```
print(“Relatório inicial”)
print(“Convidados:”, convidados)
print(“Comeram maionese:”, comeu_maionese)
print(“Tiveram sintomas:”, teve_sintomas)
```

Se executarmos esse bloco de código, a saída produzida será:

Python - output

```
Relatório inicial
Convidados: [‘João’, ‘Maria’, ‘Carlos’, ‘Tatiane’, ‘Ana’]
Comeram maionese: [‘João’, ‘Carlos’, ‘Ana’]
Tiveram sintomas: [‘João’, ‘Ana’]
```

Observe que, ao exibir as listas, Python mostra colchetes e aspas indicando os elementos. Por exemplo, `Convidados: [‘João’, ‘Maria’, ...]`. Isso é útil para visualizar rapidamente o conteúdo de cada lista.

Essa etapa é puramente **sequencial**: uma instrução após a outra exibindo os valores atuais das listas, sem nenhuma tomada de decisão ou repetição – ou seja, um simples relato dos dados coletados.

Estrutura Condicional

Diferente da sequência simples, a **estrutura condicional** (também conhecida como estrutura de decisão) permite que o algoritmo tome **rumos diferentes** de acordo com alguma condição. Em Python, usamos principalmente a instrução `if` (se) para implementar decisões, possivelmente acompanhada de `else` (senão) e `elif` (else -if, ou seja, um “senão, se...”).

Python - input

```
caso = “Carlos”

if caso in teve_sintomas:  # se “Carlos” está na lista de quem teve sintomas
    print(caso, “teve sintomas.”)
else:
    print(caso, “não teve sintomas.”)
```

Nesse código, `caso` é a variável que armazena o nome que queremos verificar (iniciamos com `“Carlos”` como exemplo). A condição `caso in teve_sintomas` verifica se o valor em caso está presente na lista `teve_sintomas`. Se esta condição for verdadeira (`True`), o programa executa o bloco logo abaixo (depois dos dois-pontos), imprimindo “`<nome> teve sintomas.`”. Se a condição for falsa (`False`), o programa salta para o bloco sob else: e executa aquela parte, imprimindo “`<nome> não teve sintomas.`”.

Fique atento!

Observe que as linhas com o `print(...)` estão ligeiramente recuadas para a direita. Esse recuo é chamado de **indentação**, e, em Python, **ele indica que aquele comando faz parte da estrutura anterior** (nesse caso, do `if` ou do `else`).

A **indentação é obrigatória em Python**. Se ela estiver ausente ou malfeita, o código não funcionará.

No nosso exemplo, a variável caso vale `"Carlos"`. Sabendo que, na lista `teve_sintomas = ["João", "Ana"]`, o nome `"Carlos"` **não** está presente, então a condição `caso in teve_sintomas` será `False`. Portanto, o programa segue para o bloco do `else` e a saída será:

Python - output

```
Carlos não teve sintomas.
```

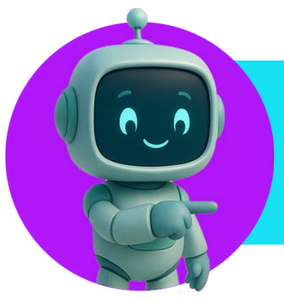
Se trocássemos `caso = "Ana"`, por exemplo, então `Ana in teve_sintomas` seria `True` (pois `"Ana"` está na lista), e o programa imprimiria `"Ana teve sintomas."` em vez da mensagem do else.

Essa estrutura condicional simples nos permite tratar diferentemente quem teve sintomas de quem não teve, uma lógica muito útil em análises. Por exemplo, poderíamos contar quantas pessoas tiveram sintomas (casos) e quantas não tiveram (controles) separadamente, ou tomar ações específicas para cada grupo.

Estrutura de Repetição (*Loop*)

Por fim, temos a estrutura de **repetição**, também chamada de *loop* (laço, em português). Ela permite executar um conjunto de instruções várias vezes, até que uma condição seja alcançada ou para cada elemento de um conjunto de dados. Em Python, os principais comandos de repetição são `for` e `while`.

No nosso exemplo, queremos **iterar** por todos os convidados e verificar, para cada um, se comeu maionese e se teve sintomas.



Iterar é percorrer, passo a passo, cada item de uma sequência (como uma lista) e aplicar uma ação ou verificação em cada elemento.

Podemos, por exemplo, gerar automaticamente frases que combinam as informações das três listas que temos (convidados, quem comeu maionese, quem teve sintomas). Assim, para cada convidado, o programa vai dizer se ele comeu ou não a maionese e se teve ou não sintomas. Vamos implementar isso com um loop `for` :

Python - input

```
for pessoa in convidados:  # para cada pessoa na lista de convidados
    if pessoa in comeu_maionese and pessoa in teve_sintomas:
        print(pessoa, "comeu maionese e teve sintomas.")
    elif pessoa in comeu_maionese and pessoa not in teve_sintomas:
        print(pessoa, "comeu maionese, mas não teve sintomas.")
    elif pessoa not in comeu_maionese and pessoa in teve_sintomas:
        print(pessoa, "não comeu maionese, mas teve sintomas.")
    else:
        print(pessoa, "não comeu maionese e não teve sintomas.")
```

Vale lembrar!

Observe que, assim como na estrutura condicional anterior, usamos a indentação (recuo) para indicar **o que pertence** ao `for` e ao `if` dentro dele.

Vamos analisar esse código por partes.

- `for pessoa in convidados:` irá **percorrer** a lista `convidados`, fazendo com que a variável `pessoa` assuma, a cada iteração do *loop*, o valor de um dos convidados. Assim, no primeiro ciclo `pessoa = "João"`, no segundo `pessoa = "Maria"`, e assim por diante, até o último nome da lista.
- Dentro do *loop*, usamos estruturas condicionais aninhadas (`if/elif/else`) para verificar a situação de cada `pessoa`:
  - se a `pessoa` **comeu maionese e teve sintomas**, imprimimos "`pessoa comeu maionese e teve sintomas.`";
  - senão, se a `pessoa` comeu maionese e **não** teve sintomas, imprimimos "`pessoa comeu maionese, mas não teve sintomas.`";
  - senão, se a pessoa **não** comeu maionese, **mas** teve sintomas, imprimimos "`pessoa não comeu maionese, mas teve sintomas.`" (esse caso em nosso exemplo talvez não ocorra, mas incluímos por completude);
  - caso contrário (**não** comeu maionese e **não** teve sintomas), imprimimos "`pessoa não comeu maionese e não teve sintomas.`".

FIQUE ATENTO!

Neste capítulo, focaremos especialmente no pré-processamento de dados, que é uma etapa crítica para garantir que os algoritmos aprendam a partir de dados de qualidade.

Ao executar o *loop* acima com nossas listas de exemplo, o programa produzirá as seguintes saídas, linha a linha para cada convidado:

Python - output

```
João comeu maionese e teve sintomas.

Maria não comeu maionese e não teve sintomas.

Carlos comeu maionese, mas não teve sintomas.

Tatiane não comeu maionese e não teve sintomas.

Ana comeu maionese e teve sintomas.
```

Vamos conferir com as listas:

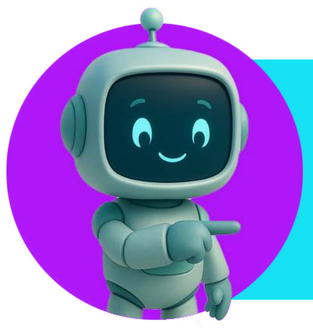
- **João** está em `comeu_maionese` e em `teve_sintomas` ➔ mensagem corresponde ao primeiro caso (comeu e teve);
- **Maria** não está em `comeu_maionese` nem em `teve_sintomas` ➔ entra no caso final (não comeu e não teve);
- **Carlos** está em `comeu_maionese` mas não em `teve_sintomas` ➔ segundo caso (comeu e não teve);
- **Tatiane** não está em `comeu_maionese` nem em `teve_sintomas` ➔ caso final (não comeu e *não* teve);
- **Ana** está em `comeu_maionese` e em `teve_sintomas` ➔ primeiro caso (comeu e teve).

O uso do *loop* nos permitiu automatizar a análise de cada convidado sem repetir manualmente o código para cada nome – bastou definir o padrão dentro do `for` que o Python repetiu para cada item da lista. Essa combinação de estruturas (repetição + condicional) é extremamente poderosa: aqui fizemos uma simples análise de associação entre exposição (comer maionese) e adoecimento (sintomas) de forma automatizada. Em contextos reais de Vigilância em Saúde, esse padrão se repete, por exemplo, ao verificar listas de pacientes e testes diagnósticos, listas de vacinados e ocorrência de doença etc.

Conclusão

Nesta aula, introduzimos conceitos fundamentais de programação em Python voltados para profissionais de saúde, incluindo variáveis, tipos de dados, operações aritméticas, operadores lógicos e estruturas básicas (sequência, decisão e repetição).

Programar nada mais é do que uma extensão do raciocínio lógico que os profissionais de vigilância já aplicam em investigações epidemiológicas. Com Python, conseguimos automatizar tarefas repetitivas, ganhar rapidez nas análises e explorar conjuntos de dados de forma mais eficaz.



Nas próximas aulas, continuaremos aprofundando nossas habilidades em programação, explorando novas estruturas e aplicando-as a problemas do dia a dia em saúde coletiva. Bom estudo!

Referências

McKINNEY, Wes. **Python para análise de dados**: tratamento de dados com pandas, NumPy e IPython. 2. ed. Sebastopol: O'Reilly Media, 2018.

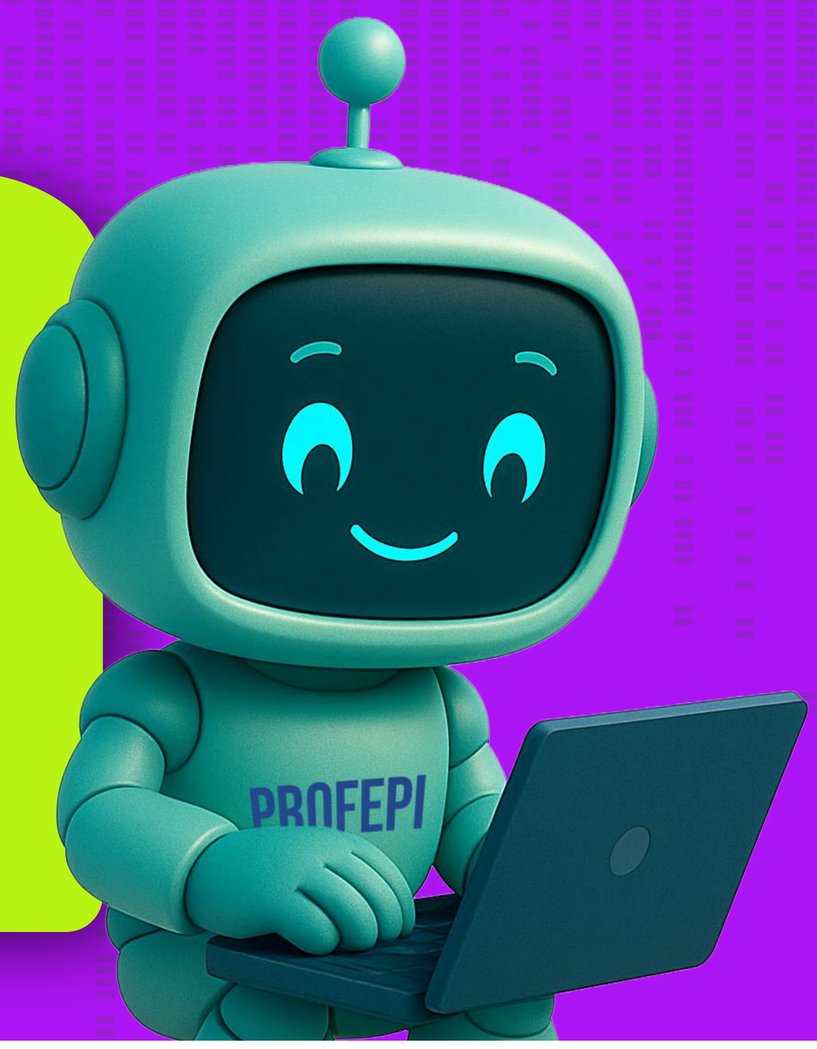
MENEZES, Ivan. **Algoritmos**: lógica para desenvolvimento de programação de computadores. São Paulo: Cengage Learning, 2016.

PYTHON SOFTWARE FOUNDATION. **Documentação Oficial do Python 3**. Disponível em: https://docs.python.org/pt-br/3/

VANDERPLAS, Jake. **Python para ciência de dados**: fundamentos da análise de dados com Python. 1. ed. Sebastopol: O'Reilly Media, 2017.

capítulo 3

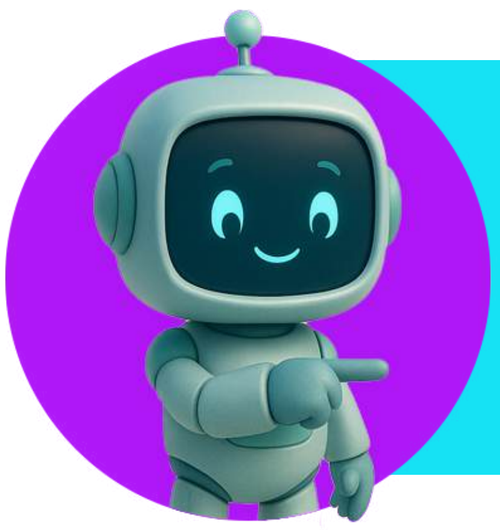
MÓDULO 1



Pré-processamento de dados em inteligência artificial para a saúde

1. Introdução

O pré-processamento de dados é uma etapa essencial em qualquer projeto de Inteligência Artificial (IA), especialmente no contexto da saúde, no qual decisões clínicas e de políticas públicas podem ser diretamente influenciadas pela qualidade das análises. Dados brutos, se não forem adequadamente tratados, podem comprometer gravemente a performance preditiva dos modelos, gerando riscos para a prática clínica e para a Vigilância em Saúde.



Segundo Wichmann, Fernandes e Chiavegatto Filho (2023), uma das principais causas de baixa performance de algoritmos na saúde é justamente a utilização de dados mal preparados ou inadequados para o contexto local, problema que é potencializado quando modelos desenvolvidos em países de alta renda são aplicados diretamente em países de baixa ou média renda, como o Brasil.

Os principais motivos técnicos que frequentemente levam a um desempenho preditivo abaixo do esperado incluem: extrapolação inadequada dos resultados (aplicar um modelo em um contexto diferente daquele em que foi desenvolvido); pré-processamento insuficiente ou inadequado dos dados; ocorrência de sobreajuste do modelo (*overfitting*); e validação inadequada da qualidade dos algoritmos.

FIQUE ATENTO!

Neste capítulo, focaremos especialmente no pré-processamento de dados, que é uma etapa crítica para garantir que os algoritmos aprendam a partir de dados de qualidade.

2. O que é o pré-processamento de dados?

O pré-processamento de dados consiste em um conjunto de técnicas e procedimentos aplicados aos dados brutos com o objetivo de organizá-los, limpá-los e estruturá-los para que possam ser utilizados de maneira eficaz em análises estatísticas ou em modelos de aprendizado de máquina. Como explicam Kuhn e Johnson (2013), essa etapa é fundamental para garantir que os algoritmos possam "compreender" os dados, extraindo padrões úteis e evitando vieses ou erros.

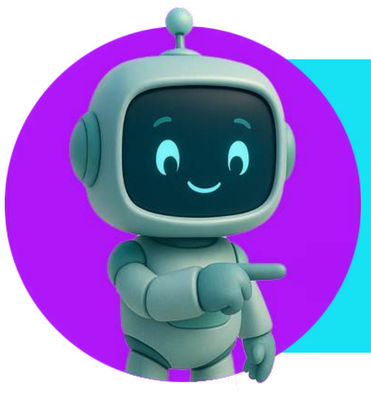
De forma geral, o pré-processamento envolve atividades como:

- organização e padronização dos dados;
- correção de erros e inconsistências;
- tratamento de valores ausentes;
- transformação e integração de bases de dados.

Essa etapa é indispensável para assegurar que os modelos consigam aprender de forma adequada e gerar predições confiáveis, especialmente na área da saúde, em que a qualidade da informação é determinante para decisões clínicas e epidemiológicas.

Eric Topol (2019) ilustra bem essa importância ao comparar o pré-processamento de dados com o preparo de amostras laboratoriais: não importa o quão sofisticado seja o equipamento ou o profissional, se a coleta e o armazenamento da amostra forem inadequados, o resultado do exame será inválido. Da mesma forma, dados desorganizados, incompletos ou inconsistentes comprometem as análises e os modelos preditivos.

Além disso, segundo Hastie, Tibshirani e Friedman (2009), é comum que o pré-processamento consuma até 80% do tempo total de um projeto de ciência de dados, dado o esforço necessário para transformar dados brutos em informação útil e estruturada. Na saúde, esse percentual pode ser ainda maior, considerando a complexidade e a fragmentação dos dados clínicos e de vigilância em saúde.



O pré-processamento é uma etapa fundamental que precede a construção de modelos, e inclui tarefas como limpeza de dados, seleção e transformação de variáveis, codificação de categorias, tratamento de valores ausentes, entre outras.

Uma analogia prática: imagine um laboratório realizando exames de sangue para diagnóstico. Se as amostras de sangue forem coletadas de forma inadequada, por exemplo, armazenadas na temperatura errada ou misturadas indevidamente, os resultados dos exames serão inválidos, não importando a qualidade dos equipamentos ou dos profissionais que farão a análise. Da mesma forma, se os dados estão incompletos, despadronizados ou cheios de erros, os modelos de IA "diagnosticarão" incorretamente, mesmo que o algoritmo em si seja avançado. Em outras palavras, se entra lixo, sai lixo. Essa expressão resume bem a importância do pré-processamento: alimentar um modelo com "lixo" (dados de baixa qualidade) resultará inevitavelmente em "lixo" na saída (predições pouco confiáveis).

3. Por que o pré-processamento é indispensável em saúde?

O principal motivo que torna o pré-processamento indispensável é o fato de que os algoritmos de IA aprendem padrões diretamente a partir dos dados disponíveis. Se esses dados estiverem incorretos, incompletos ou mal estruturados, o modelo inevitavelmente aprenderá padrões distorcidos, fenômeno conhecido como "garbage in, garbage out", conforme destacado por Hastie, Tibshirani e Friedman (2009).

No contexto da saúde, esse problema assume proporções ainda mais relevantes. Bases de dados epidemiológicas e clínicas frequentemente apresentam:

- **valores ausentes**, decorrentes de falhas de registro;
- **registros duplicados**, que podem distorcer estatísticas;
- **dados inconsistentes**, com erros de digitação ou de formato;
- **informações desbalanceadas** entre diferentes populações, o que pode introduzir viés nos modelos.

Eric Topol (2019) enfatiza que, quando essas falhas não são corrigidas, os riscos são grandes: os modelos podem gerar predições incorretas, enviesadas ou clinicamente inaceitáveis, levando a diagnósticos errôneos, decisões inadequadas e potencial aumento das desigualdades em saúde.

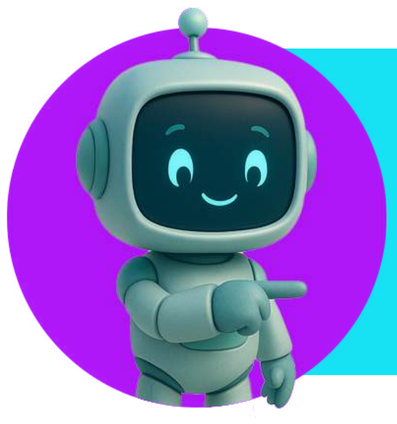
Além disso, um editorial recente da revista The Lancet (2023) alerta que a maioria dos dados de saúde usados em modelos de IA é proveniente de países de alta renda, como Estados Unidos e países da Europa. Isso significa que muitos algoritmos refletem realidades socioeconômicas, genéticas e estruturais muito diferentes das encontradas em países de baixa e média renda, como o Brasil. Sem um pré-processamento cuidadoso e sem a adaptação aos contextos locais, o uso desses modelos pode agravar desigualdades históricas e comprometer a segurança e a eficácia das decisões em saúde.

Esse problema foi evidenciado no estudo conduzido por Wichmann, Fernandes e Chiavegatto Filho (2023), que analisaram o desempenho de modelos de IA para predizer óbitos por covid-19 em uma coorte multicêntrica com dados de dezoito hospitais nas cinco regiões do Brasil. O estudo mostrou que os melhores resultados foram obtidos quando os algoritmos foram treinados com dados específicos de cada hospital. Por outro lado, o uso de dados agregados de diferentes regiões resultou em pior desempenho preditivo, reforçando a importância de um pré-processamento criterioso e adaptado ao contexto de aplicação.

Portanto, o pré-processamento de dados não é apenas uma questão técnica, mas também ética e metodológica. Ele assegura que os modelos sejam seguros, justos e efetivamente úteis na prática clínica e na saúde pública.

4. Seleção de Variáveis Relevantes

A seleção de variáveis é uma etapa inicial fundamental. É necessário identificar quais variáveis possuem plausibilidade clínica ou epidemiológica para o desfecho de interesse. Como destacam Hastie, Tibshirani e Friedman (2009), incluir variáveis apenas por apresentarem correlação com o desfecho, sem relação causal plausível, pode gerar modelos instáveis e predições enganosas.



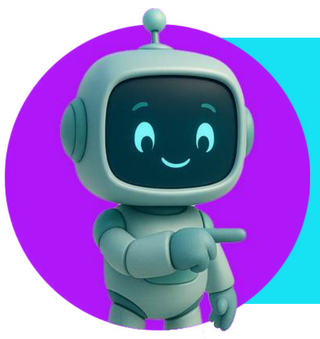
Por exemplo, em modelos de predição de surtos de dengue, é razoável incluir variáveis como temperatura, chuvas e histórico de casos notificados. No entanto, incluir variáveis sem relação causal, como o número de cinemas no bairro, mesmo que apresentem correlação estatística, pode levar a modelos absurdos e sem utilidade prática.

Portanto, **selecionar variáveis é também filtrar o que não faz sentido**. Devemos eliminar variáveis claramente irrelevantes ou enganosas, mesmo que, por acaso, tenham correlação alta com o desfecho nos dados históricos, pois essa relação provavelmente não se sustentará em novos dados, levando a erros de predição. Também é recomendável envolver especialistas no assunto (epidemiologistas, biólogos, médicos, enfermeiros etc.) para validar se as variáveis escolhidas têm plausibilidade biológica ou ambiental.

Uma prática comum é fazer uma revisão da literatura para ver quais fatores são apontados como importantes para prever aquele desfecho. Além disso, análises exploratórias de correlação podem ajudar a identificar variáveis potencialmente redundantes.

5. Limpeza de dados

Uma vez definidas as variáveis que vamos usar, entramos na etapa de limpeza de dados propriamente dita. **Limpar os dados significa detectar e corrigir problemas nos dados brutos, garantindo que o conjunto final seja confiável para alimentar o modelo**. Essa é geralmente a primeira etapa prática do pré-processamento e inclui tratar valores ausentes, remover duplicatas, identificar *outliers*, corrigir inconsistências e verificar a integridade geral do *dataset*. A limpeza de dados é considerada uma das etapas mais fundamentais e trabalhosas do pré-processamento, especialmente em projetos de saúde. Segundo Kuhn e Johnson (2013), ela representa o alicerce para qualquer análise preditiva bem-sucedida, pois garante que o algoritmo seja treinado com informações consistentes, válidas e confiáveis. É crucial porque melhora o desempenho dos modelos, evita vieses e conclusões equivocadas, e é especialmente importante nas áreas de saúde e meio ambiente, nas quais erros podem prejudicar decisões públicas.



Pense na limpeza de dados como “varrer” e “organizar” uma casa antes de receber uma visita importante. Se os dados estão bagunçados, o algoritmo (visita) não vai se sentir muito “à vontade” nem conseguir encontrar o que precisa.

A seguir, veremos os principais aspectos da limpeza de dados.

5.1 Tratamento de valores ausentes

Em qualquer base de dados na área da saúde, seja de vigilância em saúde, registros clínicos ou pesquisas populacionais, é praticamente inevitável encontrar informações que não foram preenchidas. Esses “buracos” nos registros são chamados de valores ausentes, ou missing data.

Contudo, nem sempre a ausência de um dado é simplesmente um problema ou um erro de coleta. Na verdade, a ausência em si pode carregar um significado importante, dependendo do contexto.

Segundo Little e Rubin (2002), há três mecanismos principais que explicam por que um dado pode estar ausente:

1. **MCAR (Missing Completely At Random)** – quando a ausência ocorre de forma totalmente aleatória, sem relação com outras informações;
2. **MAR (Missing At Random)** – quando a ausência depende de outras variáveis observadas. Ou seja, o dado pode faltar mais em certos grupos ou condições específicas;
3. **MNAR (Missing Not At Random)** – quando a ausência está diretamente relacionada ao próprio valor que está faltando.

FIQUE ATENTO!

Por isso, o primeiro passo, ao lidar com valores ausentes, é investigar a causa dessa ausência. Isso é crucial, pois a estratégia de tratamento adequada depende desse diagnóstico inicial.

Aqui entra uma diferença importante entre análises preditivas e estudos estatísticos tradicionais: **nos modelos preditivos, a ausência de dados pode ter valor preditivo por si só**. Como destacam Kuhn e Johnson (2013), um campo não preenchido pode indicar algo relevante, como uma condição clínica específica ou um comportamento de risco.

Por exemplo, imagine uma pesquisa que pergunta: “O que você jantou ontem?”. Em um grupo de idosos com declínio cognitivo, é possível que muitos deixem a resposta em branco. Nesse caso, o dado ausente não é apenas uma lacuna, ele pode ser um forte indicativo de problemas de memória ou de saúde cognitiva.

Por isso, uma dica prática recomendada por Kuhn e Johnson (2013) é criar uma categoria especial para valores ausentes em variáveis categóricas, como “Não informado”. Já para variáveis numéricas, além de realizar uma imputação (preenchimento do valor), é interessante incluir uma variável indicadora adicional, que mostre ao modelo que aquele campo estava originalmente ausente. Assim, o algoritmo pode aprender também a partir da ausência.

5.2 Principais estratégias para tratar valores ausentes

Depois de entender a origem dos dados ausentes, o próximo passo é decidir como lidar com eles. As principais abordagens incluem as etapas a seguir.

1. Exclusão de observações incompletas

Uma das soluções mais simples é remover registros ou variáveis com muitos dados faltantes. Essa abordagem só é recomendada quando:

- a base de dados é muito grande;
- a ausência dos dados é aleatória (MCAR);
- a exclusão não comprometerá a representatividade da amostra.

Gelman e Hill (2007) alertam que essa técnica pode causar perda de informação e viés caso não haja uma análise prévia cuidadosa do mecanismo de ausência.

2. Imputação simples (média, mediana ou moda)

Esta é uma estratégia bastante usada para preencher dados ausentes de forma rápida:

- média ou mediana para variáveis numéricas;
- moda (categoria mais frequente) para variáveis categóricas.

Segundo Gelman e Hill (2007), embora simples, essa abordagem pode mascarar a variabilidade real dos dados e reduzir a acurácia do modelo, pois todos os valores ausentes serão substituídos pelo mesmo valor. Ainda assim, é útil quando o percentual de missing é pequeno e a variável é relativamente estável.

3. Interpolação e extrapolação

Muito utilizada em dados temporais ou ordenados, a interpolação estima um valor ausente com base nos valores vizinhos. Por exemplo, se um sensor falhou no registro da temperatura no dia 10, podemos estimar esse valor considerando os registros dos dias 9 e 11.

Já a extrapolação tenta prever valores fora do intervalo observado, com base em tendências conhecidas ou modelos auxiliares. Gelman e Hill (2007) ressaltam que a extrapolação deve ser aplicada com cautela, pois traz um risco maior de erro.

4. Imputação com modelos preditivos (machine learning)

Aqui, o problema dos dados ausentes é tratado como uma tarefa de predição. Ou seja, treinamos um modelo para "adivinhar" o valor ausente, utilizando as demais variáveis como entrada.

Van Buuren (2018) explora em profundidade essa abordagem, que é considerada uma das mais avançadas e eficazes, especialmente quando combinada com técnicas de imputação múltipla. No entanto, ele adverte que é essencial evitar o uso de variáveis que não estariam disponíveis no momento da coleta real, para não gerar *data leakage* (vazamento de dados).

Além disso, imputações baseadas em modelos precisam ser validadas com muito rigor, pois introduzem incertezas e podem distorcer os resultados se não forem aplicadas corretamente.

O que levar em conta na escolha da técnica?

Não existe uma resposta única. A melhor abordagem depende:

- da quantidade de dados faltando;
- do motivo da ausência;
- do impacto no desempenho preditivo;
- da sensibilidade da variável no modelo.

Em muitos casos, uma combinação de técnicas será a solução mais equilibrada. Entretanto, o ponto mais importante é: nunca ignore os dados ausentes. Sempre registre quantos são, onde estão e o que foi feito com eles. A transparência é essencial, principalmente em projetos que envolvem saúde pública e decisões clínicas.

6. Remoção de Duplicatas

Quando falamos de bases de dados em saúde, um problema que surge com frequência é a duplicação de registros. Em muitos casos, o mesmo paciente aparece mais de uma vez no banco, seja por erro de digitação, seja por falhas na integração de sistemas diferentes.

FIQUE ATENTO!

Registros duplicados podem **inflar de forma artificial** o número de casos de uma doença, distorcer indicadores epidemiológicos e comprometer completamente a performance dos modelos preditivos

Como reforçam Kuhn e Johnson (2013), remover duplicatas não é apenas uma questão de organização, é uma etapa crucial para garantir a integridade do modelo.

O que são registros duplicados na prática?

São registros que representam, na verdade, a mesma pessoa ou evento, mas que aparecem mais de uma vez na base de dados. Isso pode acontecer, por exemplo:

- quando o mesmo paciente faz o mesmo exame em diferentes unidades de saúde;
- quando bases distintas são integradas, mas sem uma chave de identificação robusta;
- por erro de digitação no nome ou nos documentos;
- por falhas nos sistemas de notificação.

Quais os riscos de não tratar duplicatas?

Os riscos vão muito além de números inflados. Manter registros duplicados pode:

- superestimar taxas de incidência e prevalência;
- gerar sobrecontagem de hospitalizações ou óbitos;
- introduzir viés nos modelos de predição, já que o mesmo indivíduo pode aparecer várias vezes como se fossem casos independentes;
- comprometer decisões de saúde pública e políticas de alocação de recursos.

Além disso, como apontam Gelman e Hill (2007), duplicatas podem distorcer a variabilidade dos dados e, consequentemente, comprometer a performance de modelos estatísticos e de *machine learning*.

Como identificar e remover duplicatas?

O processo de identificação de duplicatas deve ser feito com atenção, especialmente na área da saúde, em que diferentes registros podem parecer similares, mas representar situações distintas (por exemplo, casos de violência de repetição ou reingressos hospitalares).

A seguir estão os principais passos recomendados.

1

Definir critérios claros para o que é considerado duplicata

Em geral, considera-se duplicata um registro que tenha:

- mesmo nome completo ou código de identificação;
- mesma data de nascimento ou outro identificador secundário;
- mesmo evento (por exemplo, mesma data de notificação e mesmo agravo).

Porém, em áreas como vigilância de violência, um mesmo paciente pode ter notificações diferentes em datas distintas. Nesses casos, é preciso tomar cuidado para não excluir registros legítimos.

2

Normalizar os dados antes de comparar

Antes de comparar registros, é importante padronizar variáveis, como:

- nomes (padronização de maiúsculas, remoção de acentos, correção de grafias);
- datas (padronização de formatos);
- categorias (unificar termos como “Masculino” e “masc.”).

3 **Utilizar técnicas de comparação fonética e linkage probabilístico**

Quando a base de dados não tem um identificador único confiável (como CPF), pode-se recorrer a métodos de *linkage* probabilístico ou a algoritmos de comparação fonética, como o Soundex ou o Double Metaphone, que identificam registros similares mesmo com pequenas diferenças nos nomes.

4 **Validar manualmente os casos limítrofes**

Após a identificação automática, é recomendável revisar manualmente registros que estejam no limite da definição de duplicata, principalmente em bases sensíveis como as de saúde.

7. Outliers

***Outliers* são pontos fora do padrão esperado dos dados, valores anômalos que destoam fortemente da maioria**. No contexto de saúde, **podem ser resultados de erros** (como um erro de digitação que registrou a idade de um paciente como 523 anos, em vez de 52) **ou podem ser casos reais, mas raros** (por exemplo, um paciente com 105 anos de idade; é raro, mas pode ser real). *Outliers* importam porque podem distorcer análises estatísticas e modelagens: por exemplo, a média é bastante sensível a valores muito extremos, e uma regressão linear simples pode ser "puxada" na direção de um ponto fora da curva. Além disso, outliers de origem errônea (erros de medição ou digitação) introduzem *noise*, que não ajuda em nada o modelo.

Como lidar com outliers? Primeiro, é preciso identificá-los. Algumas técnicas comuns:

- **Métricas estatísticas:** calcular o Z-*score* (quantos desvios-padrão o ponto está longe da média) – valores com |Z| muito alto (ex.: > 3) podem ser candidatos a *outlier*. Ou usar o intervalo interquartil (IQR): calcula Q1 (25° percentil) e Q3 (75° percentil) e define limites como Q1 - 1.5IQR e Q3 + 1.5IQR; pontos fora desse intervalo são considerados *outliers*.
- **Métodos visuais:** plotar um gráfico de caixa (*boxplot*) é uma forma rápida de visualizar pontos que estão além dos "bigodes" do *boxplot* (que tipicamente representam 1.5*IQR). Um histograma ou *scatterplot* também ajuda a ver se há pontos isolados.
- **Modelos de detecção de anomalias:** em conjuntos complexos, algoritmos específicos (ex.: *isolation forest*, *autoencoders*) podem apontar observações anômalas.

FIQUE ATENTO!

Uma vez identificados, decide-se o tratamento. Nem todo *outlier* deve ser removido indiscriminadamente.

Deve-se investigar a causa: se for claramente um erro (ex.: um valor impossível ou unitário errado), pode ser corrigido se possível ou então excluído. Se o outlier for um caso real excepcional, então é uma decisão de negócio: se estamos interessados em modelar o caso típico, talvez excluamos para não atrapalhar; mas se casos raros também são relevantes (ex.: detecção de fraude, detecção de surtos), então devemos manter ou até dar atenção especial a esses outliers. **Em problemas de saúde pública, outliers podem representar comunidades com situações atípicas ou erros de notificação, cada caso exige análise contextual.**

Em resumo, *outliers* pedem cautela: identifique, tente explicar e então trate caso a caso. Nunca elimine dados só porque "parecem esquisitos", sem checar o porquê, pois você pode remover justamente um dado importante. Por outro lado, não tratar outliers errôneos pode prejudicar seriamente o modelo.

8. Inconsistências e Validação de Integridade

Além de valores ausentes, duplicatas e outliers, há uma categoria de problemas mais sutis que chamamos de inconsistências. Isso inclui:

- **Formatos diferentes para o mesmo tipo de dado:** por exemplo, datas registradas em formatos distintos ("14/05/1990" em alguns registros versus "1990-05-14" em outros); ou campos de texto em que às vezes "Sim" está escrito como "sim", "SIM" ou "S" – variações que precisam ser unificadas.
- **Unidades ou escalas diferentes:** por exemplo, um dataset em que o peso de pacientes às vezes está em quilogramas e às vezes em libras, sem indicação clara – isso precisa ser padronizado para uma unidade única.
- **Valores padronizados/categorizados de forma inconsistente:** nomes próprios com e sem acentuação, uso de abreviações não uniformes etc. Talvez seja necessário transformar tudo para letras minúsculas sem acento, por exemplo, para evitar que "São Paulo" e "Sao Paulo" sejam tratados como coisas distintas.
- **Dados desnormalizados:** às vezes, faixas etárias precisam ser agrupadas (ex.: idades numéricas convertidas para categorias "0-5", "6-12" etc., conforme necessidade de análise), ou valores contínuos transformados em categorias (ex.: renda transformada em "baixa, média, alta") – isso são transformações planejadas, mas é importante executar de forma consistente e documentada.

O processo de **validação de integridade** envolve checar regras de negócio e limites lógicos.

Exemplos: verificar se não existem pacientes com data de alta anterior à data de admissão (inconsistência temporal), ou se não há valores fisicamente impossíveis (temperatura corporal de 80°C, por exemplo). Ferramentas de **automação** ajudam a flagrar algumas dessas incoerências, mas muitas vezes é necessária uma **verificação manual** cuidadosa em amostras dos dados para garantir que tudo faz sentido.

Em termos práticos, consertar inconsistências pode envolver desde simples substituições (padronizar "M" e "F" para "Masculino" e "Feminino", por exemplo) até integrações mais complexas entre bases (que veremos adiante). O importante é buscar um *dataset* final **coeso**, em que cada campo siga um padrão único e todos os registros respeitem as regras lógicas esperadas.

Anota essa informação!

Às vezes, a etapa de limpeza inclui também **integração de dados** de múltiplas fontes. Por exemplo, podemos querer unir a base de mortalidade (SIM) com a base de notificação de violência (SINAN) para uma análise conjunta. Isso requer encontrar uma **chave de ligação** entre as bases, como nome, data de nascimento etc. e utilizar técnicas de *record linkage* determinístico ou probabilístico para combinar as informações. A integração, quando feita, deve vir seguida de uma checagem extra de inconsistências, pois a união de bases aumenta a chance de duplicatas ou conflitos de informação.

9. Categorização e codificação de variáveis categóricas: como traduzir informações qualitativas para modelos de IA?

Na vida real e nos dados de saúde, muitas informações não vêm na forma de números. Sexo, estado civil, tipo de doença, região de moradia, ocupação, todas essas são variáveis **categóricas**, ou seja, compostas por rótulos ou classes.

Por mais importantes que essas informações sejam, os algoritmos de machine learning não conseguem processá-las diretamente. Isso porque a maioria dos algoritmos funciona com números, eles realizam operações matemáticas, e letras ou palavras não podem ser multiplicadas ou somadas.

Portanto, transformar variáveis categóricas em números é uma etapa obrigatória no pré-processamento de dados. Contudo, essa transformação precisa ser feita com cuidado, pois, se for mal conduzida, pode causar erros ou vieses nos modelos preditivos.

Por que essa etapa é tão crítica na saúde?

Na área da saúde, as variáveis categóricas muitas vezes carregam significados clínicos importantes. Por exemplo, a variável "tipo de doença" pode ter categorias como:

- Diabetes Tipo 1;
- Diabetes Tipo 2;
- Hipertensão;
- Doença Renal Crônica.

A simples maneira como essas categorias são convertidas em números pode influenciar diretamente a forma como o modelo interpreta os dados. Como alertam Kuhn e Johnson (2013), uma codificação incorreta pode induzir o algoritmo a ver relações inexistentes entre as categorias, distorcendo completamente as predições.

Principais Métodos de Codificação

Aqui estão as técnicas mais utilizadas para codificar variáveis categóricas, explicadas de forma simples e com exemplos práticos.

1

Label Encoding (Codificação por Rótulo)

Nesta técnica, cada categoria é substituída por um número inteiro. Por exemplo, uma variável "sexo" poderia ser codificada assim:

- Feminino → 0
- Masculino → 1

Apesar de simples, essa abordagem tem um problema importante: ela cria uma ordem entre as categorias que não necessariamente existe. Nesse exemplo, o algoritmo pode interpretar que "masculino" (1) tem mais peso ou importância que "feminino" (0), o que não faz sentido.

Por isso, o *label encoding* só é recomendado quando existe uma ordem natural entre as categorias, como em variáveis de gravidade de sintomas (leve, moderado, grave), segundo Gelman e Hill (2007).

2 One-Hot Encoding (Codificação Binária por Categoria)

Este é o método mais utilizado em saúde pública e machine learning. Ele cria uma variável (ou coluna) para cada categoria e preenche com:

- 1 quando o registro pertence àquela categoria;
- 0 caso contrário.

Diabetes Tipo 1	Diabetes Tipo 2	Hipertensão	Doença Renal Crônica
1	0	0	0

Essa abordagem evita o problema de ordem artificial e é facilmente compreendida pelos algoritmos. No entanto, quando há muitas categorias, o *one-hot encoding* pode gerar um número muito grande de variáveis, o que dificulta a análise e pode causar o chamado "problema da maldição da dimensionalidade".

Para lidar com esse risco, Kuhn e Johnson (2013) recomendam a criação de **variáveis *dummies***, que seguem o mesmo princípio do *one-hot encoding*, mas deixam de fora a categoria mais frequente para servir como referência, evitando redundância e problemas de colinearidade nos modelos.

3 *Target Encoding* (Codificação por Desfecho)

Este é um método mais recente e avançado, que tem ganhado popularidade em projetos preditivos. Nele, cada categoria é substituída pelo valor médio do desfecho entre os indivíduos daquela categoria.

Por exemplo, suponha que estamos prevendo risco de hospitalização e temos a variável "região de moradia". No *target encoding*, o valor atribuído a cada região seria a taxa média de hospitalizações naquela região.

Essa técnica pode capturar padrões importantes de forma compacta, mas traz riscos de overfitting (o modelo aprende demais com o conjunto de treino e perde capacidade de generalização). Van Buuren (2018) ressalta que o t*arget encoding* deve ser usado com muito cuidado, sempre com validação rigorosa.

Exemplo prático: variável "Estado Civil"

Vamos imaginar que estamos analisando um banco de dados de saúde pública, e uma das variáveis é o "estado civil". Suas categorias são:

- Solteiro;
- Casado;
- Divorciado;
- Viúvo.

Veja como as três técnicas de codificação lidariam com essa variável:

Técnica	Como funciona	Possível problema
Label Encoding	Solteiro = 0, Casado = 1, Divorciado = 2, Viúvo = 3	O modelo pode "achar" que Viúvo tem peso maior que Solteiro
One-Hot Encoding	Cria 4 variáveis binárias (1 para a categoria, 0 para as outras)	Aumenta o número de variáveis no modelo
Target Encoding	Substitui cada categoria pela taxa média de hospitalização naquele grupo	Pode causar overfitting se não for bem validado

Como escolher a melhor técnica?

A escolha depende de vários fatores:

- **quantidade de categorias:** muitas categorias favorecem técnicas mais compactas, como *target encoding* (com cautela);
- **risco de ordem artificial:** sempre que não houver ordem real entre as categorias, evite o *label encoding*;
- **objetivo da análise:** modelos interpretáveis podem se beneficiar mais do *one-hot encoding*, enquanto modelos complexos podem usar técnicas mais avançadas;
- **tamanho da base de dados:** bases grandes toleram melhor o aumento de variáveis gerado pelo *one-hot encoding*.

10. Padronização de variáveis numéricas

A padronização de variáveis numéricas, também conhecida como *standardization* ou *z-score normalization*, constitui uma das etapas mais fundamentais do pré-processamento de dados em projetos de modelagem preditiva. Sua aplicação visa corrigir os efeitos indesejados das diferentes escalas das variáveis, assegurando que todas contribuam de maneira equânime para o ajuste dos modelos, sobretudo daqueles sensíveis à magnitude dos dados.

Conforme explicitam Kuhn e Johnson (2013), a ausência de padronização pode conduzir a resultados distorcidos, com sobrevalorização de variáveis cujo domínio numérico é naturalmente mais amplo, em detrimento de variáveis igualmente ou mais relevantes do ponto de vista preditivo, mas expressas em escalas reduzidas.

Definição formal e fundamentação matemática

A padronização refere-se à transformação de uma variável contínua de modo que sua distribuição seja centralizada na média zero e possua desvio-padrão unitário. Trata-se, portanto, de um processo de linearização que preserva a forma da distribuição, mas altera sua escala e localização.

Matematicamente, a transformação é realizada a partir da seguinte equação:

$$z_i = \frac{x_i - \mu}{\sigma}$$

Onde:

- z_i representa o valor padronizado da i-ésima observação;
- x_i corresponde ao valor original da variável;
- μ é a média aritimética da variável x;
- σ é o desvio padrão da variável x.

Essa trasnformação resulta em uma distribuição com média zero ($E[z] = 0$) e variância unitária ($Var\ [z] = 1$)

De acordo com Gelman e Hill (2007), a padronização é particularmente recomendada para variáveis que não compartilham a mesma unidade de medida, pois impede que o modelo atribua importância desproporcional a variáveis cujo domínio numérico é maior apenas por questões de escala.

Relevância da padronização em modelos preditivos

A importância da padronização reside no fato de que diversos algoritmos de aprendizado supervisionado operam sob princípios matemáticos que consideram a magnitude das variáveis durante o processo de otimização. Destacam-se, nesse contexto, os seguintes algoritmos:

- **regressão linear e regressão logística com regularização (Lasso, Ridge, Elastic Net):** a penalização dos coeficientes durante o ajuste do modelo depende diretamente da escala das variáveis. Sem padronização, variáveis com maior variância tendem a sofrer menor penalização, influenciando indevidamente o modelo;
- **k-Nearest Neighbors (k-NN) e Support Vector Machines (SVM):** ambos utilizam distâncias euclidianas para determinar similaridades entre observações. Variáveis com grandes magnitudes numéricas podem dominar essas medidas de distância, distorcendo a definição de vizinhança ou margem ótima;
- **redes neurais artificiais:** a padronização auxilia na convergência dos algoritmos de otimização, como o gradiente descendente, reduzindo o risco de trajetórias irregulares no espaço de parâmetros.

Por outro lado, algoritmos baseados em árvores de decisão, como Random Forests e Gradient Boosting Machines, não são sensíveis à escala das variáveis, uma vez que as divisões são realizadas por meio de partições binárias e não dependem de operações algébricas sobre os valores numéricos.

Aplicações no contexto da saúde

Bases de dados na área da saúde frequentemente apresentam variáveis com escalas altamente heterogêneas. Considere, por exemplo, um modelo preditivo que utiliza as seguintes variáveis:

- Idade (anos), variando entre 0 e 100;
- Pressão arterial sistólica (mmHg), variando entre 80 e 200;
- Glicemia (mg/dL), variando entre 60 e 400;
- Índice de Massa Corporal (kg/m²), variando entre 15 e 60.

Sem padronização, variáveis como glicemia e pressão arterial, que naturalmente assumem valores numéricos mais elevados, tenderiam a dominar o processo de ajuste do modelo, ainda que não apresentem maior relevância clínica ou epidemiológica para o desfecho de interesse.

Van Buuren (2018) salienta que a padronização, ao permitir a comparação entre efeitos padronizados de diferentes variáveis, facilita a interpretação dos coeficientes de modelos lineares e a seleção criteriosa de preditores em análises multivariadas, além de contribuir para a estabilidade dos algoritmos em aplicações com múltiplas variáveis correlacionadas.

Considerações metodológicas e boas práticas

Para assegurar a validade dos resultados, a padronização deve ser implementada exclusivamente sobre o conjunto de treinamento, com posterior aplicação dos parâmetros (μ e σ) obtidos ao conjunto de teste. Tal procedimento visa prevenir o fenômeno conhecido como *data leakage*, que ocorre quando informações do conjunto de teste influenciam o ajuste do modelo, comprometendo a generalização.

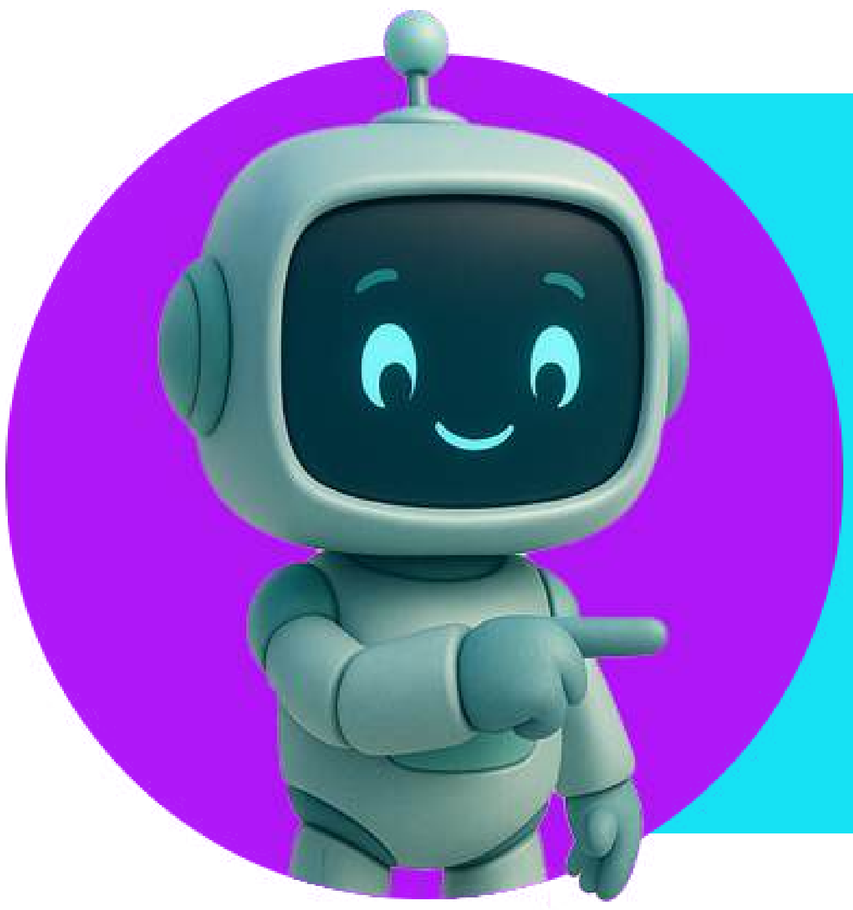
Adicionalmente, é imprescindível avaliar a distribuição original da variável antes da padronização. Em casos de distribuição altamente assimétrica ou com presença de outliers extremos, pode ser necessário aplicar transformações adicionais, como logaritmo ou Box-Cox, para reduzir a influência de valores aberrantes, conforme sugerem Kuhn e Johnson (2013).

Alternativas à padronização

Embora a padronização seja amplamente recomendada, existem outras técnicas de escalonamento, sendo a mais comum a normalização min-max, que reescala os dados para um intervalo predeterminado, usualmente entre 0 e 1:

$$x'_i = \frac{x_i - min(x)}{max(x) - min(x)}$$

Essa técnica é simples e intuitiva, mas apresenta sensibilidade elevada a *outliers*, razão pela qual seu uso deve ser restrito a situações em que as variáveis possuam distribuição aproximadamente simétrica e ausência de valores extremos.



A padronização de variáveis numéricas constitui uma etapa indispensável no pré-processamento de dados destinados a modelos preditivos em saúde, especialmente quando se empregam algoritmos sensíveis à escala. Sua correta aplicação assegura:

- equidade na contribuição das variáveis durante o ajuste do modelo;
- melhor desempenho computacional;
- interpretação adequada dos resultados.

Por essas razões, sua implementação deve ser rigorosamente planejada e documentada, integrando-se como etapa obrigatória nos fluxos de trabalho de ciência de dados aplicados à saúde.

11. Colinearidade

A presença de colinearidade entre variáveis explicativas é um dos desafios mais recorrentes em projetos de modelagem preditiva, particularmente em bases de dados de saúde, nas quais as variáveis frequentemente apresentam correlação intrínseca devido à interdependência biológica, clínica ou social entre os fenômenos analisados.

Em termos estatísticos, **a colinearidade refere-se à situação em que duas ou mais variáveis explicativas possuem elevado grau de correlação entre si, de modo que uma variável pode ser, em grande medida, prevista a partir das demais**. Quando esse fenômeno ocorre em grau extremo, utiliza-se o termo multicolinearidade (Kuhn & Johnson, 2013).

1. Fundamentos Estatísticos da Colinearidade

A colinearidade pode ser formalmente quantificada por meio do coeficiente de correlação linear de Pearson (r), que mede a força da associação linear entre duas variáveis contínuas. **Valores próximos a |r|=1 indicam alta correlação**.

Entretanto, para situações com múltiplas variáveis, o diagnóstico mais robusto é feito por meio do Fator de Inflação da Variância (Variance Inflation Factor – VIF), que estima quanto a variância de um coeficiente de regressão aumenta em razão da colinearidade. De acordo com Gelman e Hill (2007), valores de VIF acima de 5 ou 10 sugerem colinearidade significativa, que pode comprometer a estabilidade do modelo.

2. Impactos da Colinearidade nos Modelos

A presença de variáveis colineares acarreta múltiplos efeitos adversos sobre os modelos preditivos, entre os quais se destacam:

- **instabilidade dos coeficientes estimados:** pequenas variações nos dados podem gerar grandes flutuações nos coeficientes, dificultando sua interpretação e confiabilidade;
- **perda de precisão:** a variância dos coeficientes aumenta, o que reduz a capacidade do modelo de estimar efeitos individuais com acurácia;
- **redundância informacional:** a inclusão de variáveis altamente correlacionadas não agrega informação nova, podendo aumentar desnecessariamente a complexidade do modelo;
- **risco de *overfitting*:** modelos excessivamente ajustados podem capturar ruídos específicos do conjunto de treinamento, comprometendo sua capacidade de generalização.

Van Buuren (2018) adverte que, em contextos de saúde, esses problemas podem comprometer a identificação de fatores de risco relevantes e afetar decisões clínicas baseadas nos modelos.

3. Exemplos de Colinearidade em Saúde

Na prática, a colinearidade é particularmente frequente em bases de saúde. Exemplos típicos incluem:

- **idade e tempo de exposição a fatores de risco:** frequentemente, quanto maior a idade, maior o tempo acumulado de exposição a comportamentos como tabagismo ou sedentarismo;
- **peso e índice de massa corporal (IMC):** como o IMC é calculado a partir do peso e da altura, incluir simultaneamente essas variáveis sem ajuste pode gerar alta colinearidade;
- **glicemia em jejum e hemoglobina glicada (HbA1c):** ambas são medidas relacionadas ao controle glicêmico, com forte correlação entre si em populações diabéticas.

4. Estratégias para Mitigação da Colinearidade

O tratamento adequado da colinearidade depende do objetivo analítico e da estrutura dos dados. Entre as principais estratégias, destacam-se:

a) Remoção de Variáveis Redundantes

Consiste na exclusão de uma ou mais variáveis altamente correlacionadas, preservando-se aquelas com maior relevância clínica ou interpretabilidade.

Kuhn e Johnson (2013) sugerem adotar limiares de correlação ou VIF para guiar essa decisão, mantendo o equilíbrio entre simplicidade e capacidade preditiva.

b) Combinação de Variáveis (Criação de Índices Compostos)

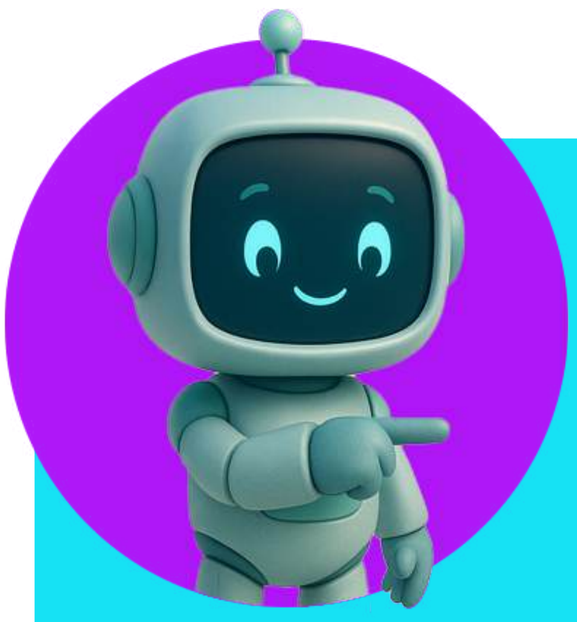
Em casos específicos, pode-se combinar variáveis correlacionadas em um único indicador sintético. Por exemplo, a soma de fatores de risco cardiovascular pode gerar um escore único, mais robusto e interpretável.

c) Aplicação de Técnicas de Redução de Dimensionalidade

O uso de métodos como a Análise de Componentes Principais (PCA) permite transformar variáveis correlacionadas em componentes ortogonais (não correlacionados), preservando a maior parte da variabilidade dos dados. Essa abordagem é especialmente útil em bases de alta dimensionalidade, embora possa reduzir a interpretabilidade clínica direta.

d) Regularização de Modelos

Técnicas como o Lasso (Least Absolute Shrinkage and Selection Operator) ou o Elastic Net aplicam penalizações aos coeficientes durante o ajuste do modelo, mitigando automaticamente os efeitos da colinearidade e promovendo a seleção de variáveis.



A presença de variáveis colineares constitui um desafio metodológico relevante na modelagem preditiva em saúde, com potenciais implicações na robustez, interpretabilidade e utilidade clínica dos modelos.

Portanto, a identificação sistemática da colinearidade e a aplicação criteriosa de estratégias para sua mitigação devem ser incorporadas como práticas-padrão em projetos de ciência de dados na área da saúde.

A escolha da abordagem mais adequada deve considerar:

- a finalidade analítica (predição vs. explicação);
- o tipo de modelo adotado;
- as características das variáveis envolvidas.

Acima de tudo, é fundamental garantir que as decisões sobre inclusão ou exclusão de variáveis estejam ancoradas tanto em critérios estatísticos quanto em fundamentos clínico-epidemiológicos sólidos.

12. Redução de dimensionalidade

Bases de dados na área da saúde frequentemente apresentam um elevado número de variáveis, refletindo a complexidade dos fenômenos clínicos, epidemiológicos e socioeconômicos envolvidos. Em muitos casos, contudo, nem todas as variáveis contribuem de forma significativa para o desempenho dos modelos preditivos, o que pode gerar problemas, como:

- aumento desnecessário da complexidade computacional;
- redundância informacional;
- dificuldade de interpretação clínica dos resultados;
- maior risco de *overfitting* (ajuste excessivo aos dados de treinamento).

Esse cenário caracteriza o problema conhecido **como maldição da dimensionalidade** (*curse of dimensionality*), termo popularizado por Bellman (1957), que descreve o impacto negativo do crescimento exponencial do espaço de variáveis sobre a análise de dados e a modelagem preditiva.

Nesse contexto, técnicas de redução de dimensionalidade tornam-se ferramentas fundamentais para otimizar o desempenho dos modelos, facilitando simultaneamente sua interpretação e aplicação prática.

Definição e objetivos da redução de dimensionalidade

A redução de dimensionalidade consiste no processo de transformar um conjunto de variáveis originais em um novo conjunto com dimensionalidade reduzida, mantendo, sempre que possível, a maior quantidade de informação relevante contida nos dados.

Conforme definido por Hastie, Tibshirani e Friedman (2009), os principais objetivos dessa abordagem incluem:

- melhorar a eficiência computacional;
- mitigar o risco de overfitting;
- facilitar a visualização e a compreensão dos dados;
- eliminar redundâncias e ruídos informacionais.

A redução pode ocorrer por duas vias principais:

1. **seleção de variáveis (*feature selection*):** identificação e retenção de um subconjunto das variáveis originais, excluindo-se aquelas consideradas irrelevantes ou redundantes;
2. **extração de variáveis (*feature extraction*):** construção de novas variáveis a partir de combinações das variáveis originais, usualmente por meio de transformações matemáticas.

Principais técnicas de redução de dimensionalidade

a) Análise de Componentes Principais (Principal Component Analysis – PCA)

A PCA é, possivelmente, a técnica mais amplamente utilizada para redução de dimensionalidade. Trata-se de um método estatístico que transforma um conjunto de variáveis possivelmente correlacionadas em um novo conjunto de variáveis não correlacionadas, denominadas componentes principais.

Matematicamente, a PCA busca encontrar as direções no espaço dos dados que maximizam a variância, de modo que o primeiro componente principal (PC1) representa a direção de maior variância, o segundo (PC2) representa a maior variância remanescente e assim por diante.

Cada componente é uma combinação linear das variáveis originais, e a quantidade de variância explicada por cada componente é representada por seus autovalores associados.

Kuhn e Johnson (2013) ressaltam que a PCA é particularmente eficaz na identificação de padrões subjacentes em conjuntos de dados altamente correlacionados, sendo aplicável em contextos como:

- predição de riscos cardiovasculares com múltiplos marcadores metabólicos;
- análises de imagens médicas;
- estudos genômicos e metabolômicos.

b) Análise de Discriminante Linear (Linear Discriminant Analysis – LDA)

Enquanto a PCA visa maximizar a variância global dos dados, a LDA foca na maximização da separação entre grupos predefinidos. Trata-se de uma técnica supervisionada, utilizada para encontrar as combinações lineares de variáveis que melhor discriminam entre categorias do desfecho.

A LDA é comumente aplicada em contextos de classificação clínica, como na distinção entre casos e controles em estudos epidemiológicos.

c) Codificadores Automáticos (Autoencoders)

Autoencoders são redes neurais artificiais utilizadas para a compressão e reconstrução de dados. Por meio de uma arquitetura simétrica, essas redes aprendem uma representação compacta e codificada dos dados de entrada, com capacidade para capturar relações não lineares entre as variáveis.

Embora mais complexos e menos interpretáveis que técnicas lineares como a PCA, os *autoencoders* têm demonstrado excelente desempenho em tarefas de redução de dimensionalidade em dados de alta complexidade, como imagens médicas e registros eletrônicos de saúde (Van Buuren, 2018).

Considerações sobre interpretabilidade e aplicação clínica

Apesar das vantagens computacionais e analíticas, a redução de dimensionalidade, sobretudo por meio de técnicas de extração como a PCA e os *autoencoders*, pode reduzir a interpretabilidade clínica dos modelos. Isso ocorre porque as variáveis transformadas perdem sua correspondência direta com os indicadores originais, dificultando a tradução dos resultados para o contexto clínico ou epidemiológico.

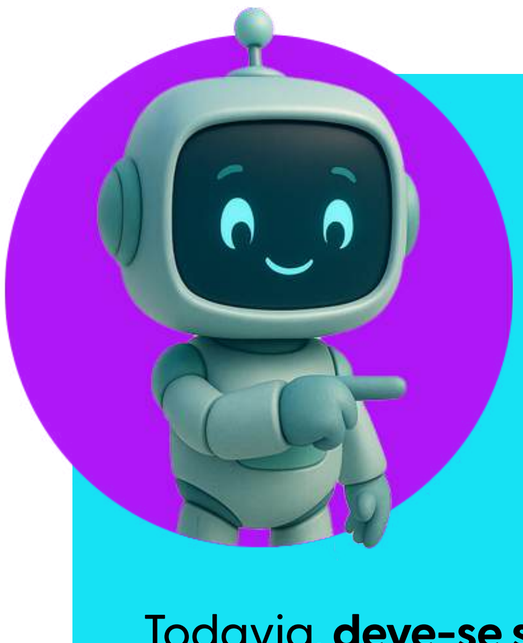
FIQUE ATENTO!

Gelman e Hill (2007) recomendam cautela ao aplicar técnicas de extração de variáveis em estudos que visam não apenas a predição, mas também a explicação dos fenômenos clínicos. Nesses casos, a seleção de variáveis (feature selection), associada à análise de correlação e ao conhecimento clínico, pode representar uma abordagem mais apropriada.

Exemplos aplicados em saúde

Na área da saúde, a redução de dimensionalidade tem sido aplicada em uma ampla gama de contextos, incluindo:

- **análises genômicas:** redução do número de variantes genéticas em estudos de associação genômica ampla (GWAS);
- **predição de risco cardiovascular:** agregação de múltiplos biomarcadores para construção de escores de risco;
- **imagens médicas:** compressão de imagens de alta resolução para análise automatizada;
- **bases de registros eletrônicos de saúde (EHR):** seleção de variáveis-chave para modelagem preditiva de desfechos clínicos.



A redução de dimensionalidade constitui uma etapa essencial na modelagem preditiva contemporânea, especialmente em bases de dados complexas e de alta dimensionalidade, como as frequentemente encontradas na saúde. Sua aplicação adequada permite:

- otimização da performance preditiva;
- redução do tempo computacional;
- melhora na estabilidade e na robustez dos modelos.

Todavia, **deve-se sempre ponderar os benefícios em termos de desempenho frente aos potenciais custos em termos de interpretabilidade**, garantindo que as decisões analíticas sejam compatíveis com os objetivos do estudo e as demandas do contexto clínico.

Todavia, deve-se sempre ponderar os benefícios em termos de desempenho frente aos potenciais custos em termos de interpretabilidade, garantindo que as decisões analíticas sejam compatíveis com os objetivos do estudo e as demandas do contexto clínico.

12. Considerações finais: a importância estratégica do pré-processamento de dados na modelagem preditiva em saúde

O pré-processamento de dados não deve ser encarado como uma mera etapa preliminar ou como um requisito técnico acessório na construção de modelos preditivos. Ao contrário, trata-se de um processo estruturante e essencial, com impacto direto sobre a qualidade, a robustez e a aplicabilidade dos modelos desenvolvidos.

Como demonstrado ao longo deste capítulo, a ausência de um pré-processamento rigoroso pode comprometer seriamente a validade dos resultados, mesmo quando se utilizam algoritmos de última geração. Os modelos de machine learning, independentemente de sua complexidade, são fundamentalmente dependentes dos dados nos quais foram treinados e testados. Em outras palavras, o modelo não pode, por si só, compensar a baixa qualidade dos dados de entrada.

Os riscos da negligência no pré-processamento

Negligenciar o pré-processamento pode resultar em:

- modelos que se ajustam aos ruídos e erros dos dados, em vez de aprenderem padrões clínicos genuínos;
- predições instáveis e inconsistentes;
- falsas associações e inferências incorretas;
- discriminações algorítmicas, ampliando desigualdades pré-existentes.

Pega essa dica!

Além disso, como apontam Kuhn e Johnson (2013), a ausência de um pipeline de pré-processamento bem estruturado compromete a reprodutibilidade dos estudos, inviabilizando tanto a validação externa quanto a aplicação prática dos modelos.

Pré-processamento como compromisso ético e científico

No campo da saúde, a importância do pré-processamento ultrapassa as questões técnicas e adquire uma dimensão ética. Dados de saúde representam pessoas, histórias de vida e condições clínicas reais. Portanto, falhas no tratamento desses dados podem não apenas comprometer a ciência, mas também afetar diretamente a vida de indivíduos e populações.

FIQUE ATENTO!

Nesse sentido, o pré-processamento é mais do que um procedimento técnico, é uma salvaguarda contra erros graves e um compromisso com a construção de modelos:

- mais justos;
- mais transparentes;
- mais eficazes;
- clinicamente relevantes.

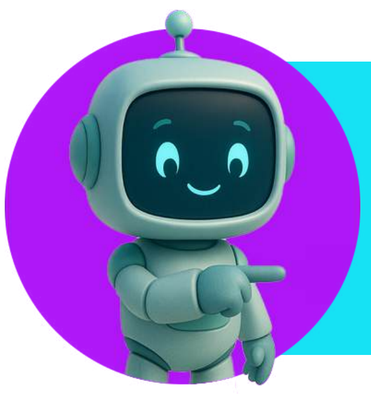
A centralidade do pré-processamento no ciclo de ciência de dados

Dentro do ciclo completo de projetos de ciência de dados, que envolve desde a definição do problema até a implementação do modelo, o pré-processamento ocupa um papel central e prolongado, geralmente consumindo a maior parte do tempo total do projeto.

Como síntese dos aprendizados deste capítulo, pode-se afirmar que um pré-processamento bem conduzido deve assegurar:

- completude e integridade dos dados (eliminação de duplicatas, tratamento de valores ausentes);
- consistência e acurácia (limpeza, padronização e validação de registros);
- representatividade (garantia de que as variáveis codificadas e padronizadas refletem adequadamente os fenômenos clínicos de interesse);
- parcimônia e eficiência (redução de dimensionalidade e controle da colinearidade para evitar redundância e *overfitting*).

Mais do que preparar os dados para o algoritmo, o pré-processamento estabelece as bases para análises científicas responsáveis, rigorosas e úteis no mundo real. Investir tempo, conhecimento e reflexão nessa etapa não é um custo adicional, mas uma garantia de que os modelos desenvolvidos poderão, de fato, contribuir para o avanço da saúde pública, para a melhoria dos cuidados clínicos e para a geração de evidências sólidas.



Nas palavras de Hastie, Tibshirani e Friedman (2009), “os dados são o combustível da aprendizagem estatística – e, como tal, precisam ser devidamente refinados antes que possam alimentar qualquer motor preditivo com segurança”.

Referências

BEAM, A. L.; KOHANE, I. S. Big data and machine learning in health care. **Jama**, v. 13, n. 319, p. 1.317-1.318, 2018.

BELLMAN, R. **Dynamic Programming**. Princeton: Princeton University Press, 1957.

GELMAN, A.; HILL, J. **Data Analysis Using Regression and Multilevel/Hierarchical Models**. New York: Cambridge University Press, 2007.

HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. **The Elements of Statistical Learning**: Data Mining, Inference, and Prediction. 2nd ed. New York: Springer, 2009.

KUHN, M.; JOHNSON, K. **Applied Predictive Modeling**. New York: Springer, 2013.

LITTLE, R. J. A.; RUBIN, D. B. **Statistical Analysis with Missing Data**. 2nd ed. Hoboken, NJ: Wiley, 2002.

THE LANCET. **Artificial intelligence and global health**: it’s time for action. Editorial, 12 Aug. 2023.

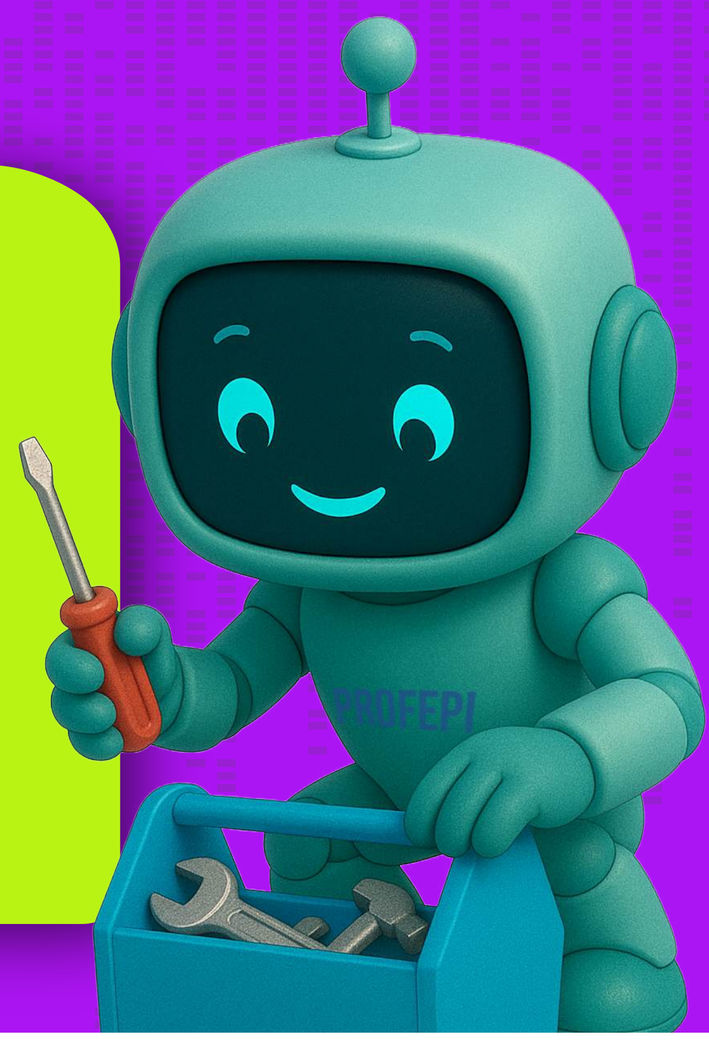
TOPOL, E. J. High-performance medicine: the convergence of human and artificial intelligence. **Nature Medicine**, v. 1, n. 25, p. 44-56, 2019.

VAN BUUREN, S. **Flexible Imputation of Missing Data**. 2nd ed. Boca Raton, FL: CRC Press, 2018.

WICHMANN, R. M.; FERNANDES, F. T.; CHIAVEGATTO FILHO, A. D. P. Improving the performance of machine learning algorithms for health outcomes predictions in multicentric cohorts. **Scientific Reports**, v. 1.022, n. 13, 2023.

capítulo 4

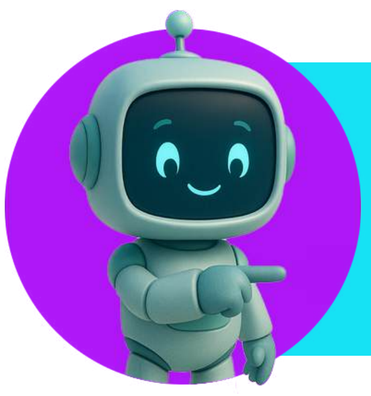
MÓDULO 1



Bibliotecas e Funções para Análise e Ciência de Dados

1. O que são as bibliotecas?

As bibliotecas (em inglês, *modules* ou *packages*) são conjuntos de ferramentas, funções e códigos prontos, agrupados por finalidade, que facilitam a programação. **Em vez de escrever tudo do zero, podemos reutilizar funções já desenvolvidas por outros programadores ou comunidades para realizar tarefas comuns**. Isso é semelhante a usar uma "caixa de ferramentas" especializada: cada biblioteca fornece ferramentas específicas para determinados problemas.



Por exemplo, há bibliotecas feitas para **análise de dados**, para **visualização de gráficos**, para **processamento de textos**, entre muitas outras áreas. Usando bibliotecas, economizamos tempo e evitamos erros, pois aproveitamos soluções que já foram testadas e validadas.

1.1. Como utilizar as bibliotecas?

Para utilizar uma biblioteca em nossos códigos Python, precisamos instalá-la (quando ela não vem instalada por padrão) e então importá-la no nosso script ou notebook.

Instalação

Algumas bibliotecas já vêm instaladas no ambiente (por exemplo, no Google Colab, muitas bibliotecas populares já estão disponíveis). Outras, porém, precisamos instalar manualmente. No Google Colab ou em um Jupyter Notebook, usamos o comando `!pip install nome_da_biblioteca` para instalar. O ponto de exclamação ! indica que estamos executando um comando de terminal (*shell*) para o gerenciador de pacotes pip.

Exemplo

Para instalar a biblioteca Pandas, usamos `!pip install pandas`. Esse comando baixa e instala o Pandas no ambiente. Podemos fazer o mesmo para outras bibliotecas necessárias. No notebook desta aula, instalamos todas as bibliotecas que não fossem padrão de uma vez só, por exemplo:

Python - input

```
!pip install pandas
!pip install numpy
!pip install seaborn
!pip install matplotlib
```

Importação

Após garantir que a biblioteca está instalada, precisamos **importá-la** no código para poder usar suas funções. Importar é como pegar a ferramenta na caixa para usá-la. Usamos o comando `import nome_da_biblioteca`. Muitas vezes, usamos um **apelido** para a biblioteca ao importar, para abreviar seu nome e facilitar a escrita do código. Por exemplo, é comum fazer `import pandas as pd`, o que importa a biblioteca Pandas e nos permite chamá-la de `pd` no código. Assim, em vez de escrever `pandas.funcao()`, escrevemos `pd.funcao()`. Outros exemplos:

Python - input

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

Pega essa dica!

Podemos importar as bibliotecas com qualquer apelido, mas, geralmente, as importamos com os apelidos:

- `pd` para Pandas,
- `np` para NumPy,
- `sns` para Seaborn,
- `plt` para Matplotlib (pyplot)

Também podemos importar apenas partes específicas de uma biblioteca. No nosso caso, por exemplo, importamos **OneHotEncoder** do submódulo `sklearn.preprocessing` assim: `from sklearn.preprocessing import OneHotEncoder`. Isso traz apenas a classe OneHotEncoder da biblioteca scikit-learn (sklearn), que utilizaremos mais adiante.

Pega essa dica!

Perceba que não estamos mais utilizando o ! nos comandos. Precisamos colocar o sinal de exclamação somente para instalar as bibliotecas; o restante dos comandos não é acompanhado pelo sinal.

2. Bibliotecas utilizadas

Vamos listar as bibliotecas usadas nesta aula e sua finalidade em termos simples.

- **Pandas:** biblioteca para manipulação de dados tabulares (planilhas, tabelas) de forma eficiente. Facilita leitura de arquivos (CSV, Excel), limpeza de dados, filtragem etc.
- **NumPy:** biblioteca para cálculos numéricos eficientes, fornecendo suporte a vetores, matrizes e funções matemáticas de alto desempenho (muito usada em conjunto com Pandas).
- **Matplotlib/Pyplot:** biblioteca básica de geração de gráficos. O submódulo `pyplot` fornece funções simples para criar gráficos de linha, barras, dispersão etc., de forma semelhante a comandos do MATLAB.
- **Seaborn:** biblioteca de visualização de dados baseada no Matplotlib, porém com estilos mais bonitos e funções de alto nível que facilitam a criação de gráficos estatísticos (por exemplo, gráficos de dispersão com regressão, boxplots, histogramas aprimorados).
- **Scikit-learn (sklearn):** biblioteca de machine learning e estatística, com diversas ferramentas para pré-processamento de dados, algoritmos de aprendizado de máquina, avaliação de modelos etc. Nesta aula, usamos apenas uma funcionalidade de pré-processamento (OneHotEncoder) como exemplo.

Todas essas bibliotecas, juntas, nos permitem carregar um conjunto de dados, examinar suas características, limpar e preparar os dados e visualizar informações relevantes, preparando o terreno para análises mais avançadas ou para a construção de modelos preditivos.

A documentação oficial de cada uma dessas ferramentas é o **guia indispensável** que descreve em detalhes o que cada função faz, quais "ingredientes" (parâmetros) ela precisa para funcionar corretamente e quais resultados ela entrega.

Ignorar a leitura da documentação é como tentar administrar um novo medicamento sem conhecer a posologia: você pode até ter uma ideia geral do que fazer, mas são nos detalhes que residem a segurança, a precisão e a eficácia do procedimento.

Por meio da documentação de uma biblioteca, compreendemos o uso correto das ferramentas, exploramos seu potencial máximo e evitamos erros que poderiam comprometer os resultados de uma pesquisa ou a implementação de uma solução tecnológica na área da saúde.

3. Importação dos dados

Para praticar, utilizaremos um conjunto de dados real de saúde (neste caso, dados de tuberculose). Esses dados foram fornecidos em um arquivo CSV (valores separados por vírgulas) chamado `dados_tuberculose.csv`. O objetivo é carregar esse arquivo em memória e manipular seus conteúdos.

Antes de importar, vamos relembrar alguns conceitos básicos de Python importantes para quem não tem familiaridade com programação.

- **Variáveis e atribuição:** para guardar dados em Python, usamos variáveis. No código, o operador = serve para **atribuir** um valor a uma variável. Por exemplo, ao escrever `x = 5`, estamos criando a variável `x` e armazenando o número `5` nela.

FIQUE ATENTO!

É diferente da matemática. No Python, **=** não significa igualdade, mas sim atribuição. Já o operador de comparação de igualdade é **==**. Ou seja, **x == 5** verifica se o valor de **x** é igual a **5**, enquanto **x = 5** coloca **5** dentro de **x**.

- **Comentários no código:** textos precedidos do símbolo # são comentários. Comentários são ignorados na execução, servindo apenas para explicar o código aos leitores (inclusive a você mesmo). No nosso notebook, usamos comentários para descrever o que cada comando está fazendo. Por exemplo: # Importando os dados de um csv... é apenas um comentário descritivo.

Com isso em mente, vamos **ler os dados do arquivo CSV** usando o Pandas. A função do Pandas para ler um arquivo CSV é `pd.read_csv()`. Ela carrega o conteúdo do arquivo em um **DataFrame**, que é a estrutura de dados principal do Pandas (semelhante a uma tabela ou planilha, com linhas e colunas).

Ao ler arquivos texto, pode haver detalhes a ajustar, como o tipo de codificação de caracteres (*encoding*) e o separador de valores. Nosso arquivo está em português e contém caracteres acentuados, e está separado por vírgulas – isso é o padrão de CSV (*Comma-Separated Values*). No entanto, às vezes, o Pandas pode interpretar incorretamente caracteres especiais se não informarmos a codificação correta. No Brasil, muitos arquivos CSV, em português, usam a codificação Latin-1 (também chamada ISO-8859-1) em vez de UTF-8.

Saiba mais!

A tabela com os códigos de todas as codificações pode ser encontrada em https://docs.python.org/3/library/codecs.html#standard-encodings.

No notebook, para evitar problemas com acentos e cedilhas, especificamos o parâmetro `encoding='latin1'`. Além disso, explicitamos `sep=","` apenas para indicar que o separador é vírgula (embora seja o parâmetro padrão da função `read_csv()`, decidimos deixá-la explícita).

A importação dos dados com a biblioteca Pandas fica da seguinte maneira:

Python - input

```
# Importando os dados de um csv e associando-os à variável “df”
df = pd.read_csv(‘/content/dados_tuberculose.csv’, encoding='latin1', sep=",")
```

Aqui, `df` será nossa variável que guarda o DataFrame com todos os dados do arquivo. Estamos dizendo: “Leia o CSV localizado em `/content/dados_tuberculose.csv` usando a codificação **latin1** e o separador de coluna como **vírgula**. Armazene o resultado na variável `df`.”

O Pandas contém diversas funções para importar dados de diferentes formatos. Para importar uma planilha Excel (geralmente com o formato .xlsx ou .xls), utilizamos a função `pd.read_excel()`.

Para fazer o download de um DataFrame em .csv ou como uma planilha Excel, utilizamos as funções `to_csv( )` e `to_excel( )`, respectivamente.

Saiba mais!

As diferentes funções de importação e salvamento do Pandas contêm parâmetros semelhantes e podem ser encontradas em https://pandas.pydata.org/docs/reference/io.html.

Sobre possíveis avisos: ao executar essa leitura, o Pandas emitiu um **aviso (*warning*)** indicando que algumas colunas têm tipos mistos (ou seja, em certas colunas há mistura de números e texto, o que faz o Pandas não ter certeza se deveria interpretá-las como números ou *strings*). O aviso sugere que poderíamos especificar manualmente os tipos ou usar uma opção (`low_memory=False`) para melhorar a leitura.

Fique atento!

Avisos não impedem que manipulemos os dados no Python, mas devem ser compreendidos, porque podem indicar uma importação falha, o que impede que tenhamos corretamente todos os dados em nosso ambiente de programação.

4. Acessando elementos no Python

Antes de explorarmos os dados, é útil saber **como acessar partes específicas** do DataFrame (e de outros objetos). Isso inclui selecionar colunas, linhas ou células individuais.

No Python, utilizamos colchetes `[...]` para acessar elementos de estruturas.

- Para acessar colunas pelo nome, usamos `df['NomeDaColuna']`. Isso retorna uma série (coluna específica). Por exemplo, `df['IDADE_ANOS']` nos daria todos os valores da coluna de idade.
- Para acessar linhas por índice, podemos usar os índices numéricos. O Python utiliza índice base 0, ou seja, a primeira linha tem índice 0, a segunda índice 1, e assim por diante.
    - `df[0:10]` – retorna as linhas de índice 0 até 9 (intervalo do início até antes do 10). Essa sintaxe de fatiamento (`inicio:fim`) inclui a linha de início e exclui a de fim. No caso, pega 10 linhas (0 a 9).
    - `df[10:20]` – pega as linhas de índice 10 a 19.
    - `df[:10]` – do início até a linha 9 (primeiras 10 linhas).
    - `df[10:]` – da linha 10 até o final do DataFrame.

Essas são maneiras rápidas de inspecionar fatias do dataset.

Fique atento!

Atenção: Existe também o acesso por índices nomeados ou booleanos (usando `.loc` ou `.iloc` no Pandas), mas ambos se apoiam na lógica do acesso por meio de colchetes. Tenha em mente que `df[10:20]` funciona para fatiar linhas, mas `df['COL']` funciona para pegar coluna.

5. Explorando os dados iniciais

Agora que o df contém o DataFrame com os dados das notificações de tuberculose, é hora de inspecioná-lo e compreender sua estrutura. O dataset é grande (mais de 1 milhão de registros, no caso, notificações de tuberculose de diversos anos e locais). Vamos aprender algumas operações básicas para conhecer o DataFrame, além do acesso a linhas ou colunas específicas.

- **Visualizar o DataFrame inteiro:** se simplesmente digitarmos `df` em uma célula do notebook e executarmos, o Pandas exibirá uma representação da tabela. Como a tabela é muito grande, ele não mostrará todas as linhas, mas tipicamente mostra as primeiras e últimas linhas e indica quantas linhas e colunas existem no total.
- **Primeiras e últimas linhas:** podemos usar funções do Pandas para ver apenas o início ou o fim do DataFrame. A função `df.head(n)` mostra as primeiras `n` linhas, e `df.tail(n)` as últimas n linhas.
- **Dimensões do DataFrame:** poderíamos obter isso programaticamente usando `df.shape`. Perceba que não colocamos parênteses após o comando shape; isso acontece porque shape é um atributo do DataFrame, não uma função.
    - Ao utilizarmos `.shape`, recebemos a quantidade de linhas e colunas no formato (*`quantidade_linhas`*, *`quantidade_colunas`*), que podem ser acessadas individualmente com os colchetes.
- **Nome das colunas:** para listar todas as colunas disponíveis no DataFrame, podemos usar `df.columns`.
- **Tipos de dados de cada coluna:** cada coluna em um DataFrame tem um tipo de dado (números inteiros, números decimais, texto, datas etc.). Para ver os tipos, podemos usar `df.dtypes` ou o método `df.info()`.
    - Colunas com números inteiros, como a `NU_ANO`, aparecem como `int64` (inteiro de 64 bits).
    - Colunas com valores decimais ou com dados faltantes aparecem como `float64` (ponto flutuante).
    - Colunas de texto aparecem como `object` (esse é o tipo Pandas para *strings* ou mix de tipos).
    - Datas podem aparecer como `object` se não foram automaticamente interpretadas como data.

Tipo de dados e conversão

Cada coluna do DataFrame tem um tipo definido (número, texto, data etc.) e o Pandas tenta determinar automaticamente esses tipos ao ler o arquivo. Entretanto, às vezes, precisamos **converter** manualmente para um tipo mais adequado.

No nosso dataset, por exemplo, a coluna de data (`DT_DIAG`) veio como texto (*string* ou *object*) e devemos convertê-la para um tipo datetime (data). Isso facilita qualquer análise temporal, permitindo extrair mês, ano, ou calcular diferenças de tempo.

Para essa operação, utilizamos a função `to_datetime()`.

Python - input

```
df['DT_DIAG'] = pd.to_datetime(df['DT_DIAG'])
```

Esse comando pega a coluna `DT_DIAG` de `df` e tenta interpretá-la como datas, convertendo-a internamente para um formato de data. Depois disso, se rodarmos novamente `df.dtypes ou df.info()`, veríamos `DT_DIAG` com um tipo datetime64. Assim, operações de data (como filtrar por ano, calcular intervalos) se tornam mais fáceis.

As funções de conversão mais utilizadas são:

Função	Objetivo
`pd.to_datetime(df['NOME_COLUNA'])`	Converter uma coluna para o formato de data.
`pd.to_numeric(df['NOME_COLUNA'])`	Converter uma coluna para o formato numérico – tanto inteiro (*int*), quanto decimal (*float*).
`df['NOME_COLUNA'].astype('str')`	Converter uma coluna para *string* (ou *object*).

Lidando com dados ausentes (missings)

Datasets (sobretudo os relacionados à saúde) costumam ter valores **ausentes** (faltantes ou *missings*) em várias colunas – seja porque a informação não foi coletada, seja por erro de registro. É importante identificar quantos e quais dados faltam, pois isso influencia como vamos analisar ou modelar os dados.

Para encontrar valores ausentes no Python, usamos `df.isna()` ou `df.isnull()` (são equivalentes). Essas funções geram um DataFrame de valores booleanos (`True`/`False`) indicando se cada elemento é NaN (*Not a Number*, marcador de dado faltante, que também pode ser representado como na ou *null*) ou não.

Podemos somar valores booleanos em Python, no qual True recebe o valor 1 e False recebe o valor 0, o que permite contar quantos True (ou seja, quantos faltantes) há em cada coluna. O comando fica sendo:

Python - input

```
df.isna().sum()
```

Nos datasets, possivelmente encontraremos colunas com bastantes dados faltantes (um exemplo comum: resultados de certos exames, ou informações complementares que nem sempre são preenchidas).

Para facilitar a visualização, também ordenamos essas contagens de *missings*:

Python - input

```
df.isna().sum().sort_values(ascending=False)
```

Em complemento, é informativo calcular a **proporção (percentual) de valores ausentes** em relação ao total de linhas. Para isso, precisamos dividir a soma de *missings* pela quantidade de linhas, que pode ser identificada com o comando `df.shape[0]`.

Python - input

```
df.isna().sum().sort_values(ascending=False) / df.shape[0]
```

Filtrando o DataFrame

Filtrar o DataFrame para obter apenas as linhas que satisfazem certa condição (por exemplo, casos de um determinado estado, ou de um determinado ano) é uma operação bastante útil. Vamos aprender a filtrar usando operadores lógicos.

Os operadores comuns para filtros são:

- `==` (igual a);
- `!=` (diferente de);
- `>` (maior que);
- `<` (menor que);
- `>=` (maior ou igual);
- `<=` (menor ou igual);
- `&` (pertence a esse e àquele);
- `|` (pertence a esse ou àquele).
    - A função `isin( )` é um substituto para o operador **ou** (`|`).

Quando aplicamos uma comparação em uma coluna do DataFrame, obtemos uma série de True/False indicando para cada linha se a condição é satisfeita. Por exemplo, `df['NU_ANO'] == 2018` resulta em uma série booleana, na qual será True nas linhas cujo ano é 2018 e False nas demais.

Para filtrar, colocamos essa condição entre colchetes do DataFrame: `df[ condição ]`. O Python então retorna apenas as linhas em que a condição é True.

Exemplo 1: Suponha que queremos os registros do Estado de Minas Gerais. Precisamos saber como o estado está codificado. Olhando a coluna `SG_UF_NOT`, vimos valores como 31, 42, 25 etc., e notamos que o registro de índice 3 tinha `SG_UF_NOT = 31` e correspondia a Minas Gerais (deduzimos isso pelo contexto ou conhecendo o código IBGE de MG que é 31). Para confirmar os códigos de estados presentes, podemos usar:

Python - input

```
df['SG_UF_NOT'].unique()
```

Isso retorna os valores únicos dessa coluna (lista de códigos de estados presentes no dataset). Além disso, `df['SG_UF_NOT'].value_counts()` mostra quantas ocorrências tem cada código de estado.

Sabendo que **31 é o código de Minas Gerais**, podemos filtrar:

Python - input

```
df_filtro = df[df['SG_UF_NOT'] == 31]
```

Isso cria um novo DataFrame chamado `df_filtro` contendo somente as linhas em que a coluna SG_UF_NOT tem valor igual a 31, ou seja, registros de Minas Gerais.

Exemplo 2: Suponha que queremos só os casos do ano 2020.

Python – input

```
df_2020 = df[df['NU_ANO'] == 2020]
```

Se quisermos combinar condições, por exemplo, casos de Minas Gerais **no ano de 2020:**

Python – input

```
df_mg_2020 = df[(df['SG_UF_NOT'] == 31) & (df['NU_ANO'] == 2020)]
```

Aqui usamos & para "E" lógico – ambas condições devem ser verdadeiras para a linha ser selecionada. Note a necessidade de colocar cada comparação entre parênteses.

Exemplo 3: Filtrar múltiplos anos.

Python – input

```
df_17_18 = df[(df['NU_ANO'] == 2017) | (df['NU_ANO'] == 2018)]
```

Isso retorna linhas cujo ano seja 2017 ou 2018.

Uma maneira mais conveniente de gerar o mesmo resultado é usar o método `.isin(lista)`

Python – input

```
filtro_anos = [2017, 2018]
df_17_18 = df[df['NU_ANO'].isin(filtro_anos)]
```

No geral, aprendemos que, com essas ferramentas, podemos **fazer consultas** nos dados de forma flexível. É parecido com filtrar linhas em Excel usando critérios, mas aqui escrevemos as condições logicamente.

No nosso caso prático, decidimos focar a análise no Estado de **Minas Gerais**. Então aplicamos o filtro por `SG_UF == 31` para obter `df_filtro`.

Explorando mais os dados (após filtrar)

Com o DataFrame filtrado para Minas Gerais em mãos (`df_filtro`), passamos a realizar algumas limpezas e análises adicionais.

Removendo colunas desnecessárias: muitas colunas do dataset original podem não ser úteis para a análise ou modelo que queremos construir. Por exemplo:

- identificadores geográficos que agora são redundantes (como códigos de município, porque poderíamos não precisar de detalhamento por município numa análise estadual, ou porque não vamos usar essa variável);
- colunas quase totalmente vazias (que vimos na análise de *missing*);
- colunas de identificação ou que não fazem sentido incluir (como códigos internos, datas de encerramento se já temos a situação de encerramento etc.);
- colunas que representam informações que decidimos codificar de outra forma.

No notebook, foi preparada uma lista de colunas a retirar, chamada `colunas_para_retirar`. Ela inclui diversas colunas identificadas pelos nomes. Entre elas (lista resumida): `'SG_UF_NOT', 'ID_MUNICIP', 'ID_REGIONA', 'DT_DIAG', 'NU_IDADE_N', 'SG_UF', 'ID_MN_RESI', 'ID_RG_RESI', 'ID_PAIS', 'EXTRAPU1_N', 'EXTRAPU2_N', 'SG_UF_AT', 'ID_MUNIC_A', 'DT_NOTI_AT', 'SG_UF_2', 'ID_MUNIC_2', 'BACILOSC_1' ... 'BACILOSC_6', 'BAC_APOS_6', 'OUTRAS', 'OUTRAS_DES', 'DT_MUDANCA', 'SITUA_9_M', 'SITUA_12_M', 'DT_ENCERRA'` entre outras.

Basicamente, estão sendo removidos:

- informações de residência redundantes (`ID_MN_RESI, ID_RG_RESI` etc.;
- resultados de acompanhamento de baciloscopia enumerados (1 a 6);
- campos de situação aos 9 e 12 meses (`SITUA_9_M, SITUA_12_M`) e datas de mudança e encerramento (`DT_MUDANCA, DT_ENCERRA`);
- o próprio código do estado (`SG_UF` e variantes) – uma vez que filtramos só MG, a coluna que indica o estado será constante (31 para todas as linhas), então não traz informação nova e pode ser removida.

Em resumo, retiramos colunas de **identificação geográfica duplicada ou irrelevante, colunas de acompanhamento intermediário e colunas que não seriam usadas**. Isso simplifica o conjunto de dados para trabalharmos apenas com as variáveis mais importantes.

A remoção de colunas foi feita com:

Python - input

```
df_modelagem = df_filtro.drop(columns=colunas_para_retirar)
```

Aqui criamos um DataFrame `df_modelagem`, que será a base filtrada e limpa para análise e modelagem. Optou-se por criar outro DataFrame para preservar o original caso precisemos voltar à informação primeira, mas poderíamos ter removido no próprio `df_filtro` também.

Para obter uma visão rápida das principais métricas estatísticas descritivas, utilizamos a função `.describe()`:

Python - input

```
df_modelagem.describe().T
```

O `.describe()` nos permite gerar contagem, média, desvio-padrão, mínimos e percentis de cada coluna numérica. Como a saída tem muitas colunas, aplicamos `.T` (transpor) para girar a tabela, listando as colunas verticalmente para facilitar a leitura. Assim podemos inspecionar rapidamente, por exemplo, a idade média ou ver distribuições de variáveis contínuas e códigos (por exemplo, podemos ver que `NU_ANO` vai de 2013 a 2023, ou que alguma variável binária tem máximo 1 e mínimo 0, etc.). Isso nos ajuda a detectar valores fora do comum e entender os dados.

6. Visualização de dados

Visualizar os dados graficamente pode ser muito esclarecedor. Vamos produzir alguns gráficos simples para entender melhor o comportamento de certas variáveis no tempo e por grupos.

Para visualizarmos um gráfico da idade média durante os anos, primeiro a calculamos e depois geramos um gráfico com a biblioteca **seaborn**.

Python - input

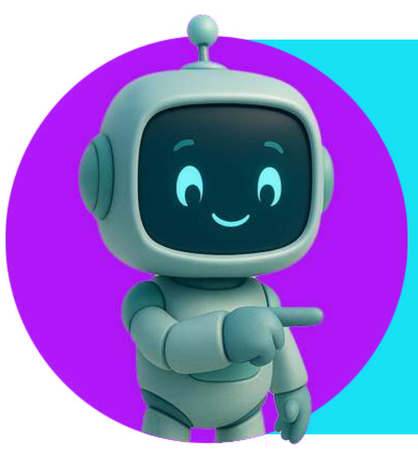
```
idade_media_anual = df_modelagem.groupby(
    ['NU_ANO'])['IDADE_ANOS'].mean().reset_index()
```

Para realizarmos essa operação, primeiro agrupamos os nossos dados por ano, utilizando a coluna `NU_ANO` e depois passamos a função `.mean()`, que nos retorna a média. A função `.reset_index()` organiza o resultado em uma tabela mais simples.

Com os valores calculados, vamos utilizar a biblioteca **seaborn** em conjunto com a biblioteca **matplotlib** para gerar gráficos.

Python - input

```
plt.figure(figsize=(10,5))
sns.barplot(data=idade_media_anual, x='NU_ANO', y='IDADE_ANOS' ,
color='red')
plt.title('Idade Média dos Indivíduos Notificados com Tuberculose')
plt.xlabel('Ano')
plt.ylabel('Idade Média')
```



Veja a lógica dos gráficos como etapas de uma tela de pintura: primeiro informamos o tamanho da tela (tamanho do gráfico) com `plt.figure(figsize=(10,5))`, depois desenhamos o conteúdo (geramos o gráfico) com `sns.barplot(data=idade_media_anual, x='NU_ANO', y='IDADE_ANOS', color='red')` e, por fim, ajustamos alguns detalhes com as demais linhas de código do exemplo.

Pega essa dica!

Diversos elementos do gráfico são personalizáveis e podemos encontrar os comandos na documentação do **seaborn**: https://seaborn.pydata.org/examples/index.html.

Como nosso dataset filtrado cobre notificações de 2013 a 2023 em MG, um aspecto interessante é ver a distribuição das idades dos pacientes ao longo dos anos. Uma forma de fazer isso é usando um **boxplot**.

Um **boxplot** mostra a distribuição de uma variável numérica (no caso, idade) para diferentes categorias (no caso, ano) através de caixas e linhas.

- **A linha central** em cada caixa é a **mediana** (50% dos casos estão abaixo dela, 50% acima).
- As **bordas da caixa** são o primeiro e terceiro **quartis** (Q1 e Q3): 25% dos dados estão abaixo de Q1, 75% abaixo de Q3. A caixa portanto contém os 50% centrais dos dados.
- A **altura da caixa** (Q3 - Q1) é chamada de **intervalo interquartil (IQR)**.
- Os "**bigodes**" (*whiskers*) geralmente se estendem até o valor mínimo e máximo que não sejam considerados outliers. Uma convenção comum (usada por Seaborn e Matplotlib) é estender os bigodes até no máximo 1,5 vezes o IQR além de Q1 e Q3.
- **Valores discrepantes (ou outliers)** são pontos plotados individualmente além dos bigodes. Eles representam observações que estão significativamente fora do range esperado (mais de 1.5 * IQR abaixo de Q1 ou acima de Q3).

No contexto das idades:

- se um boxplot de um ano tem muitos *outliers* no topo, pode indicar que houve alguns pacientes muito mais velhos que o usual naquele ano, por exemplo;
- uma mediana subindo com os anos poderia sugerir que a idade mediana dos casos notificados está aumentando (pode ser demográfico, ou relacionado a detecção tardia etc.).

Com esse boxplot, podemos comparar rapidamente os anos. Por exemplo, podemos ver se 2020 (ano de pandemia de covid-19) teve alguma diferença no perfil de idade ou se há outliers indicando poucos casos. Boxplots nos dão uma visão de dispersão e centralidade simultaneamente.

Além desse, também poderíamos querer ver diferenças por sexo. Por exemplo, será que a distribuição de idade difere entre homens e mulheres? Para isso, poderíamos plotar um boxplot separado por sexo. Seaborn permite adicionar uma dimensão de agrupamento usando o parâmetro `hue`.

Python - input

```
sns.boxplot(data=df_modelagem, x='NU_ANO', y='IDADE_ANOS', hue='CS_SEXO')
```

Isso desenharia duas caixas por ano: uma para sexo masculino (por exemplo, M) e outra para feminino (F). Assim, podemos ver se em certos anos as mulheres notificadas eram em média mais jovens ou vice-versa, ou se a dispersão difere.

Entender gráficos nos ajuda a obter insights que tabelas às vezes não mostram claramente.

Preparação para a modelagem

Para preparar os dados para alimentar um modelo de machine learning, transformamos variáveis categóricas em numéricas, criamos novas variáveis, se necessário (*feature engineering*) e selecionamos o que será predito (variável desfecho).

Criação do desfecho (variável-alvo)

No contexto de tuberculose, um possível desfecho de interesse é saber se o paciente foi a óbito por tuberculose durante o tratamento. No dataset, existe a coluna `SITUA_ENCE` (situação de encerramento do caso) que codifica o resultado final de cada caso. Os códigos possíveis (segundo o dicionário de dados apresentado) são:

- **1** – Cura;
- **2** – Abandono do tratamento;
- **3** – Óbito por tuberculose;
- **4** – Óbito por outras causas;
- **5** – Transferência;
- **6** – Mudança de diagnóstico;
- **7** – TB-DR;
- **8** – Mudança de Esquema;
- **9** – Falência;
- **10** – Abandono Primário.

Dessas categorias, para predizer quem morreu de tuberculose, precisamos criar uma variável binária indicando **óbito por tuberculose = sim ou óbito por tuberculose = não** para cada linha.

No dataset original, não havia uma coluna explicitamente chamada "ÓBITO_TB", mas podemos construir essa informação a partir de `SITUA_ENCE`. Uma forma de fazer isso no Python é usar a função `np.where()` do NumPy para criar uma nova coluna condicionalmente.

Python - input

```
df_modelagem['OBITO'] = np.where(df_modelagem['SITUA_ENCE'] == 3, 1, 0)
```

Isso cria uma coluna nova chamada `OBITO` em `df_modelagem`, em que:

- atribui valor 1 se `SITUA_ENCE` for 3 (casos em que o desfecho foi "Óbito por TB");
- atribui 0 caso contrário (todas as outras situações).

Assim, `OBITO` se torna nossa variável de desfecho binária:

- 1 representa morreu por tuberculose;
- 0 representa não morreu por tuberculose (seja por cura, abandono etc., qualquer outro resultado que não óbito por TB).

Criar essa variável é um exemplo de ***feature engineering*** simples: estamos extraindo uma informação importante de uma variável categórica existente e transformando-a em um formato mais utilizável para certos modelos (modelos de classificação binária, por exemplo).

Após criar `OBITO`, como já extraímos a informação necessária de `SITUA_ENCE`, poderíamos remover a coluna `SITUA_ENCE` do DataFrame para não ter duplicidade ou para evitar usá-la indevidamente (especialmente se formos treinar um modelo supervisionado, no qual não queremos alimentar o conjunto de preditoras com a resposta que precisa ser predita).

Python - input

```
df_modelagem = df_modelagem.drop(columns=['SITUA_ENCE'])
```

Agora `df_modelagem` não tem mais a coluna de situação de encerramento, mas tem a coluna `OBITO` indicando o óbito por tuberculose.

One-Hot Encoding (Codificação de Variáveis Categóricas)

Muitas colunas do nosso dataset são variáveis categóricas codificadas numericamente. Exemplos: sexo (M/F); gestante (sim/não/categorias); raça/cor (1 a 5 e 9 = ignorado); escolaridade em níveis numéricos etc. Embora esses códigos sejam numéricos, eles representam categorias **qualitativas**, não quantidades. Modelos de machine learning típicos não conseguem interpretar automaticamente que os valores em `CS_RACA` por exemplo são categorias sem relação de ordem (1 não é menor nem maior "raça" que 2 etc., são apenas códigos). Se deixássemos os números assim, um modelo linear poderia interpretar que uma categoria com valor 4 é "maior" que valor 2, o que não faz sentido intrínseco (são apenas rótulos).

Portanto, uma prática comum é aplicar **One-Hot Encoding (OHE)** às variáveis categóricas. Essa técnica consiste em criar colunas *dummys* binárias – uma para cada categoria possível –, marcando com 1 quando a observação pertence àquela categoria e 0 quando não. Assim, representamos a informação categórica de maneira expansiva, mas numérica pura, sem indicar qualquer ordem.

Primeiro, vamos entender `CS_RACA`: o OHE detectou sete valores únicos na coluna `CS_RACA` e criou sete colunas.

Python - input

```
ohe = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
ohe_cols = ohe.fit_transform(df_modelagem[['CS_RACA']])
```

- Primeiro, instanciou o encoder ohe.
- Depois, usou `fit_transform` em `df_modelagem[['CS_RACA']]`. Note que passamos `[['CS_RACA']]` com dois colchetes para manter como DataFrame.

Python - output

	0	1	2	3	4	5	6
0	0.0	0.0	1.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	1.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0	1.0
3	1.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	1.0	0.0	0.0	0.0	0.0	0.0

Cada coluna (0,1,2,...,6) corresponde a uma categoria de raça.

Observando as primeiras linhas:

- Linha 0 tem `1.0` na coluna 2, indicando categoria 3 para `CS_RACA`;
- Linha 1 tem `1.0` na coluna 4, indicando categoria 5;
- Linha 2 tem `1.0` na coluna 6, indicando categoria 9;
- Linha 3 tem `1.0` na coluna 0, indicando categoria 1;
- Linha 4 tem `1.0` na coluna 1, indicando categoria 2.

Próximos passos com OHE: normalmente, após obter essas colunas, as incorporaríamos ao DataFrame principal e removeríamos a coluna categórica original.

Por que usar One-Hot?

Muitos algoritmos de machine learning exigem entrada numérica. Se passássemos CS_RACA como uma coluna numérica 1-5,9, um modelo linear ou de árvore poderia até lidar, mas:

- modelos lineares tratariam os números literalmente (podendo supor impacto linear ou ordinal das categorias);
- árvores de decisão conseguem lidar com categorias numéricas na divisão, mas não calculam distâncias corretamente se for um método baseado em distância etc.;
- One-hot garante que cada categoria é independente e é tratada como presença/ausência, sem ordem implícita.

O lado negativo é aumentar o número de colunas e a esparsidade, mas com datasets moderados isso é gerenciável.

Podemos sintetizar com uma analogia: é como se transformássemos a variável "Raça/Cor" que tem opções múltiplas em várias colunas de "É branca (S/N)", "É preta (S/N)", "É amarela (S/N)" e assim por diante. Dessa forma, cada paciente marca "sim" apenas na coluna correspondente à sua categoria e "não" em todas as outras. Essa redundância evita dar um peso ou hierarquia numérica equivocada às categorias.

Referências

MATPLOTLIB. **Matplotlib 3.10.3 documentation**. Disponível em: https://matplotlib.org/stable/index.html.

NUMPY. **NumPy documentation**. Disponível em: https://numpy.org/devdocs/.

PANDAS. **Pandas documentation**. 2025. Disponível em: https://pandas.pydata.org/docs/.

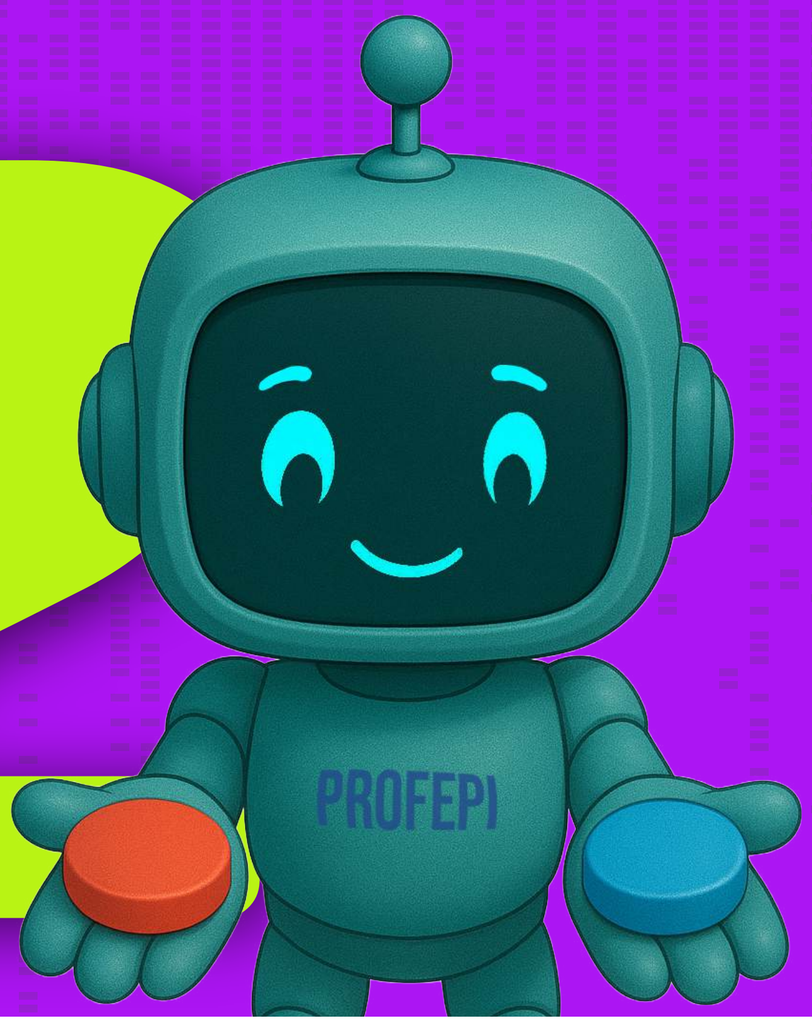
SCIKIT LEARN. **OneHotEncoder**. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html.

SEABORN. **Seaborn**: statistical data visualization. Disponível em: https://seaborn.pydata.org/index.html.



capítulo 5

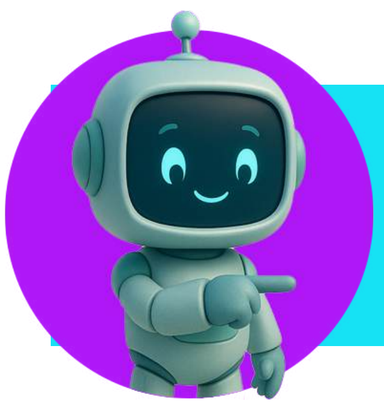
MÓDULO 2



Avaliação e Seleção de Modelos de *Machine Learning*

Introdução

O uso de algoritmos de *machine learning* (ML) em saúde tem avançado em velocidade impressionante, com aplicações que vão da predição de mortalidade neonatal ao diagnóstico assistido por imagem. Entretanto, a força de qualquer modelo está menos na sofisticação de seu algoritmo e mais na qualidade da sua avaliação. Modelos que parecem perfeitos em ambientes controlados podem falhar gravemente quando expostos à realidade clínica, gerando vieses, erros diagnósticos ou decisões administrativas incorretas.



É nesse ponto que entram os métodos de **avaliação e seleção de modelos**. Sem eles, estaríamos aplicando ferramentas de alto impacto em saúde sem qualquer garantia de validade.

Um princípio norteador é o **teorema do "não há almoço grátis" (no free lunch theorem)**: em um número infinito de problemas diferentes, nenhum algoritmo é universalmente superior. Assim, o único caminho é testar, comparar e validar. Embora, na prática, alguns algoritmos tenham desempenho superior em muitas situações reais – como gradient boosting, random forests ou TabPFN – a escolha final só pode ser feita com base em evidências empíricas.

Função alvo (target function): representa a verdadeira relação entre as variáveis preditoras (x) e o desfecho de interesse (y), isto é, a função y = f(x) que origina o fenômeno real. Em problemas de saúde, por exemplo, pode ser a relação entre características clínicas de um paciente e a probabilidade de desenvolver determinada doença. Essa função, no entanto, é quase sempre desconhecida e difícil de ser observada em sua totalidade.

Modelo: trata-se de uma aproximação da função alvo, construída a partir dos dados disponíveis. Formalmente, o modelo gera uma predição $\hat{y} = f(x)$. Diferentes modelos, como árvores de decisão ou redes neurais, oferecem maneiras distintas de representar essa aproximação. Nenhum modelo é uma cópia perfeita da realidade, mas sim uma tentativa de capturar padrões relevantes o suficiente para que as previsões sejam úteis.

Algoritmo de aprendizado: é o conjunto de instruções que utilizamos para ajustar o modelo a partir dos dados. Ele define como os parâmetros internos do modelo são otimizados para que as predições se aproximem ao máximo dos valores observados. No fundo, o algoritmo é a "receita" que ensina o modelo a aprender com os exemplos apresentados, seja minimizando erros em uma função de perda ou maximizando alguma métrica de desempenho.

O grande desafio, entretanto, não é apenas construir um modelo que se ajuste bem aos dados já observados, mas avaliar sua **capacidade de generalização**. Isso significa estimar o quão bem o modelo se comportará diante de dados novos, nunca vistos anteriormente. Um modelo que memoriza demais os exemplos de treino pode ter baixo erro naquele conjunto, mas falhar completamente em situações reais – fenômeno conhecido como sobreajuste. Por outro lado, modelos muito simples podem não capturar padrões essenciais, resultando em sub-ajuste.

Estratégias de Avaliação

A avaliação de modelos em *machine learning* é fundamental para garantir que os resultados sejam confiáveis e possam ser aplicados de forma prática. Diferentes estratégias são desenvolvidas para medir a capacidade de generalização dos modelos, equilibrando simplicidade, eficiência e rigor estatístico.

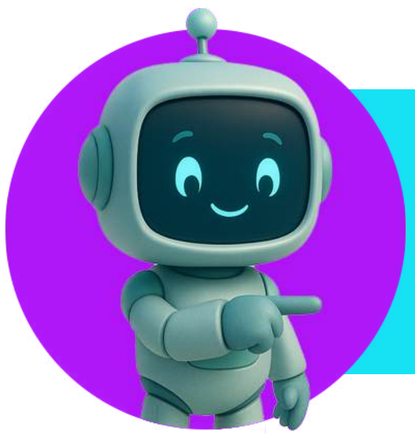
Método Holdout

O método holdout consiste em dividir o conjunto de dados em duas partes: uma para treinar o modelo e outra para testá-lo. Proporções como 70/30 ou 80/20 são comuns, mas a escolha depende do tamanho do banco de dados e da variabilidade do problema.

Apesar de simples, esse método apresenta duas limitações importantes:

Viés pessimista: quando a parte usada para treino é pequena, o modelo pode não captar todos os padrões relevantes, levando a uma avaliação mais negativa do que seria se houvesse mais dados disponíveis.

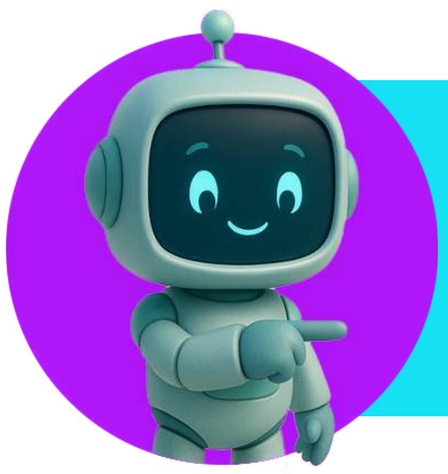
Alta variabilidade: diferentes formas de dividir treino e teste podem gerar resultados bastante distintos. Por exemplo, se por acaso todos os casos raros ficarem concentrados em apenas um dos conjuntos, a avaliação será distorcida.



Por isso, o holdout é útil em análises iniciais, quando se deseja uma visão rápida do desempenho, mas não deve ser usado como avaliação final. Para estimativas mais estáveis, a técnica mais utilizada é a **validação cruzada**, que reduz tanto o viés quanto a variabilidade.

Estratificação

Em saúde, é comum lidar com classes muito desbalanceadas. Imagine um estudo em que apenas 1% dos pacientes tem um determinado tipo de câncer. Se a divisão dos dados for feita de forma totalmente aleatória, pode acontecer de o conjunto de treino ou o de teste quase não conter pacientes com câncer, prejudicando o aprendizado do modelo e também a avaliação.



A solução é usar a **amostragem estratificada**, que garante que a proporção entre classes seja mantida tanto no treino quanto no teste. Assim, mesmo em situações de desbalanceamento extremo, como doenças raras ou complicações pouco frequentes, asseguramos que o modelo terá contato com exemplos relevantes e que a avaliação refletirá melhor o problema real.

FIQUE ATENTO!

Esse cuidado é ainda mais importante na área da saúde. Sem estratificação, um modelo pode parecer muito bom simplesmente porque aprendeu a predizer a classe majoritária (por exemplo, "não tem a doença"). Isso gera uma falsa sensação de desempenho, mas na prática pode significar diagnósticos errados, atrasos em tratamentos e risco para os pacientes.

Viés e Variância

Um dos conceitos mais fundamentais em *machine learning* é o equilíbrio entre viés e variância, conhecido como *trade-off* viés-variância. Esse dilema reflete a tensão entre modelos excessivamente simples e modelos excessivamente complexos. Nenhum modelo é perfeito, e todo processo de predição envolve algum nível de erro. Compreender de onde vem esse erro é necessário para desenvolver soluções mais robustas.

Viés: O viés representa erros sistemáticos introduzidos quando o modelo é incapaz de capturar a verdadeira relação entre as variáveis. Isso ocorre tipicamente em modelos muito simples, que reduzem demais a complexidade do problema. Esse cenário, chamado de **sub-ajuste**, resulta em previsões ruins tanto nos dados de treino quanto nos de teste.

Exemplo: utilizar apenas a média de idade dos pacientes para predizer risco cardiovascular. Esse modelo ignora variáveis como pressão arterial, colesterol, histórico familiar ou hábitos de vida, levando a erros previsíveis e recorrentes.

Variância: A variância está associada à instabilidade de modelos muito complexos, que aprendem não apenas os padrões reais dos dados, mas também o "ruído" – variações aleatórias e irrelevantes. Nesse caso, temos o **sobreajuste:** o modelo apresenta desempenho excelente nos dados de treino, mas falha ao generalizar para novos conjuntos.

Exemplo: uma árvore de decisão que cresce demais, ajustando-se perfeitamente a cada paciente do conjunto de treino, mas incapaz de predizer corretamente o risco de novos pacientes.

O ponto de equilíbrio

O desafio central em machine learning é encontrar um modelo que seja suficientemente complexo para captar padrões relevantes, mas não tão complexo a ponto de se tornar instável e pouco generalizável. Esse ponto de equilíbrio é identificado por meio de técnicas de validação e reamostragem, como validação cruzada, bootstrap e regularização. Essas estratégias permitem testar diferentes modelos e ajustar hiperparâmetros de forma controlada, aproximando o modelo do ponto ótimo entre viés e variância.

Implicações práticas em saúde

Na prática, esse equilíbrio é muito importante: um modelo com alto viés pode falhar em detectar pacientes em risco (falso-negativo), enquanto um modelo com alta variância pode gerar muitos alarmes falsos (falso-positivo). Em sistemas de saúde, ambos os cenários trazem consequências sérias, desde diagnósticos perdidos até sobrecarga de exames desnecessários. Por isso, compreender e gerenciar o trade-off viés-variância não é apenas uma questão técnica, mas também ética e de impacto direto na atenção à saúde.

Validação Cruzada e Hiperparâmetros

Para avaliar de forma robusta o desempenho de modelos em machine learning, uma das estratégias mais utilizadas é a validação cruzada, que permite reduzir a influência do acaso na separação treino/teste e, ao mesmo tempo, fornece uma estimativa mais confiável da sua capacidade de generalização.

k-Fold Cross-Validation

Na validação cruzada do tipo k-fold, o conjunto de dados é dividido em k partes (ou folds) de tamanho aproximadamente igual. O modelo é então treinado k vezes, e a cada rodada um dos folds é reservado para validação, enquanto os demais servem para o treino. O desempenho final do modelo é calculado como a média dos erros obtidos em cada uma das k iterações. O desempenho final é a média dos k testes:

$$Erro\ Médio = \frac{1}{k}\sum_{i=1}^{k} \text{Erro}_i$$

Esse processo garante que cada observação seja usada tanto para treino quanto para validação, mas nunca nas duas funções ao mesmo tempo. Diversos estudos mostraram que escolher k = 10 representa um bom compromisso entre viés e variância: valores menores (como k = 5) podem produzir estimativas mais rápidas, porém menos estáveis, enquanto valores maiores (como leave-one-out) tendem a ter variância elevada e custo computacional muito maior.

Seleção de Hiperparâmetros

Outro passo importante é a escolha dos hiperparâmetros, que são configurações externas ao modelo e definem sua complexidade e comportamento. Diferentemente dos parâmetros internos (como os coeficientes em uma regressão ou os pesos em uma rede neural), os hiperparâmetros não são aprendidos diretamente a partir dos dados e precisam ser ajustados por meio de experimentação.

Entre os principais métodos de busca de hiperparâmetros, destacam-se:

1. **Grid Search:** avalia de forma exaustiva todas as combinações possíveis de valores dentro de um conjunto previamente definido. É simples e garante a exploração completa do espaço especificado, mas pode ser extremamente custoso em termos computacionais.

2 **Random Search:** em vez de testar todas as combinações, seleciona pontos aleatórios do espaço de hiperparâmetros. Essa abordagem costuma ter boa eficiência, pois evita gastar tempo em regiões pouco promissoras.

3 **Otimização Bayesiana:** utiliza modelos probabilísticos para guiar a busca. A ideia é estimar quais regiões do espaço de hiperparâmetros têm maior probabilidade de conter boas soluções e direcionar os experimentos nessa direção. Essa técnica é mais sofisticada e, em geral, mais eficiente quando o espaço de busca é amplo ou quando cada treino de modelo é muito custoso.

Métricas em Regressão

Nos problemas de regressão, em que o objetivo é predizer valores contínuos (como a pressão arterial de um paciente, o número de internações esperadas em uma região ou o tempo de sobrevida após um diagnóstico), é necessário avaliar não apenas se o modelo acerta aproximadamente, mas também como e em que magnitude erra. Para isso, utilizamos métricas que quantificam a diferença entre os valores observados e os valores preditos.

- **Erro Quadrático Médio (MSE):**

$$Erro\ Quadrático\ Médio = \frac{1}{n}\sum_{i=1}^{n}(y - \hat{y}_i)^2$$

O MSE mede o erro médio ao quadrado, dando maior peso a discrepâncias grandes. Por isso, ele é bastante útil quando queremos penalizar fortemente previsões muito distantes da realidade. Em saúde, isso pode ser vantajoso em contextos como predição de glicemia ou dosagem de medicamentos, onde grandes erros podem ser clinicamente críticos. Porém, por elevar ao quadrado, o MSE é altamente influenciado por outliers.

- **Raiz do Erro Quadrático Médio (RMSE):**

$$Erro\ Quadrático\ Médio = \sqrt{\text{MSE}}$$

O RMSE é simplesmente a raiz quadrada do MSE. Sua vantagem é trazer o erro de volta para a mesma escala da variável de interesse, o que facilita a interpretação. Por exemplo, se estamos prevendo tempo de internação em dias e o RMSE for 2, significa que, em média, os erros do modelo estão na ordem de 2 dias. Assim como o MSE, também é bastante sensível a valores extremos.

- **Erro Absoluto Médio (MAE):**

$$Erro\ Absoluto\ Médio = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

O MAE calcula a média das diferenças absolutas, sem elevar ao quadrado. Dessa forma, cada erro contribui proporcionalmente ao resultado final, tornando o MAE mais robusto a outliers. Ele é especialmente útil em contextos em que os valores extremos podem estar ligados a ruídos de medição ou situações excepcionais, como erros laboratoriais em exames clínicos.

Métricas em Classificação

Nos problemas de classificação, em que o objetivo é prever categorias (por exemplo, presença ou ausência de uma doença), a avaliação do modelo vai além de simplesmente contar quantas vezes ele acerta.

> É preciso analisar os diferentes tipos de erro que podem ocorrer, especialmente porque, em saúde, **um falso negativo pode ter consequências muito diferentes de um falso positivo**.

Matriz de Confusão

A matriz de confusão é a base para praticamente todas as métricas de classificação. Ela organiza os resultados possíveis em quatro grupos:

Verdadeiros Positivos (TP):
casos em que o modelo previu corretamente a presença da condição.

Falsos Negativos (FN):
casos em que o modelo não identificou a condição, embora ela estivesse presente.

Falsos Positivos (FP):
casos em que o modelo indicou a presença da condição, mas na realidade ela não existia.

Verdadeiros Negativos (TN):
casos em que o modelo previu corretamente a ausência da condição.

Principais métricas:

A matriz de confusão é a base para praticamente todas as métricas de classificação. Ela organiza os resultados possíveis em quatro grupos:

- **Acurácia:**

$$Acurácia = (\mathrm{TP} + \mathrm{TN}) / (\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN})$$

Representa a proporção total de acertos do modelo. Embora seja intuitiva, pode ser **enganosa em cenários de classes desbalanceadas**. Por exemplo, se apenas 1% dos pacientes têm câncer e o modelo sempre prevê “não tem câncer”, ele terá 99% de acurácia, mas será inútil na prática.

- **Sensibilidade (Recall):**

$$Sensibilidade = \mathrm{TP} / (\mathrm{TP} + \mathrm{FN})$$

Mede a capacidade do modelo de identificar corretamente os casos positivos. É frequentemente usado na saúde, já que indica a fração de pacientes doentes que realmente foram detectados. Alta sensibilidade reduz falsos negativos, algo crítico em diagnósticos precoces de doenças graves.

- **Especificidade:**

$$Especificidade = \mathrm{TN} \,/\, (\mathrm{TN} + \mathrm{FP})$$

Avalia a capacidade do modelo de reconhecer os casos negativos. Alta especificidade significa poucos falsos positivos, importante em exames que envolvem procedimentos caros ou invasivos, como biópsias.

- **Precisão (Precision):**

$$Prec = TP \,/\, (TP + FP)$$

Mostra a proporção de previsões positivas que estavam corretas. Ou seja, dado que o modelo indicou "doente", qual a chance de isso ser verdade? É especialmente relevante quando o custo de um falso positivo é elevado (ex.: indicar uma cirurgia desnecessária)

- **F1-Score:**

$$F1 = 2 * (\mathrm{Precisão} * \mathrm{Sensibilidade}) \,/\, (\mathrm{Precisão} + \mathrm{Sensibilidade})$$

É a média harmônica entre precisão e sensibilidade. Funciona como um equilíbrio quando precisamos considerar tanto os falsos positivos quanto os falsos negativos. Em contextos de saúde, é bastante usado quando queremos modelos que sejam ao mesmo tempo sensíveis e precisos.

Interpretação prática em saúde

- Em **rastreamento populacional** (ex.: triagem de câncer de colo do útero), normalmente prioriza-se **alta sensibilidade**, para reduzir falsos negativos e não deixar casos reais sem diagnóstico.
- Em **testes confirmatórios** (ex.: exame de alta precisão antes de indicar cirurgia), prioriza-se **alta especificidade e precisão**, para minimizar falsos positivos.
- Quando existe a necessidade de equilíbrio entre os dois extremos, o **F1-score** pode ser uma boa métrica de referência.

Assim, a escolha da métrica depende menos de uma "receita universal" e mais do **contexto clínico ou epidemiológico**, equilibrando o impacto de cada tipo de erro na prática real.

Métricas de Ranking

Nos problemas de classificação em saúde, os modelos não retornam apenas uma previsão binária ("doente" ou "não doente"), mas sim uma **probabilidade** associada a cada indivíduo. Isso permite ordenar (ou ranquear) os casos de acordo com o risco predito, sendo muito útil em contextos como triagem de pacientes, alocação de recursos hospitalares ou priorização de exames. Para avaliar esse tipo de saída probabilística, usamos métricas baseadas em **curvas de desempenho**.

Curva ROC e AUC

A **curva ROC** (Receiver Operating Characteristic) mostra a relação entre a **taxa de verdadeiros positivos (TPR)** e a **taxa de falsos positivos (FPR)** para diferentes limiares de decisão:

$$TPR = TP \,/\, (TP + FN)$$

$$FPR = FP \,/\, (FP + TN)$$

Cada ponto da curva representa um possível limiar de corte (por exemplo, considerar "doente" se a probabilidade for maior que 0,3; 0,5; 0,7 etc.). A área sob a curva ROC, chamada AUC-ROC, mede a capacidade de o modelo distinguir entre as classes. Um AUC de 0,5 equivale a um chute aleatório, enquanto um AUC de 1,0 indica discriminação perfeita.

Na prática em saúde:

- Um **AUC próximo de 0,9** sugere excelente discriminação, como em modelos para diagnóstico de doenças já bem estudadas.
- Um **AUC em torno de 0,7** pode ser aceitável em contextos mais desafiadores, como predição de doenças raras ou desfechos multifatoriais.
- O grande valor do AUC-ROC é que ele é independente do ponto de corte escolhido, avaliando o desempenho global do modelo em diferentes cenários.

Métricas de Probabilidade e Calibração

Em muitas aplicações de machine learning em saúde, não basta classificar corretamente os pacientes em grupos de risco, é fundamental que as probabilidades preditas sejam confiáveis. Afinal, decisões médicas e de políticas públicas frequentemente dependem de valores absolutos de risco. Por exemplo, indicar ou não um tratamento invasivo, decidir a prioridade em listas de espera, ou calcular o número esperado de leitos em uma epidemia.

Calibração de Modelos

Um modelo é considerado **bem calibrado** quando suas probabilidades preditas refletem a frequência real dos eventos. Por exemplo, se em um grupo de pacientes o modelo estima risco de 70% para óbito em 30 dias, espera-se que aproximadamente 70% de fato venham a óbito nesse período. Da mesma forma, para um risco predito de 20%, o evento deveria ocorrer em torno de 20% dos pacientes.

Esse alinhamento é diferente da **discriminação** (avaliada por métricas como AUC-ROC), que mede apenas a capacidade do modelo de distinguir entre quem terá e quem não terá o desfecho. É possível ter um modelo com excelente discriminação, mas má calibração – e vice-versa. Em saúde, ambos são igualmente importantes: discriminação indica quem está em maior risco, calibração indica quanto é esse risco.

Métrica de Calibração

- **Brier Score:** mede o erro quadrático médio entre as probabilidades previstas e os desfechos reais. Quanto menor o valor, melhor a calibração.

$$Brier\ Score = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)$$

Importância em Saúde

A calibração é importante porque decisões de saúde são muitas vezes sensíveis à magnitude exata do risco:

- Exames preventivos: um risco de 10% pode justificar exames adicionais, mas 2% talvez não.
- Terapias intensivas: em modelos de mortalidade hospitalar, uma predição mal calibrada pode superlotar UTIs ou, ao contrário, negar suporte a quem realmente precisa.
- Política pública: previsões de risco mal calibradas podem levar a erros de planejamento, como subestimar o número de vacinas necessárias em uma região.

Rebalanceamento de Classes

Em problemas de machine learning aplicados à saúde, é muito comum lidar com dados desbalanceados, em que a maioria dos pacientes não apresenta o desfecho de interesse (classe majoritária), enquanto apenas uma pequena fração realmente o apresenta (classe minoritária). Exemplos típicos incluem predição de doenças raras, identificação de complicações pós-cirúrgicas ou detecção de óbitos em bases hospitalares. Quando esse desbalanceamento é ignorado, os algoritmos tendem a favorecer a classe majoritária, resultando em modelos que apresentam alta acurácia aparente, mas são incapazes de identificar adequadamente os casos mais importantes.

Técnicas de Rebalanceamento

- **Down-sampling (subamostragem da classe majoritária):** reduz artificialmente o número de exemplos da classe mais frequente, tornando a base mais equilibrada. Isso diminui o viés em favor da classe majoritária, mas pode descartar informações úteis, especialmente quando o conjunto de dados é pequeno.
  Exemplo: em um banco com 10.000 pacientes sem câncer e apenas 200 com câncer, poderíamos selecionar aleatoriamente 200 ou 400 negativos para manter uma proporção mais equilibrada.
- **Up-sampling (superamostragem da classe minoritária):** replica exemplos da classe menos representada até atingir uma proporção mais próxima da classe majoritária. Isso aumenta a chance do modelo aprender padrões da minoria, mas pode levar ao overfitting, já que os mesmos exemplos são repetidos muitas vezes.
- **SMOTE (Synthetic Minority Oversampling Technique):** em vez de replicar os exemplos minoritários, o SMOTE gera **novos exemplos sintéticos** com base na interpolação de vizinhos próximos. Essa técnica enriquece o espaço de treinamento, oferecendo mais diversidade à classe minoritária e reduzindo o risco de sobreajuste. Existem variações como Borderline-SMOTE e ADASYN, que focam em gerar exemplos mais informativos, próximos às fronteiras de decisão.

Evidências e Impacto nos Algoritmos

Estudos recentes mostram que o rebalanceamento traz mais benefícios em algoritmos considerados “fracos”, como máquinas de vetores de suporte (SVM) e árvores de decisão simples, pois esses métodos tendem a ser muito sensíveis ao desbalanceamento. Já modelos mais sofisticados e “fortes”, como XGBoost, LightGBM e CatBoost, muitas vezes já incorporam estratégias internas de ponderação de classes e, portanto, podem não apresentar ganhos significativos – ou até sofrer degradação de desempenho se o rebalanceamento for mal aplicado.

Conclusão

Ao longo deste capítulo percorremos os principais conceitos, estratégias e métricas que sustentam a aplicação de *machine learning* em saúde, sempre com foco na avaliação crítica de modelos. Vimos que não basta treinar algoritmos, é importante compreender fundamentos como o trade-off entre viés e variância, escolher métodos de validação robustos, ajustar hiperparâmetros de forma criteriosa e selecionar métricas adequadas ao problema em questão. Exploramos ainda a importância de avaliar modelos sob diferentes perspectivas, seja pelo erro em regressão, pela matriz de confusão em classificações binárias, pelas curvas de ranking em problemas probabilísticos, ou pela calibração em cenários onde decisões clínicas dependem de estimativas de risco confiáveis.

Discutimos também os desafios do desbalanceamento de classes, uma realidade comum em dados de saúde, e as diversas técnicas para corrigi-lo ou atenuar seus efeitos. Ficou claro que a avaliação não é apenas um passo técnico, mas uma etapa central que define a utilidade real de um modelo. Em saúde, um algoritmo mal avaliado pode gerar consequências éticas e práticas graves: atrasos em diagnósticos, tratamentos desnecessários ou falhas no planejamento de recursos.

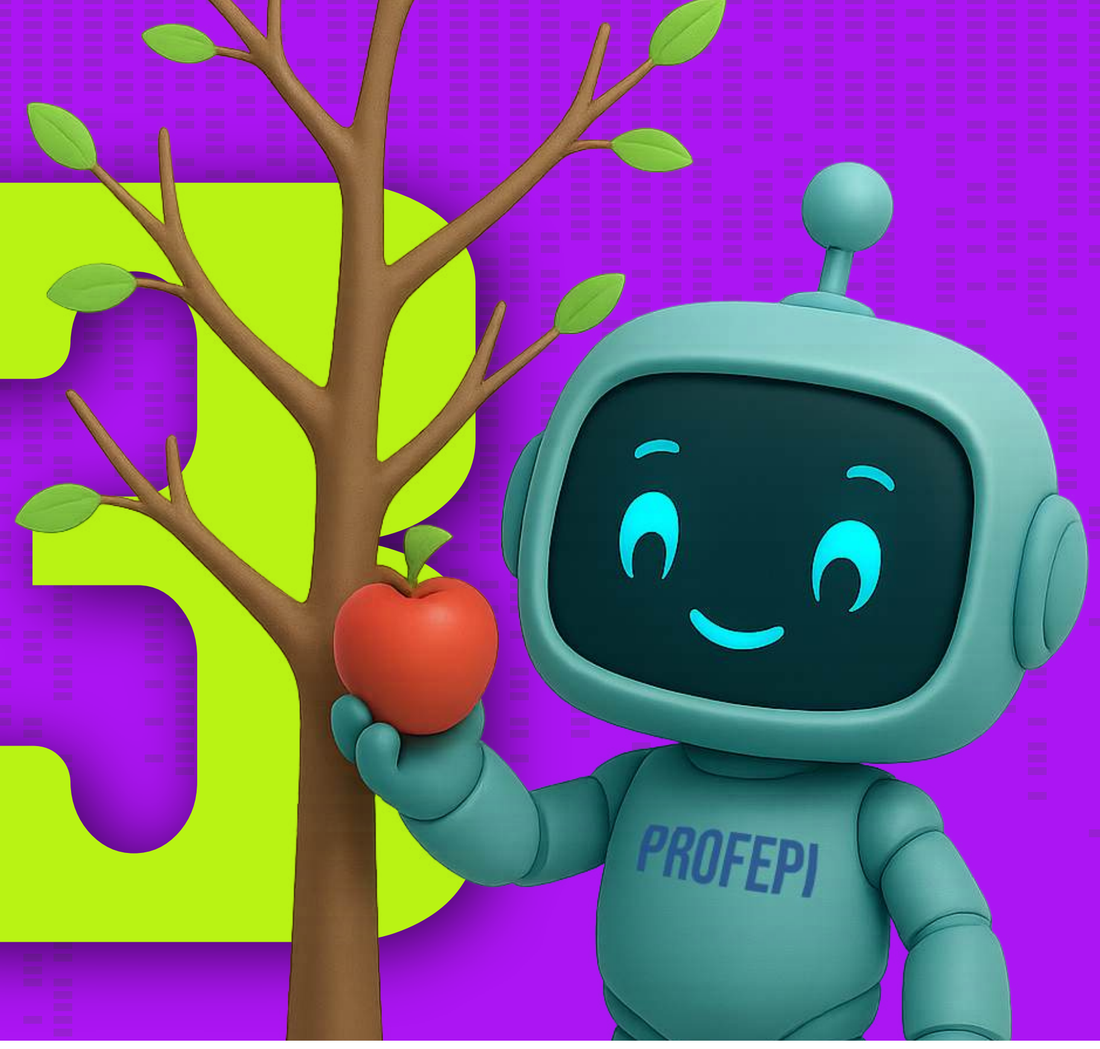
Mais do que buscar modelos sofisticados ou métricas isoladas, precisamos adotar uma postura crítica e contextualizada, reconhecendo limitações, entendendo implicações clínicas e sociais, e priorizando sempre a **generalização e a confiabilidade** das previsões. *Machine learning* só se transforma em ferramenta de impacto positivo na saúde quando aliado a rigor metodológico, avaliação cuidadosa e compromisso com a prática real.

Esse equilíbrio entre teoria, técnica e aplicação é o que sustenta o avanço da inteligência artificial na saúde, permitindo que modelos não apenas façam previsões, mas contribuam para decisões mais seguras, efetivas e justas.



capítulo 6

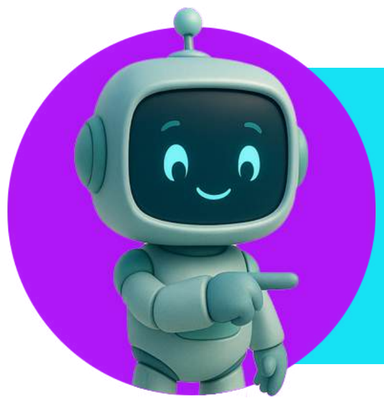
MÓDULO 3



Algoritmos Baseados em Árvores

Introdução aos Algoritmos

A palavra *algoritmo* refere-se a uma sequência finita e bem definida de instruções que, ao serem executadas, permitem resolver um problema ou realizar uma tarefa específica (Cormen et al., 2009).



No contexto da ciência de dados e da inteligência artificial (IA), os algoritmos constituem o núcleo do processo de aprendizado, sendo responsáveis por ajustar modelos a partir de dados e gerar previsões, classificações ou recomendações.

No campo da vigilância em saúde e ambiente, os algoritmos têm desempenhado um papel cada vez mais relevante, dada a crescente disponibilidade de grandes volumes de dados oriundos de sistemas de informação, como o Sistema de Informação sobre Mortalidade (SIM), o Sistema de Informação de Agravos de Notificação (SINAN) e plataformas de monitoramento climático. Por meio de técnicas de aprendizado de máquina (machine learning), esses algoritmos podem identificar padrões complexos, detectar anomalias e prever eventos futuros que auxiliem na tomada de decisão em saúde pública.

Entre as aplicações práticas destacam-se:

Previsão de surtos e epidemias: algoritmos preditivos podem antecipar o aumento de casos de doenças transmissíveis, como dengue e influenza, integrando dados de notificação, clima e mobilidade populacional (Nsoesie et al., 2014).

Antecipação de riscos ambientais: modelos treinados com dados de poluição do ar, temperatura e umidade relativa podem estimar a probabilidade de elevação de internações por doenças respiratórias (Guo et al., 2018).

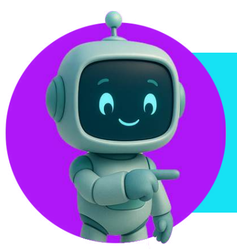
Detecção de padrões em grandes bases de dados: algoritmos de mineração de dados permitem identificar áreas geográficas ou populações de maior vulnerabilidade a doenças crônicas não transmissíveis, como hipertensão e diabetes (Chiavegatto Filho et al., 2020).

A relevância dos algoritmos para a saúde pública decorre de sua capacidade de generalizar – ou seja, aprender padrões a partir de dados históricos e aplicá-los para prever situações futuras, inclusive em contextos não observados anteriormente. Essa característica é fundamental para que políticas e ações sejam implementadas de forma proativa, reduzindo a morbimortalidade e otimizando recursos.

No entanto, a aplicação de algoritmos em saúde exige atenção a aspectos como a qualidade dos dados, a interpretabilidade dos modelos e a viabilidade computacional. Em ambientes onde as decisões afetam diretamente a vida das pessoas, a precisão preditiva deve ser equilibrada com a transparência e a robustez das soluções desenvolvidas (WHO, 2021).

Por que existem diferentes algoritmos?

A diversidade de algoritmos em machine learning decorre, principalmente, da variedade de tipos de dados, problemas e objetivos de aplicação.



Cada algoritmo apresenta pontos fortes e limitações que influenciam seu desempenho em diferentes contextos.

Não existe um modelo universalmente superior para todos os problemas – conceito formalizado pelo Teorema do "No Free Lunch" (Wolpert & Macready, 1997). Esse teorema estabelece que, considerando todos os possíveis conjuntos de dados, nenhum algoritmo é garantidamente melhor que outro sem conhecer previamente a estrutura e as propriedades dos dados.

Diferenças na natureza dos dados

Dados tabulares:

Comuns em bancos administrativos e de vigilância em saúde, como registros de hospitalizações e notificações de doenças. Modelos baseados em árvore (p. ex., Random Forest, XGBoost) e regressão logística são frequentemente eficazes.

Dados de imagem:

Relevantes em diagnósticos radiológicos e exames de patologia digital, geralmente processados por redes neurais convolucionais (Convolutional Neural Networks – CNNs).

Dados textuais:

Utilizados em prontuários eletrônicos ou redes sociais para vigilância sindrômica, processados com técnicas de processamento de linguagem natural (*Natural Language Processing* – NLP).

Séries temporais:

Úteis para previsão de tendências epidemiológicas, como curvas de incidência de doenças ou evolução de parâmetros ambientais.

Diferenças nos objetivos e métricas

- Os algoritmos também se distinguem em função dos objetivos de aplicação:
- Alta precisão: modelos mais complexos, como *gradient boosting*, tendem a apresentar maior acurácia em dados tabulares, porém podem sacrificar interpretabilidade.
- Interpretação clínica: modelos lineares ou árvores de decisão simples facilitam a explicação de resultados para gestores e profissionais de saúde.
- Velocidade e escalabilidade: algoritmos como LightGBM ou regressões regulares são mais rápidos e eficientes para grandes bases.

Exemplo aplicado à saúde

- Em um projeto de vigilância epidemiológica para prever internações por doenças respiratórias a partir de dados de poluição atmosférica, temperatura e umidade, diferentes algoritmos podem ser comparados:
- Regressão linear múltipla: oferece boa interpretabilidade, mas pode ter desempenho inferior se as relações forem não lineares.
- Árvore de decisão: capta interações não lineares, porém pode sobreajustar os dados.
- Random Forest: aumenta a robustez ao combinar múltiplas árvores, mas perde parte da interpretabilidade.
- XGBoost ou LightGBM: podem atingir maior acurácia, especialmente após ajuste fino de hiperparâmetros, mas exigem maior poder computacional.

Implicações para a vigilância em saúde

A escolha do algoritmo depende de um equilíbrio entre:

- Precisão do modelo
- Facilidade de explicação para tomada de decisão
- Viabilidade computacional
- Qualidade e quantidade de dados disponíveis

FIQUE ATENTO!

Em saúde pública, a decisão final sobre qual modelo adotar deve considerar não apenas métricas estatísticas, mas também aspectos éticos, transparência e a possibilidade de implementação prática (WHO, 2021).

Como os algoritmos "aprendem"

O processo de aprendizado em machine learning consiste no ajuste de parâmetros internos de um modelo a partir de dados, com o objetivo de minimizar o erro entre as previsões e os valores reais. Esse processo, denominado treinamento, permite que o algoritmo identifique padrões nos dados de forma a realizar generalizações – ou seja, fornecer previsões precisas mesmo para dados que nunca foram observados anteriormente (Mitchell, 1997).

Função de perda

O ponto central do aprendizado é a função de perda (loss function), que quantifica o erro cometido pelo modelo. O objetivo do algoritmo é encontrar os parâmetros que minimizam essa função. Exemplos comuns incluem:

- **Erro Quadrático Médio (MSE):** usado em regressões para medir a média dos quadrados das diferenças entre valores previstos e observados.
- **Entropia cruzada (cross-entropy):** amplamente utilizada em problemas de classificação, penalizando fortemente previsões incorretas com alta confiança.
- **Log loss:** semelhante à entropia cruzada, também empregada em classificação binária.

Exemplo na saúde

Ao prever risco de mortalidade hospitalar, a função de perda pode ser configurada para penalizar mais fortemente os falsos negativos (pacientes de alto risco classificados como baixo risco), de modo a priorizar a segurança do paciente.

Métodos de otimização

Para minimizar a função de perda, os algoritmos utilizam métodos de otimização, sendo o gradiente descendente (gradient descent) o mais comum. Ele ajusta iterativamente os parâmetros na direção de maior redução do erro. Variantes incluem:

Gradiente descendente estocástico (SGD): utiliza subconjuntos aleatórios de dados a cada iteração, acelerando o treinamento.

Adam: combina momento e adaptação da taxa de aprendizado, sendo eficaz em problemas complexos.

Overfitting e underfitting

- **Overfitting:** ocorre quando o modelo aprende detalhes e ruídos específicos do conjunto de treino, perdendo capacidade de generalizar. É comum em modelos muito complexos ou com excesso de treinamento.
- **Underfitting:** acontece quando o modelo é muito simples para capturar a complexidade dos dados, resultando em baixo desempenho tanto no treino quanto no teste.

Exemplo na saúde

Um modelo que prevê internações por doenças respiratórias com base em dados climáticos pode apresentar overfitting se incluir ruídos sazonais irrelevantes, enquanto um modelo linear simples pode apresentar underfitting ao não capturar interações não lineares entre poluição, temperatura e umidade.

Validação cruzada

A validação cruzada (cross-validation) é uma técnica essencial para avaliar o desempenho real de um modelo. A forma mais utilizada é a k-fold cross-validation, que divide o conjunto de dados em k partes (dobras), usando k-1 partes para treino e a restante para teste, repetindo o processo k vezes e calculando a média das métricas obtidas (Kohavi, 1995).

Anota essa informação!

Essa abordagem é particularmente importante em vigilância em saúde, pois reduz a probabilidade de que o desempenho observado seja fruto do acaso, aumentando a confiança nas previsões.

Generalização em contextos de saúde pública

A capacidade de um modelo generalizar é crucial para prever eventos futuros de forma confiável, como:

- Prever aumento de casos de dengue com base em dados climáticos e históricos de incidência.
- Antecipar picos de internação por doenças cardiovasculares considerando variáveis meteorológicas e poluentes atmosféricos.
- Estimar demanda por leitos hospitalares durante epidemias de influenza.

Em todos esses casos, é fundamental garantir que o modelo mantenha desempenho estável quando aplicado a dados de períodos e contextos distintos daqueles utilizados no treinamento.

Algoritmos Baseados em Árvore

Os algoritmos baseados em árvore constituem uma das abordagens mais utilizadas em machine learning, especialmente quando se busca equilíbrio entre desempenho preditivo e interpretabilidade. Eles funcionam estruturando o processo de decisão em uma série de divisões binárias ou múltiplas, formando um modelo visualmente representado como uma árvore invertida, onde cada nó interno representa uma decisão, os ramos representam caminhos possíveis e as folhas contêm a previsão final (classe ou valor).

A popularidade desses modelos na vigilância em saúde e ambiente decorre da sua capacidade de lidar com dados tabulares heterogêneos, combinando variáveis numéricas, categóricas e até mesmo variáveis derivadas de fontes distintas, como dados de clima, poluição e notificações de doenças. Diferentemente de modelos lineares, as árvores de decisão captam interações complexas e não lineares entre variáveis sem a necessidade de transformação prévia dos dados (Breiman et al., 1984; Quinlan, 1993).

Funcionamento e construção

O treinamento de uma árvore de decisão segue um processo recursivo. Inicialmente, todos os dados estão em um nó raiz. O algoritmo então avalia todas as variáveis preditoras e seus possíveis pontos de corte, selecionando a divisão que maximize a separação das classes (em classificação) ou minimize a variação residual (em regressão). Esse critério de separação pode ser medido por métricas como Índice de Gini, entropia ou erro quadrático médio.

Após a primeira divisão, o processo é repetido em cada subconjunto resultante, criando novos nós e ramos até que sejam atendidas condições de parada, como:

- Profundidade máxima da árvore.
- Número mínimo de observações por nó.
- Ganho mínimo de pureza para novas divisões.

Exemplo na saúde

Na predição de risco de internação por doenças respiratórias, o nó raiz poderia ser uma variável como "Idade ≥ 60 anos". Para indivíduos acima dessa idade, o modelo poderia avaliar o histórico de doença cardiovascular; já para indivíduos mais jovens, poderia priorizar variáveis ambientais como nível médio de poluição atmosférica nos últimos sete dias.

Os algoritmos baseados em árvore constituem uma das abordagens mais utilizadas em machine learning, especialmente quando se busca equilíbrio entre desempenho preditivo e interpretabilidade. Eles funcionam estruturando o processo de decisão em uma série de divisões binárias ou múltiplas, formando um modelo visualmente representado como uma árvore invertida, onde cada nó interno representa uma decisão, os ramos representam caminhos possíveis e as folhas contêm a previsão final (classe ou valor).

A popularidade desses modelos na vigilância em saúde e ambiente decorre da sua capacidade de lidar com dados tabulares heterogêneos, combinando variáveis numéricas, categóricas e até mesmo variáveis derivadas de fontes distintas, como dados de clima, poluição e notificações de doenças. Diferentemente de modelos lineares, as árvores de decisão captam interações complexas e não lineares entre variáveis sem a necessidade de transformação prévia dos dados (Breiman et al., 1984; Quinlan, 1993).

Interpretação e aplicabilidade

Um dos principais diferenciais das árvores de decisão é a interpretabilidade. A estrutura resultante pode ser traduzida em um conjunto de regras lógicas do tipo se-então, facilmente compreendidas por profissionais de saúde e gestores. Essa característica é especialmente valiosa quando decisões rápidas e transparentes precisam ser tomadas, como na priorização de recursos hospitalares ou na definição de áreas para intervenção epidemiológica.

Além disso, árvores de decisão não exigem normalização ou padronização de variáveis e são naturalmente robustas a outliers, já que decisões são tomadas com base em divisões e não em médias ou variâncias sensíveis a valores extremos.

Limitações

Apesar das vantagens, árvores de decisão apresentam instabilidade: pequenas alterações no conjunto de treinamento podem resultar em árvores completamente diferentes. Outro problema frequente é o sobreajuste (overfitting), quando a árvore cresce excessivamente e passa a capturar ruídos específicos dos dados de treino, prejudicando o desempenho em novos dados.

Além disso, embora sejam interpretáveis, árvores profundas podem gerar regras excessivamente complexas, dificultando sua comunicação.

Estratégias para melhorar desempenho

Para contornar essas limitações, desenvolveu-se o conceito de ensemble learning – métodos que combinam múltiplas árvores para melhorar robustez e acurácia.

As duas principais abordagens são:

Bagging (Bootstrap Aggregating): construção de diversas árvores independentes a partir de amostras aleatórias dos dados, como no Random Forest.

Boosting: construção sequencial de árvores, onde cada nova árvore corrige erros das anteriores, como nos modelos Gradient Boosting Decision Trees (GBDT), implementados no XGBoost, LightGBM e CatBoost.

Essas técnicas não apenas melhoram o desempenho preditivo, mas também aumentam a capacidade de generalização, especialmente quando aplicadas em bases de dados complexas e heterogêneas, comuns na saúde pública.

Exemplos na vigilância em saúde

O uso de algoritmos baseados em árvore é amplo e documentado na literatura:

- Monitoramento de epidemias: árvores de decisão foram usadas para prever surtos de dengue, integrando variáveis meteorológicas, dados de infestação de mosquitos e notificações oficiais (Zhu et al., 2019).
- Predição de mortalidade hospitalar: Random Forest e GBDT já foram aplicados para prever risco de morte por COVID-19 com base em dados clínicos e demográficos, auxiliando no manejo de pacientes em pronto-socorro (Vaid et al., 2020).
- Gestão de recursos hospitalares: modelos de árvore têm sido usados para prever demanda por leitos e ventiladores em períodos de pico de doenças respiratórias sazonais.

Métodos de Ensemble Baseados em Árvores

Apesar de serem intuitivas e interpretáveis, as árvores de decisão simples possuem limitações significativas, como instabilidade e propensão ao overfitting. Para superar esses problemas e melhorar a capacidade de generalização, surgiram os métodos de ensemble, que combinam múltiplas árvores para gerar modelos mais robustos e precisos (Dietterich, 2000).

Entre as abordagens mais populares estão o bagging (bootstrap aggregating) e o boosting, cada uma com variações e implementações específicas amplamente utilizadas na vigilância em saúde.

Bagging e Random Forest

O bagging consiste em treinar múltiplas árvores de decisão de forma independente, cada uma utilizando uma amostra aleatória (com reposição) do conjunto de dados original. Essa técnica reduz a variância do modelo e melhora sua estabilidade, já que erros cometidos por árvores individuais tendem a se compensar na agregação final (Breiman, 1996).

A implementação mais conhecida dessa abordagem é o Random Forest (Breiman, 2001), que introduz uma segunda camada de aleatoriedade: em cada nó, apenas um subconjunto aleatório de variáveis é considerado para determinar a divisão. Isso aumenta a diversidade entre as árvores e melhora a performance.

Vantagens:

- Alta robustez contra *overfitting*.
- Funciona bem com variáveis categóricas e numéricas.
- Fornece medidas de importância das variáveis.

Exemplo na saúde

Modelos de Random Forest foram utilizados para prever risco de mortalidade hospitalar em pacientes com COVID-19, combinando dados demográficos, comorbidades e resultados laboratoriais. O modelo auxiliou na priorização de pacientes para tratamento intensivo (Vaid et al., 2020).

Boosting e Gradient Boosting Decision Trees (GBDT)

O boosting adota uma abordagem sequencial: cada nova árvore é treinada para corrigir os erros da anterior, ajustando-se aos exemplos mais difíceis de prever. Esse processo é guiado pela minimização de uma função de perda por meio de gradiente descendente, resultando nos Gradient Boosting Decision Trees (GBDT) (Friedman, 2001).

As três implementações mais conhecidas e eficientes do GBDT são XGBoost, LightGBM e CatBoost, cada uma com características particulares.

XGBoost (Extreme Gradient Boosting)

O XGBoost (Chen & Guestrin, 2016) é otimizado para velocidade e desempenho, incorporando:

- Processamento paralelo.
- Regularização L1 e L2 para controle de overfitting.
- Suporte a valores ausentes e dados esparsos.
- Flexibilidade para usar diferentes funções de perda.

Vantagens:

- Alta acurácia, especialmente em dados tabulares.
- Estável em bases complexas.
- Ampla documentação e comunidade ativa.

Limitações:

- Mais lento que LightGBM em bases muito grandes.
- Exige ajuste fino de hiperparâmetros.

Exemplo na saúde

Usado na previsão de surtos de dengue com base em variáveis meteorológicas, índices de infestação vetorial e notificações de casos, atingindo alta sensibilidade na detecção precoce (Zhu et al., 2019).

LightGBM

Desenvolvido pela Microsoft (Ke et al., 2017), o LightGBM é projetado para alta performance em grandes volumes de dados. Ele utiliza uma estratégia de crescimento de árvore leaf-wise, priorizando a expansão dos ramos que mais reduzem o erro, o que o torna mais rápido e eficiente.

Vantagens:

- Treinamento extremamente rápido.
- Boa eficiência de memória.
- Suporte nativo a variáveis categóricas.

Limitações:

- Mais suscetível a overfitting se não houver regularização adequada.
- Menor interpretabilidade comparado a modelos mais simples.

Exemplo na saúde

Aplicado para modelar risco de desenvolvimento de hipertensão em coortes populacionais, usando variáveis clínicas e comportamentais, permitindo intervenções preventivas direcionadas (Ye et al., 2020).

CatBoost

O CatBoost (Dorogush et al., 2018), desenvolvido pela Yandex, foi projetado para lidar de forma nativa com variáveis categóricas, eliminando a necessidade de codificações complexas (*one-hot encoding*). Utiliza técnicas como ordered boosting para evitar vazamento de informação (*target leakage*).

Vantagens:

- Excelente desempenho com dados categóricos.
- Menor necessidade de pré-processamento.
- Boa performance em bases pequenas e médias.

Limitações:

- Menor comunidade e material de suporte que XGBoost e LightGBM.
- Pode ser mais lento que LightGBM em grandes volumes de dados numéricos.

Exemplo na saúde

Empregado para prever risco de internação por doenças crônicas em bases administrativas que combinam variáveis demográficas, socioeconômicas e clínicas.

Comparação entre os métodos

Algoritmo	Velocidade	Memória	Ideal para	Limitações principais
Random Forest	Média	Média	Modelos interpretáveis, análise de importância de variáveis	Pode ser menos preciso que GBDT
XGBoost	Alta	Alta	Dados tabulares complexos	Mais lento em bases muito grandes
LightGBM	Muito alta	Alta	Grandes bases de dados	Mais suscetível a overfitting
CatBoost	Alta	Alta	Bases com muitas variáveis categóricas	Comunidade menor

Considerações práticas na saúde

A escolha do algoritmo ideal deve equilibrar:

- Desempenho preditivo: acurácia, sensibilidade, especificidade.
- Interpretabilidade: essencial em decisões clínicas e epidemiológicas.
- Viabilidade computacional: tempo e custo de processamento.
- Disponibilidade de expertise: facilidade de ajuste e manutenção.



Referências

Bishop, C. M. (1995). **Neural Networks for Pattern Recognition**. Oxford University Press.

Breiman, L. (1996). **Bagging predictors. Machine Learning**, 24(2), 123–140. https://doi.org/10.1007/BF00058655

Breiman, L. (2001). **Random forests. Machine Learning**, 45, 5–32. https://doi.org/10.1023/A:1010933404324

Breiman, L., Friedman, J., Stone, C. J., & Olshen, R. A. (1984). **Classification and Regression Trees**. Wadsworth International Group.

Chen, T., & Guestrin, C. (2016). **XGBoost: A scalable tree boosting system**. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. https://doi.org/10.1145/2939672.2939785

Dietterich, T. G. (2000). **Ensemble methods in machine learning**. International Workshop on Multiple Classifier Systems, 1–15. https://doi.org/10.1007/3-540-45014-9_1

Domingos, P. (2012). **A few useful things to know about machine learning**. Communications of the ACM, 55(10), 78–87. https://doi.org/10.1145/2347736.2347755

Dorogush, A. V., Ershov, V., & Gulin, A. (2018). **CatBoost: gradient boosting with categorical features support**. arXiv preprint, arXiv:1810.11363.

Friedman, J. H. (2001). **Greedy function approximation: A gradient boosting machine**. Annals of Statistics, 29(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. Y. (2017). **LightGBM: A highly efficient gradient boosting decision tree**. Advances in Neural Information Processing Systems, 30.

Kohavi, R. (1995). **A study of cross-validation and bootstrap for accuracy estimation and model selection**. Proceedings of the 14th International Joint Conference on Artificial Intelligence, 2(12), 1137–1145.

Olson, R. S., La Cava, W., Mustahsan, Z., Varik, A., & Moore, J. H. (2018). **Data-driven advice for applying machine learning to bioinformatics**. Bioinformatics, 34(4), 623–632. https://doi.org/10.1093/bioinformatics/btx344

Quinlan, J. R. (1993). C4.5: **Programs for Machine Learning**. Morgan Kaufmann.

Vaid, A., Somani, S., Russak, A. J., De Freitas, J. K., Chaudhry, F., Paranjpe, I., ... & Narula, J. (2020). **Machine learning to predict mortality and critical events in COVID-19 positive New York City patients: A cohort study**. NPJ Digital Medicine, 3(1), 1–8. https://doi.org/10.1038/s41746-020-00383-x

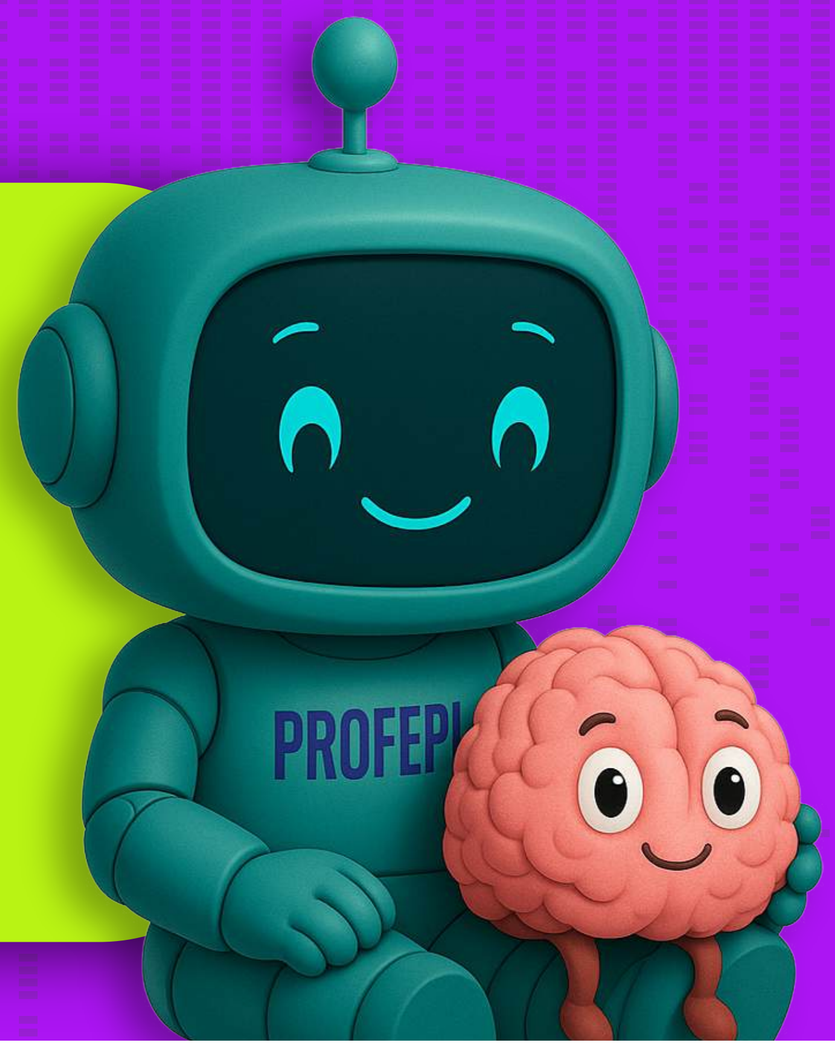
Wolpert, D. H., & Macready, W. G. (1997). **No free lunch theorems for optimization**. IEEE Transactions on Evolutionary Computation, 1(1), 67–82. https://doi.org/10.1109/4235.585893

World Health Organization (WHO). (2021). **Ethics and governance of artificial intelligence for health: WHO guidance**. WHO Press.

Zhu, G., Liu, J., Tan, Q., Shi, B., & Zhang, J. (2019). **Development and evaluation of a machine learning-based dengue risk prediction model using meteorological and entomological data**. PLoS Neglected Tropical Diseases, 13(10), e0007712. https://doi.org/10.1371/journal.pntd.0007712

capítulo 7

MÓDULO 3



Algoritmos Baseados em Redes Neurais

Introdução aos Algoritmos Baseados em Redes Neurais na Saúde

As redes neurais artificiais (RNA) são modelos computacionais inspirados no funcionamento do sistema nervoso biológico, compostos por unidades interconectadas denominadas *neurônios artificiais*, capazes de processar informações e aprender padrões complexos a partir de dados (McCulloch & Pitts, 1943; Rumelhart et al., 1986).

Embora o conceito tenha surgido na década de 1940, o uso prático das redes neurais passou por períodos de entusiasmo e estagnação (*AI winters*), ressurgindo com força no século XXI devido a três fatores centrais:

1. Aumento exponencial da capacidade computacional (uso de GPUs e TPUs).
2. Disponibilidade de grandes volumes de dados (big data).
3. Avanços em algoritmos de otimização e arquiteturas mais sofisticadas (LeCun, Bengio & Hinton, 2015).

No contexto da vigilância em saúde e da epidemiologia, as redes neurais ganharam relevância por sua habilidade em:

- Modelar relações não lineares e interações complexas entre variáveis.
- Integrar dados heterogêneos (tabulares, textuais, de imagens, séries temporais).
- Aprender padrões sutis que podem indicar riscos, surtos ou diagnósticos precoces.

Estrutura básica de uma rede neural

Uma RNA típica é organizada em três tipos de camadas:

- Camada de entrada: recebe as variáveis preditoras (ex.: idade, sexo, histórico de doenças, dados ambientais).
- Camadas ocultas: processam as informações por meio de combinações lineares de pesos e funções de ativação não lineares, permitindo que a rede capture padrões complexos.
- Camada de saída: gera a previsão final, que pode ser uma probabilidade, um valor contínuo ou uma classe.

Cada conexão entre neurônios possui um peso, ajustado durante o treinamento para minimizar uma função de perda (*loss function*), como erro quadrático médio ou entropia cruzada, dependendo do tipo de problema.

Processo de aprendizado

O treinamento de uma rede neural ocorre em três etapas principais:

1. Propagação direta (*forward pass*): os dados percorrem a rede até gerar a saída prevista.
2. Cálculo do erro: a previsão é comparada ao valor real usando a função de perda.
3. Retropropagação (*backpropagation*): o erro é propagado de volta, ajustando os pesos por meio de algoritmos de otimização, como o stochastic gradient descent (SGD) ou variações como Adam e RMSprop.

Esse processo é iterado em múltiplas épocas até que o modelo alcance um desempenho satisfatório ou atenda a um critério de parada.

Por que usar redes neurais na saúde?

As RNA apresentam vantagens decisivas para problemas de saúde pública:

- Capacidade de lidar com alta dimensionalidade: úteis em cenários onde o número de variáveis é grande e as relações são complexas.
- Integração de múltiplas fontes de dados: por exemplo, combinar dados do SINAN (notificações de agravos) e SIM (mortalidade) com variáveis ambientais e socioeconômicas.
- Detecção de padrões não triviais: essencial para identificar sinais precoces de surtos ou grupos de risco.

Exemplo real

Um estudo de Esteva et al. (2017) mostrou que redes neurais convolucionais treinadas com mais de 129 mil imagens foram capazes de diagnosticar câncer de pele com desempenho semelhante a dermatologistas. Embora esse seja um exemplo clínico, a lógica de detecção automática de padrões também pode ser aplicada em epidemiologia, por exemplo, para detectar clusters anômalos de notificações.

Desafios e limitações

Apesar do potencial, a adoção de redes neurais na saúde enfrenta desafios importantes:

- Necessidade de grandes volumes de dados: modelos profundos exigem muitos exemplos para evitar *overfitting*.
- Baixa interpretabilidade: redes neurais são frequentemente consideradas caixas-pretas, dificultando a explicação de decisões.
- Viabilidade computacional: treinamento pode ser inviável sem hardware especializado.
- Risco de vieses: se os dados de treinamento refletirem desigualdades sociais, o modelo pode reproduzi-las.

FIQUE ATENTO!

Por isso, a aplicação de redes neurais em saúde deve ser acompanhada de:

- Validação rigorosa em múltiplos cenários.
- Ferramentas de explicabilidade como SHAP e LIME.
- Supervisão ética e regulatória, garantindo que as decisões auxiliem e não prejudiquem a equidade no cuidado.

Aplicações na vigilância e gestão em saúde

As redes neurais já foram aplicadas em diversas frentes da saúde pública e epidemiologia, como:

- Predição de surtos: integração de dados climáticos, mobilidade populacional e notificações de doenças para prever epidemias de dengue ou influenza.
- Estimativa de demanda hospitalar: previsão de ocupação de leitos e necessidade de ventiladores em períodos de pico de doenças respiratórias.
- Classificação de risco individual: estratificação de pacientes com base em risco de hospitalização ou óbito.
- Detecção automática de eventos adversos: análise de textos de prontuários e sistemas de farmacovigilância para identificar reações medicamentosas.

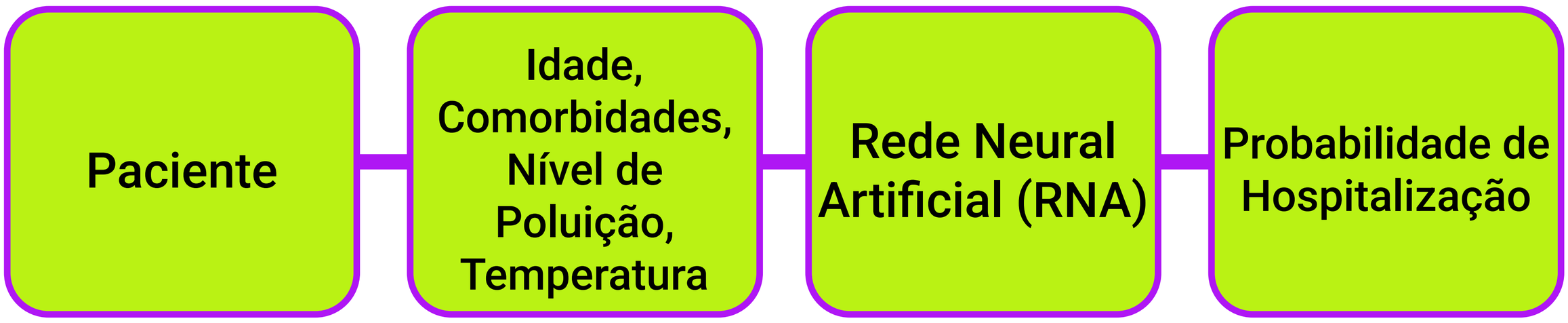
Introdução aos Algoritmos Baseados em Redes Neurais na Saúde

As Redes Neurais Artificiais (RNA), também conhecidas como *Multilayer Perceptrons* (MLP), são a forma mais clássica de rede neural. Apesar de sua simplicidade comparada a arquiteturas mais modernas, continuam sendo amplamente utilizadas em problemas de classificação, regressão e previsão em saúde pública devido à sua flexibilidade e capacidade de modelar relações não lineares.

Estrutura e componentes

Uma RNA típica é composta por:

- Camada de entrada: onde cada neurônio representa uma variável preditora (ex.: idade, índice de massa corporal, temperatura média semanal, níveis de poluição).
- Camadas ocultas: responsáveis por processar e transformar as informações. Cada neurônio de uma camada conecta-se a todos os neurônios da camada seguinte (*fully connected*).
- Camada de saída: onde é gerado o resultado final, podendo ser:
    - Uma classe (classificação binária ou multiclasse).
    - Um valor contínuo (regressão).
    - Uma probabilidade associada a cada classe.



Funcionamento interno

O processamento em uma RNA ocorre por meio de:

1. Combinação linear: cada neurônio recebe valores ponderados das entradas, somados a um viés (bias).

$$z = \sum_{i=1}^{n} w_i x_i + b$$

2. Função de ativação: transforma a combinação linear em uma saída não linear, permitindo que a rede capture relações complexas. As mais comuns incluem:
    - Sigmoide: útil para probabilidades, mas sofre com gradientes pequenos.
    - ReLU (*Rectified Linear Unit*): eficiente em redes mais profundas.
    - Tangente hiperbólica (tanh): centrada em zero, útil para dados normalizados.
3. Propagação direta (*forward pass*): os dados percorrem as camadas até gerar a saída.

4 Cálculo do erro: compara-se a previsão ao valor real usando funções de perda como *cross-entropy* (classificação) ou erro quadrático médio (regressão).

5 Retropropagação (*backpropagation*): calcula-se o gradiente do erro em relação aos pesos e ajusta-se cada conexão usando métodos como *stochastic gradient descent* ou Adam.

Hiperparâmetros críticos

O desempenho das RNA é fortemente influenciado por:

- Número de camadas ocultas e neurônios por camada: mais camadas permitem modelar relações mais complexas, mas aumentam risco de *overfitting*.
- Taxa de aprendizado (*learning rate*): controla a magnitude dos ajustes de pesos. Muito alta pode impedir a convergência; muito baixa torna o treino lento.
- Tamanho do lote (*batch size*): número de exemplos usados para calcular o gradiente a cada atualização.
- Função de ativação: impacta a capacidade de aprendizado e estabilidade.
- Regularização: técnicas como *dropout* (desativa aleatoriamente neurônios durante o treino) e penalizações L1/L2 evitam *overfitting*.

Vantagens das RNA

- Flexibilidade: aplicáveis a diferentes formatos de dados.
- Capacidade de capturar não linearidades: útil em cenários epidemiológicos complexos.
- Integração de múltiplas variáveis: combinam dados clínicos, demográficos e ambientais em um único modelo.

Limitações

- Necessidade de ajuste fino de hiperparâmetros: sem otimização, desempenho pode ser inferior a métodos mais simples.
- Menor interpretabilidade: embora mais explicáveis que redes profundas, ainda podem ser vistas como *caixa-preta*.
- Sensibilidade a dados desbalanceados: comum em saúde, onde eventos de interesse (ex.: óbito) podem ser raros.

Aplicações na saúde pública e vigilância epidemiológica

As RNA têm sido aplicadas com sucesso em diferentes contextos:

- Predição de internações hospitalares:
  Utilizando dados do SUS, RNA foram treinadas para prever internações por doenças cardiovasculares, integrando informações de prontuários, poluição atmosférica e dados meteorológicos.
  Impacto: melhor alocação de leitos e equipes médicas em períodos críticos.
- Estratificação de risco de doenças crônicas:
  Estudos com populações urbanas utilizaram RNA para identificar indivíduos com alto risco de desenvolver diabetes tipo 2, combinando fatores demográficos, hábitos de vida e indicadores laboratoriais.
- Previsão de surtos epidemiológicos:
  Redes neurais foram empregadas para prever surtos de dengue, Zika e chikungunya com base em variáveis climáticas (temperatura, chuva, umidade) e índices entomológicos.
- Apoio à triagem de pacientes em emergências:
  Modelos de RNA auxiliaram na predição de risco de óbito em prontos-socorros, otimizando protocolos de atendimento.

Exemplo prático: Prevendo surtos de dengue com RNA

Um projeto piloto no Nordeste do Brasil utilizou dados diários de:

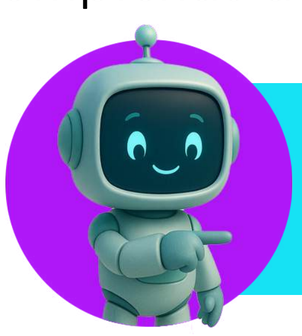
- Temperatura média e máxima.
- Precipitação acumulada.
- Índice de infestação do Aedes aegypti.
- Casos confirmados das últimas semanas.

O modelo RNA, com duas camadas ocultas (64 e 32 neurônios), função ReLU e *dropout* de 20%, alcançou AUC-ROC de 0,88 na previsão de surto para as semanas seguintes.

Benefício: permitiu que equipes de campo priorizassem regiões de maior risco, direcionando ações de controle vetorial.

Deep Learning

O *deep learning* é uma subárea das redes neurais artificiais caracterizada pelo uso de múltiplas camadas ocultas, permitindo ao modelo aprender representações hierárquicas e mais abstratas dos dados. Nas camadas iniciais, são captados padrões simples; nas camadas intermediárias, combinações desses padrões; e nas camadas finais, estruturas complexas de alto nível.



Esse avanço foi impulsionado por três fatores principais: maior capacidade computacional (GPU e TPU), disponibilidade de grandes volumes de dados e melhorias nos algoritmos de otimização.

Estrutura e funcionamento

- Camadas hierárquicas: a primeira camada recebe as variáveis de entrada, as camadas ocultas processam e transformam essas informações, e a camada de saída gera a previsão final.
- Extração automática de características: diferentemente de modelos tradicionais, o *deep* learning pode aprender automaticamente quais variáveis e combinações são mais relevantes, reduzindo a necessidade de engenharia manual de atributos (*feature engineering*).
- Funções de ativação não lineares: permitem que a rede capture padrões complexos; a ReLU é amplamente usada por evitar problemas de gradiente pequeno.
- Técnicas de regularização: *dropout*, *batch normalization* e *early stopping* ajudam a prevenir sobreajuste (*overfitting*).

Principais arquiteturas

- Redes neurais convolucionais (CNNs): ideais para análise de imagens médicas, como detecção de pneumonia em radiografias.
- Redes neurais recorrentes (RNNs) e LSTM: eficazes para séries temporais, como previsão de casos de doenças ao longo do tempo.
- Transformers adaptados para dados de saúde: aplicados ao processamento de linguagem natural (ex.: interpretação de prontuários eletrônicos).

Aplicações na saúde pública e vigilância epidemiológica

- Análise de imagens médicas: CNNs detectam padrões patológicos em exames de raio-X, tomografia e ressonância com precisão comparável à de especialistas humanos.
- Previsão de surtos: RNNs e LSTM combinam séries históricas de notificações, dados climáticos e mobilidade populacional para antecipar picos epidêmicos de doenças como dengue e influenza.
- Integração de múltiplas fontes de dados: redes profundas podem combinar dados administrativos (ex.: SINAN e SIM), laboratoriais e ambientais para modelar riscos de internação, óbito ou agravamento de doenças crônicas.
- Alertas precoces: modelos que cruzam dados de poluição atmosférica e clima com registros de pronto-socorro para emitir avisos de risco de internações por doenças respiratórias.

Desafios

- Necessidade de grandes volumes de dados: modelos profundos exigem muitos exemplos para evitar sobreajuste.
- Baixa interpretabilidade: a dificuldade em explicar decisões é crítica em saúde pública, exigindo ferramentas como SHAP ou LIME.

- Alta demanda computacional: treinamento e inferência podem requerer hardware especializado.
- Risco de vieses: dados de treinamento que refletem desigualdades sociais podem reproduzir ou amplificar essas disparidades.

FIQUE ATENTO!

Estratégias para aplicação responsável

- Uso de *transfer* learning para aproveitar modelos já treinados em domínios semelhantes.
- Implementação de técnicas de explicabilidade para aumentar a transparência das previsões.
- Validação rigorosa em diferentes cenários e populações para garantir robustez e generalização.

TabPFN (*Tabular Prior-Data Fitted Networks*)

O TabPFN (*Tabular Prior-Data Fitted Networks*) é um método extremamente recente e inovador que representa uma mudança de paradigma no uso de redes neurais para dados tabulares. Desenvolvido por Hollmann et al. (2022), ele combina conceitos de deep learning com técnicas de meta-learning e aprendizado probabilístico para criar um modelo que pode realizar previsões de alta qualidade em tabelas de dados de forma quase instantânea, mesmo com amostras muito pequenas.

Sua principal inovação é o uso de um modelo pré-treinado em milhões de tarefas sintéticas de classificação tabular, o que permite que, no momento da aplicação prática, ele já possua "experiência" prévia com uma enorme variedade de padrões possíveis. Isso contrasta com a lógica tradicional de redes neurais, que precisam ser treinadas do zero para cada novo problema, exigindo grandes volumes de dados e tempo computacional significativo.

Por que o TabPFN é considerado potente

- Velocidade de inferência: é capaz de fazer previsões em menos de um segundo para novos conjuntos de dados, mesmo sem re-treinamento.
- Desempenho competitivo: alcança resultados comparáveis ou superiores a modelos de ponta como XGBoost, LightGBM e CatBoost, especialmente em bases pequenas ou medianas.
- Capacidade de generalização: por ser treinado previamente em um vasto conjunto de problemas simulados, generaliza bem para diferentes contextos, mesmo quando os dados reais possuem padrões incomuns.
- Menor dependência de ajuste manual: dispensa, na maioria dos casos, longos processos de ajuste de hiperparâmetros.

Funcionamento

O TabPFN é baseado em arquiteturas de transformadores – a mesma base de modelos de linguagem como o GPT – adaptadas para lidar com dados tabulares. Durante seu pré-treinamento, ele aprende uma distribuição a priori sobre problemas de classificação e, posteriormente, quando recebe um novo conjunto de dados, utiliza essa distribuição para prever a probabilidade de cada classe para cada instância.

Esse mecanismo se aproxima de um raciocínio "metaestatístico": o modelo não aprende apenas a partir dos dados que lhe são fornecidos na aplicação, mas também aproveita todo o "conhecimento acumulado" do pré-treinamento.

Aplicações na saúde pública e vigilância epidemiológica

O TabPFN tem potencial disruptivo em áreas da saúde onde a escassez de dados é uma barreira para o uso de métodos tradicionais. Alguns exemplos de cenários promissores incluem:

- Doenças raras: previsão de risco ou classificação de casos quando há poucas amostras disponíveis, algo crítico para vigilância epidemiológica.
- Detecção precoce de surtos locais: classificação de municípios ou bairros como "em risco" ou "fora de risco" com base em poucas notificações iniciais e variáveis ambientais ou socioeconômicas.
- Predição de reincidência de violência ou agravos: análise de registros do SINAN e SIM para identificar padrões que indiquem maior probabilidade de repetição de eventos violentos, mesmo quando há poucos registros para treinamento.
- Triagem rápida em emergências: classificação imediata de pacientes com base em dados clínicos iniciais, útil para otimização de recursos em unidades de pronto atendimento.

Pontos fortes em relação a outros métodos

- Em bases pequenas, pode superar algoritmos clássicos como árvores de decisão, Random Forest ou GBDT, que geralmente precisam de muitos dados para alcançar alto desempenho.
- Rapidez de implementação, especialmente em contextos onde não há tempo para processos extensos de treinamento e validação.
- Maior adequação a problemas emergentes na saúde pública, onde novos agravos ou epidemias exigem respostas imediatas e não há um histórico robusto de dados.

Desafios e considerações

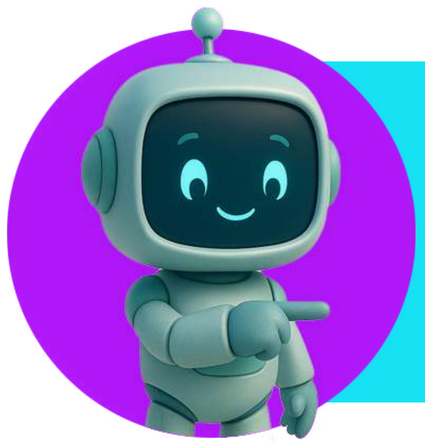
Apesar do seu potencial, o TabPFN ainda apresenta limitações para aplicação em larga escala na saúde:

- Tecnologia recente: a literatura científica e as validações no mundo real ainda são limitadas.
- Escopo restrito: atualmente, está mais voltado para tarefas de classificação supervisionada e pode exigir adaptações para outros tipos de problema (ex.: regressão, séries temporais).
- Infraestrutura e integração: embora rápido na inferência, exige recursos para integração com sistemas de saúde e pré-processamento adequado dos dados.

Perspectivas futuras

A tendência é que o TabPFN e métodos similares se tornem parte do arsenal de ferramentas para análise rápida e precisa em saúde, especialmente em situações onde:

- Há poucos dados disponíveis.
- O tempo para tomada de decisão é crítico.
- É necessário equilibrar desempenho com simplicidade de implementação.



A combinação de meta-aprendizado, deep learning e otimização para dados tabulares posiciona o TabPFN como uma tecnologia que pode redefinir a forma como utilizamos redes neurais na vigilância epidemiológica, tornando-as mais acessíveis e eficientes para problemas de baixa disponibilidade de dados.

Referências

Bishop, C. M. (1995). **Neural Networks for Pattern Recognition**. Oxford University Press.

Braga, A. P., Carvalho, A. C. P. L. F., & Ludermir, T. B. (2007). **Redes neurais artificiais: teoria e aplicações**. LTC.

Esteva, A., et al. (2017). **Dermatologist-level classification of skin cancer with deep neural networks**. Nature, 542, 115–118.

Hollmann, N., Müller, S., Müller, A. C., & Hutter, F. (2022). **TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second**. arXiv preprint, arXiv:2207.01848.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). **Deep learning**. Nature, 521, 436–444.

Hochreiter, S., & Schmidhuber, J. (1997). **Long short-term memory**. Neural Computation, 9(8), 1735–1780.

Shickel, B., Tighe, P. J., Bihorac, A., & Rashidi, P. (2018). **Deep EHR: A survey of recent advances in deep learning techniques for electronic health record (EHR) analysis**. IEEE Journal of Biomedical and Health Informatics, 22(5), 1589–1604.

Esteva, A., et al. (2017). **Dermatologist-level classification of skin cancer with deep neural networks**. Nature, 542, 115–118.

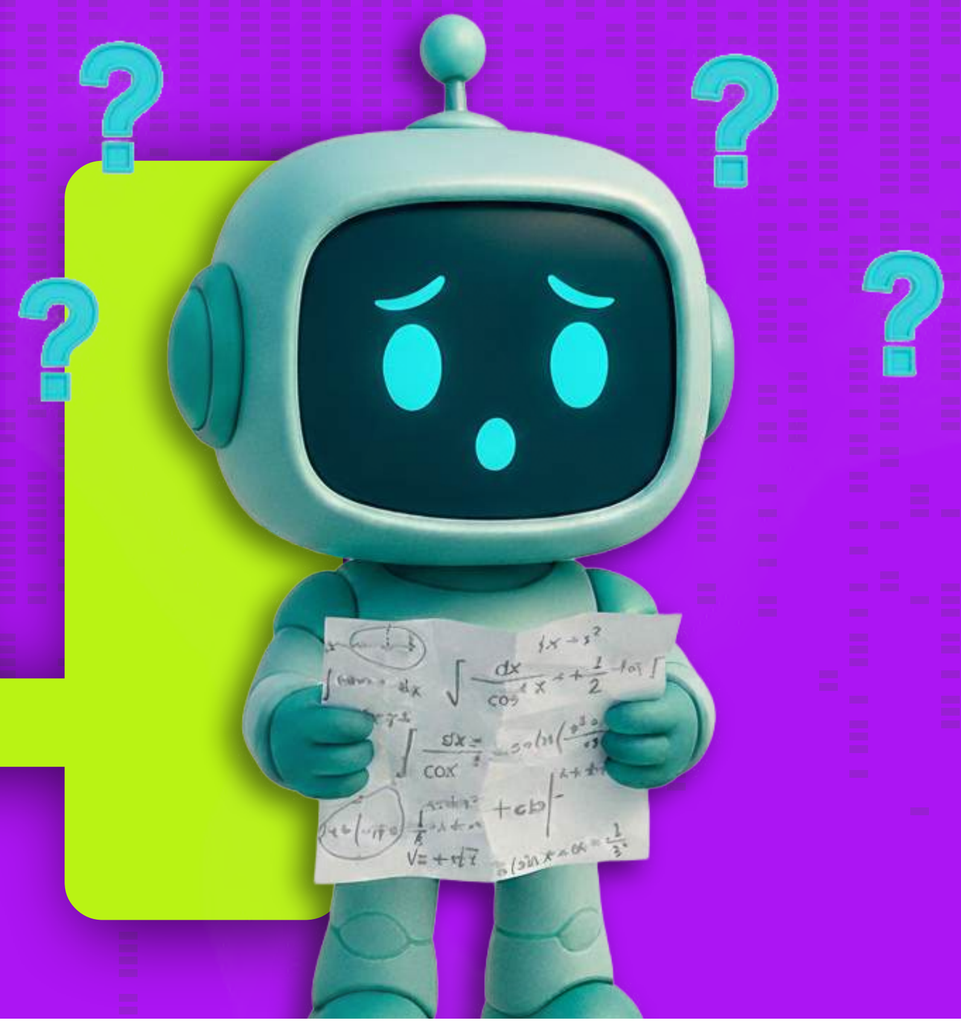
McCulloch, W. S., & Pitts, W. (1943). **A logical calculus of the ideas immanent in nervous activity**. The Bulletin of Mathematical Biophysics, 5, 115–133.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). **Learning representations by back-propagating errors**. Nature, 323, 533–536.

World Health Organization (WHO). (2021). **Ethics and governance of artificial intelligence for health**. WHO Press.

capítulo 8

MÓDULO 4



Estratégias para Seleção de Variáveis Preditoras

Introdução

O desenvolvimento de algoritmos que realizam boas predições em saúde é um dos objetivos centrais da ciência de dados contemporânea. No entanto, o caminho para alcançar uma alta performance preditiva possui imensos desafios técnicos. Entre as principais razões para o baixo desempenho de um algoritmo, estão a extrapolação inadequada dos resultados, o pré-processamento insuficiente dos dados, a validação inadequada da qualidade do modelo e, de forma ainda mais importante, o sobreajuste (*overfitting*).

Contudo, mesmo com a execução técnica correta, a performance de um modelo pode ser limitada pela ausência de variáveis preditoras verdadeiramente fortes. Este capítulo explora as estratégias principais para a seleção de variáveis preditoras, uma etapa decisiva para o sucesso de qualquer projeto de *machine learning* na área da saúde.

Pré-processamento dos Dados: a base da modelagem preditiva

O pré-processamento de dados é uma fase muito importante da análise preditiva que estabelece as bases para a construção de um modelo robusto. Muito além de uma tarefa mecânica, trata-se de um processo que exige tanto rigor técnico quanto discernimento conceitual.

Envolve desde a seleção criteriosa de variáveis que realmente possuem poder preditivo até a detecção e correção de armadilhas, como o vazamento de informação, que pode comprometer seriamente a validade do modelo.

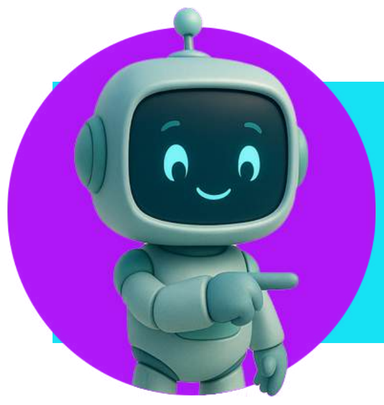
Cada uma dessas etapas, se negligenciada, pode introduzir vieses ou ruídos que inviabilizam a generalização do algoritmo para novos contextos.

> Em última análise, o pré-processamento não apenas prepara os dados para os algoritmos, mas também determina se as conclusões extraídas terão relevância clínica e aplicabilidade prática.

Preditores Plausíveis e a Natureza da Predição

O ponto de partida para a seleção de variáveis é o uso do bom senso do pesquisador para pré-selecionar preditores que sejam plausíveis. Em análises de *big data*, é comum que coincidências ocorram, levando o algoritmo a atribuir grande importância a associações espúrias.

É importante compreender que os preditores não precisam ter uma relação causal com o desfecho. Eles precisam apenas predizê-lo de forma consistente.



Um bom exemplo é a internação recente em uma UTI como preditora para o risco de óbito. A internação não é a causa da morte, mas sim um forte indicador da gravidade do estado do paciente.

Cuidado com o Vazamento de Informações (*Data Leakage*)

Um dos erros mais graves no pré-processamento é o vazamento de informações, ou -*data leakage*-. Isso acontece quando os dados de treinamento contêm informações ocultas sobre o desfecho, fazendo com que o modelo aprenda padrões que não são de seu interesse. Situações comuns incluem uma variável preditora que, na verdade, é consequência do desfecho ou a presença do mesmo indivíduo nos conjuntos de treino e teste.

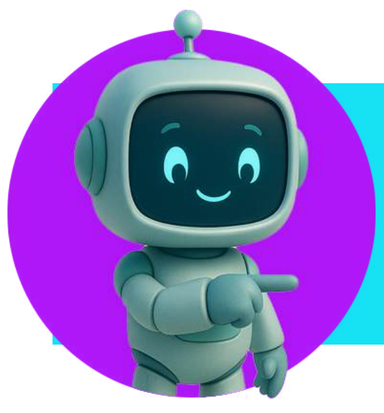
A Necessidade de Reduzir o Número de Variáveis

Após a seleção inicial de preditores plausíveis e a mitigação de vazamentos de informação, surge com frequência a necessidade de reduzir o número de variáveis utilizadas na modelagem.

Essa etapa, conhecida como redução de dimensionalidade, não é apenas uma conveniência técnica, mas um elemento estratégico para aumentar a eficiência e a robustez dos modelos.

Em contextos de saúde, a simplificação do conjunto de variáveis tem impacto direto na viabilidade do algoritmo preditivo, pois facilita a coleta de dados em regiões com menos infraestrutura e diminui os custos associados à obtenção e manutenção de informações clínicas ou laboratoriais. Além disso, ao reduzir o número de preditores, o tempo de treinamento dos algoritmos é significativamente encurtado, tornando possível explorar diferentes abordagens de modelagem em menos tempo.

A filtragem de variáveis também atua como uma segunda triagem, ajudando a eliminar preditores redundantes ou ruidosos, que poderiam confundir o algoritmo e reduzir sua capacidade de generalização.



Mais importante, concentra a atenção nos preditores que de fato carregam informação útil para antecipar o desfecho, aumentando tanto a interpretabilidade quanto a aplicabilidade dos modelos em cenários reais.

Métodos de Seleção de Variáveis

Existem diversas abordagens para selecionar o subconjunto ideal de variáveis preditoras. Elas variam desde técnicas estatísticas clássicas até métodos acoplados a algoritmos de *machine learning*.

Regressões Penalizadas: As regressões penalizadas são uma classe de algoritmos que introduzem uma penalidade aos coeficientes do modelo para evitar o sobreajuste. A penalização Ridge encolhe os parâmetros, mas não os zera, não realizando, portanto, a seleção de variáveis. Por outro lado, a penalização Lasso (L1) é capaz de zerar os coeficientes de variáveis menos importantes, realizando simultaneamente a regularização e a seleção de preditores ao penalizar os valores absolutos dos parâmetros.

Métodos Baseados em Árvores de Decisão: Algoritmos de *machine learning* baseados em árvores, como as Árvores de Decisão e o *Random Forest*, são particularmente úteis para a seleção de variáveis. Uma árvore de decisão funciona dividindo os dados em ramos baseados em testes em diferentes variáveis, com o objetivo de criar nós finais (folhas) que sejam o mais homogêneos possível em relação ao desfecho.

A importância de uma variável (*feature importance*) nesses modelos é calculada com base na redução de impureza que ela proporciona. A importância de uma variável é a soma total da redução de impureza que ela gerou em todas as divisões da árvore, sendo maior se a redução for alta e se o nó contiver muitas amostras.

Abordagens Avançadas para Seleção de Variáveis

Eliminação Recursiva de Variáveis (RFE)

Esse método busca encontrar um conjunto mínimo de variáveis, removendo recursivamente os preditores menos importantes e recalculando o ranking a cada passo, até que o conjunto com o menor erro de predição seja encontrado. A performance é sempre avaliada em observações não utilizadas no treinamento (*out-of-bag*).

Permutação

Nesse caso, os valores do desfecho são permutados aleatoriamente, mantendo a estrutura de correlação entre os preditores intacta. Uma variável é considerada importante se sua relevância no modelo com os dados reais for consistentemente maior do que nos modelos com o desfecho permutado.

Importância Variável Relativa Recorrente

Vários modelos de *Random Forest* são gerados com diferentes sementes aleatórias. A importância de cada variável é calculada e normalizada pela importância mínima em cada execução. São consideradas importantes as variáveis que atingem um limiar de importância pré-especificado.

Boruta

O Boruta é um dos métodos mais robustos para seleção de variáveis, amplamente reconhecido em estudos de simulação como a técnica de melhor desempenho para identificar preditores realmente relevantes.

Sua lógica parte de uma ideia engenhosa: criar “variáveis sombra”, que são cópias artificiais das variáveis originais, com valores aleatoriamente permutados. Essas variáveis não carregam informação preditiva real e servem como parâmetro de comparação.

Uma variável do conjunto original só é considerada importante se sua relevância, medida pela importância atribuída pelo modelo (geralmente *Random Forest*), for consistentemente superior à da melhor variável sombra.

Essa abordagem protege contra a inclusão de preditores espúrios, que poderiam gerar interpretações enganosas ou sobreajuste. Além disso, o Boruta apresenta a vantagem de ser eficaz não apenas em cenários de alta dimensionalidade, mas também em contextos com poucos preditores, garantindo flexibilidade e confiabilidade.

Por sua capacidade de equilibrar rigor estatístico com aplicabilidade prática, o Boruta se consolidou como uma ferramenta importante para aumentar a solidez e o uso prático dos modelos preditivos em saúde.

Conclusão

Estudos de simulação demonstraram que o algoritmo Boruta se destaca como o mais eficaz e intuitivo para a seleção estatística de variáveis após a definição do algoritmo com todas as preditoras.

Seus resultados são consistentes tanto para problemas de regressão quanto de classificação, e sua aplicabilidade se estende a cenários de baixa dimensionalidade, tornando-o uma ferramenta versátil e poderosa.

A seleção criteriosa de variáveis preditoras, utilizando métodos robustos como o Boruta, é, portanto, um passo importante para o desenvolvimento de modelos de *machine learning* que sejam não apenas tecnicamente corretos, mas também clinicamente úteis e viáveis de serem colocados em prática.

capítulo 9

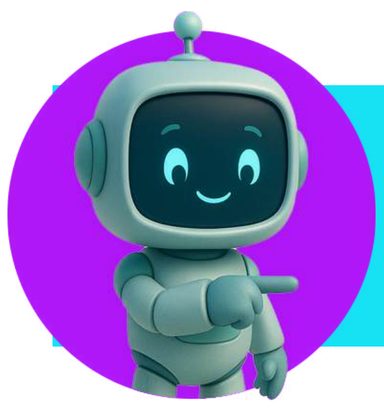
MÓDULO 4



Estratégias de Identificação de Variáveis Preditoras: SHAP values, modelos substitutos e causalidade

Introdução

Nos últimos anos, a aplicação de técnicas de inteligência artificial/*machine learning* na saúde pública vem crescendo de forma significativa. Essas ferramentas permitem utilizar grandes volumes de dados provenientes de sistemas como o SIM (Sistema de Informações sobre Mortalidade), o SINAN (Sistema de Informação de Agravos de Notificação) e o SIH/SUS (Sistema de Internações Hospitalares) para antecipar riscos, otimizar recursos e apoiar decisões estratégicas no Sistema Único de Saúde (SUS).



No entanto, a utilização desses modelos preditivos traz um desafio central: como interpretar os algoritmos? Em muitas situações, não basta apenas saber que o modelo “funciona” ou que apresenta bons indicadores de acurácia

Para que os resultados possam orientar políticas públicas e ganhar adesão dos profissionais de saúde, é essencial compreender quais variáveis estão influenciando as previsões e de que forma isso ocorre (Doshi-Velez & Kim, 2017; Ribeiro, Singh & Guestrin, 2016).

É importante destacar que o objetivo da predição não é explicar a causalidade de um fenômeno, mas antecipar sua ocorrência. Se o interesse do pesquisador for estabelecer relações de causa e efeito, devem-se utilizar métodos de inferência causal, como ensaios clínicos randomizados ou técnicas de modelagem causal. O papel dos algoritmos de ML, nesse contexto, é predizer com acurácia, e não necessariamente revelar mecanismos causais (Pearl, 2009).

A Busca por Algoritmos Interpretáveis

O tema da interpretabilidade dos algoritmos surge, portanto, como uma questão central. A busca por modelos “explicáveis” não significa reduzir a complexidade das técnicas, mas garantir que os resultados sejam compreensíveis e úteis para os profissionais que tomarão decisões baseadas neles.

Podemos destacar três grandes razões para investir em interpretabilidade:

1. **Identificação de preditores indesejáveis (*data leakage*):** um algoritmo pode estar utilizando informações que não deveriam estar disponíveis no momento da predição. Por exemplo, se um modelo de risco de mortalidade hospitalar usa "tempo de internação" como variável preditiva, temos um problema, pois essa informação só está disponível após o desfecho. A interpretação permite identificar e corrigir esse tipo de erro.

2. **Aceitação pelos profissionais de saúde:** médicos, gestores e técnicos tendem a confiar mais em modelos quando compreendem como as decisões são tomadas. A explicabilidade contribui para que os algoritmos sejam vistos como ferramentas de apoio, e não como "caixas-pretas" inacessíveis (Lipton, 2018).

3. **Garantia de maior robustez e ética:** a interpretabilidade também auxilia na detecção de vieses e injustiças. Estudos já demonstraram que algoritmos podem, inadvertidamente, reproduzir desigualdades sociais, como no caso de um modelo de alocação de recursos em saúde nos Estados Unidos que discriminava pacientes negros (Obermeyer *et al.*, 2019). Detectar e corrigir tais distorções é fundamental para garantir equidade no SUS.

   Apesar de sua relevância, a interpretabilidade não está isenta de desafios. Em alguns casos, a divulgação excessiva dos critérios pode até gerar manipulação. Por exemplo, se fosse público que morar em determinado bairro fosse critério para priorização de cirurgias, indivíduos poderiam falsificar informações para obter vantagem. Isso reforça a necessidade de equilibrar transparência e segurança

Estratégias para interpretação

Existem duas grandes abordagens para tornar modelos de *machine learning* mais interpretáveis: a interpretação intrínseca e a interpretação extrínseca (também chamada de explicabilidade pós-hoc).

1. Interpretação intrínseca

Alguns algoritmos já são naturalmente interpretáveis, pois sua estrutura permite compreender de forma clara a relação entre variáveis e desfecho.

- Regressão linear e logística: permitem observar diretamente os coeficientes, que indicam a direção (positiva ou negativa) e a intensidade da associação entre preditores e resultados.
- Árvores de decisão simples: sua lógica "se... então..." (*if-then rules*) facilita o entendimento dos caminhos usados pelo modelo para classificar ou prever.

Essas técnicas, embora transparentes, muitas vezes não alcançam a mesma performance preditiva que modelos mais complexos, como *random forests* ou redes neurais profundas.

2. Interpretação extrínseca (pós-hoc)

Quando utilizamos algoritmos mais sofisticados – por exemplo, *gradient boosting*, *XGBoost*, ou redes neurais artificiais –, a estrutura do modelo se torna uma "caixa-preta". Nesse caso, aplicam-se ferramentas externas que permitem extrair interpretações após o treinamento.

- LIME (*Local Interpretable Model-agnostic Explanations*): gera um banco de dados com pequenas variações nos valores das variáveis e constrói um modelo simples (como regressão linear) para explicar o comportamento local da predição (Ribeiro, Singh e Guestrin, 2016).
- SHAP (*SHapley Additive exPlanations*): baseado na teoria dos jogos, calcula a contribuição marginal de cada variável para uma predição específica ou para o conjunto de dados como um todo (Lundberg e Lee, 2017).

A escolha entre interpretação intrínseca e extrínseca depende do contexto. Em cenários de saúde pública, onde decisões podem afetar diretamente políticas e pessoas, muitas vezes é preferível combinar modelos robustos com técnicas de explicabilidade. Assim, conseguimos o melhor dos dois mundos: alto desempenho preditivo e clareza para orientar gestores e profissionais.

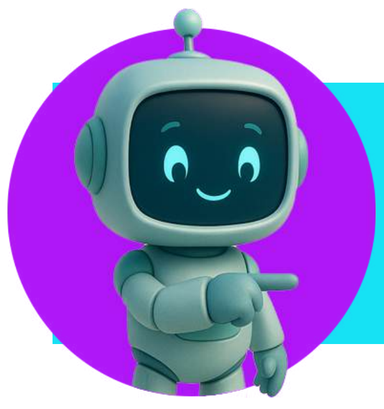
SHAP: fundamentos e propriedades

O método SHAP tem se destacado como uma das ferramentas mais sólidas e amplamente utilizadas para explicabilidade de modelos. Ele parte da teoria dos valores de Shapley, desenvolvida no campo da teoria dos jogos cooperativos, cujo objetivo é calcular como cada "jogador" contribui para o resultado final de um jogo (Shapley, 1953).

Aplicado ao *machine learning*, cada variável é vista como um jogador, e sua contribuição é medida em relação à predição do modelo. Dessa forma, o SHAP consegue responder perguntas como: "quanto a variável idade aumentou (ou diminuiu) a probabilidade de mortalidade hospitalar neste paciente específico?"

As principais propriedades do SHAP são:

- **Precisão local:** mostra exatamente por que um caso específico recebeu determinada predição.
- **Robustez a variáveis ausentes:** se uma variável não é relevante, sua contribuição é zero.
- **Consistência:** se uma variável se torna mais importante em um novo modelo, o valor SHAP aumenta proporcionalmente.



Essas características tornam o SHAP particularmente adequado para aplicações em saúde, pois permitem análises tanto individuais (locais) quanto globais (em toda a base de dados).

Aplicações do SHAP em Saúde Pública

O grande diferencial do SHAP em relação a outras métricas é sua capacidade de explicar de forma clara como cada variável contribui para uma predição. Isso é especialmente relevante na saúde pública, em que gestores e profissionais precisam compreender os fatores que influenciam riscos para priorizar intervenções, planejar recursos e garantir a equidade das ações.

1

Importância dos preditores: o SHAP ajuda a identificar quais variáveis têm maior impacto em um modelo preditivo.

- Exemplo prático: em um modelo para prever hipertensão em crianças, o SHAP pode revelar que o índice de massa corporal (IMC) é muito mais relevante do que o número de consultas médicas no último mês.
- Aplicação na gestão: esse tipo de resultado orienta secretarias de saúde a priorizarem políticas voltadas à prevenção da obesidade infantil, em vez de concentrarem esforços apenas no aumento de consultas.

2

Explicação individual (interpretação local): o SHAP também permite compreender por que um caso específico foi classificado de determinada forma.

- Exemplo prático: uma vítima registrada no SINAN como "alto risco de repetição de violência" pode ter sido classificada assim porque o histórico de notificações anteriores foi o fator mais determinante.
- Aplicação na gestão: esse nível de explicação permite direcionar intervenções mais assertivas, como acompanhamento prioritário pela rede de proteção social.

3

Resumo global do modelo: além das explicações individuais, os gráficos do SHAP oferecem uma visão geral dos fatores mais importantes em toda a base de dados.

- Exemplo prático: em análises de mortalidade do SIM, os gráficos SHAP podem mostrar que idade e escolaridade são os maiores preditores de mortalidade por doenças crônicas não transmissíveis (DCNT).
- Aplicação na gestão: gestores municipais podem usar essa informação para definir ações específicas em populações de maior vulnerabilidade, como idosos com baixa escolaridade.

4

Detecção de vieses: o SHAP ajuda a verificar se variáveis sensíveis, como raça/cor ou local de residência, estão influenciando as predições de forma injusta.

- Exemplo prático: um modelo de risco de hospitalização pode estar atribuindo maior probabilidade a pessoas negras ou moradores de regiões periféricas, sem que esses fatores tenham relação direta com a condição clínica.
- Aplicação na gestão: ao detectar esse viés, é possível ajustar o modelo para evitar discriminações e garantir maior justiça no acesso aos serviços do SUS.

5

Auditoria, ética e regulação: em contextos de vigilância e políticas públicas, muitas vezes é necessário justificar as decisões tomadas por algoritmos. O SHAP oferece uma explicação clara e auditável.

- Exemplo prático: ao utilizar um algoritmo para priorizar pacientes em cirurgias eletivas, a secretaria de saúde pode demonstrar, por meio dos valores SHAP, que a decisão se baseou em variáveis clinicamente relevantes, como idade e comorbidades, e não em fatores socioeconômicos injustos.

Vantagens para a prática em saúde

- Interpretação local e global: permite entender tanto os casos individuais quanto o conjunto populacional.
- Compatibilidade com qualquer algoritmo: pode ser aplicado em regressões, árvores de decisão, redes neurais e modelos mais complexos, como XGBoost.
- Visualizações intuitivas: gráficos como *beeswarm*, *waterfall* e *force plots* tornam os resultados acessíveis também a gestores não especialistas em ciência de dados.

Essas aplicações fazem do SHAP uma ferramenta estratégica para as secretarias de saúde, pois une rigor estatístico com clareza prática, fortalecendo a tomada de decisão baseada em evidências.

Modelos substitutos

Outra estratégia relevante para interpretação de modelos complexos são os chamados modelos substitutos. Eles consistem em construir um segundo modelo, de natureza mais simples e interpretável, que aproxima o comportamento do modelo original.

A lógica é a seguinte: o modelo de *machine learning* "caixa-preta" (como uma rede neural ou um *XGBoost*) é treinado para atingir a melhor performance preditiva possível. Em seguida, cria-se um modelo substituto – como uma regressão linear ou uma árvore de decisão simples – utilizando as mesmas variáveis de entrada, mas agora ajustado para reproduzir as decisões do modelo original.

Esse procedimento permite extrair regras e pesos interpretáveis que ajudam a compreender o raciocínio do algoritmo, mesmo sem abrir diretamente sua estrutura complexa.

LIME (*Local Interpretable Model-agnostic Explanations*)

Entre os métodos substitutos, o LIME é um dos mais conhecidos. Sua proposta é gerar pequenas perturbações nos dados originais – isto é, criar versões ligeiramente alteradas dos valores das variáveis – e observar como o modelo complexo reage.

A partir disso, o LIME constrói um modelo simples local (geralmente uma regressão linear) para explicar a predição daquele ponto específico. Em outras palavras, ele funciona como uma "lupa", mostrando, em nível individual, quais variáveis tiveram maior peso em determinada predição (Ribeiro, Singh e Guestrin, 2016).

Aplicações em saúde

- **Predição de óbito hospitalar:** o LIME pode ajudar a explicar por que um paciente foi classificado como de alto risco, mostrando que, naquele caso, o tempo de internação e a presença de comorbidades foram os fatores decisivos.
- **Modelos de violência:** em análises do SINAN, o LIME pode indicar que, para uma vítima específica, o histórico de múltiplas notificações foi determinante na classificação de alto risco de repetição de violência.
- **Doenças crônicas:** em modelos de predição de hipertensão infantil, o LIME pode evidenciar, em casos específicos, que fatores como IMC elevado e baixa prática de atividade física foram os principais responsáveis pela predição positiva.

Limitações dos modelos substitutos

Apesar de sua utilidade, é importante reconhecer algumas limitações:

- Incerteza estatística: muitos métodos de explicabilidade não quantificam o grau de incerteza associado às explicações.
- Dependência entre variáveis: quando existe alta correlação entre preditores (como peso e IMC), a atribuição de importância pode ser distorcida.
- Alto custo computacional: em grandes bases, como o SIM nacional, o uso repetido de modelos substitutos pode demandar tempo e infraestrutura robusta.

Desafios e Limitações da Interpretabilidade

Embora a interpretabilidade dos modelos de *machine learning* seja fundamental, ela enfrenta uma série de desafios metodológicos e conceituais. Entender essas limitações é essencial para que gestores e profissionais de saúde utilizem os resultados de forma crítica e responsável.

1

Inferência e incerteza estatística: muitos métodos de explicabilidade, como SHAP ou LIME, fornecem explicações sem quantificar a incerteza associada. Isso significa que não sabemos, de forma clara, o quanto aquela interpretação é estável ou poderia variar em outro conjunto de dados.

- Exemplo em saúde: ao interpretar um modelo que prevê óbito hospitalar, podemos concluir que a idade é a principal variável em um hospital específico, mas não ter segurança sobre se esse padrão se repete em outros contextos ou regiões.

2

Predição ≠ Causalidade: um dos pontos mais críticos é que a importância preditiva não equivale à causalidade (Pearl, 2009).

- Exemplo em saúde: o fato de um paciente ter sido internado em UTI pode ser um forte preditor de mortalidade futura, mas não significa que a UTI cause o óbito. Confundir esses conceitos pode levar a interpretações equivocadas e até a políticas inadequadas.

3 **Dependência entre variáveis:** quando variáveis estão altamente correlacionadas, torna-se difícil separar suas contribuições individuais.

- Exemplo em saúde: peso e IMC frequentemente caminham juntos. Em modelos de predição de hipertensão infantil, pode ser complicado determinar qual dos dois realmente tem maior impacto. Nesses casos, a importância atribuída pode se diluir entre variáveis correlacionadas.

4 **Manipulação e segurança:** a divulgação excessiva sobre como um algoritmo toma decisões pode gerar risco de manipulação do sistema.

- Exemplo em saúde: se fosse público que morar em determinado bairro aumenta a chance de priorização em filas cirúrgicas, indivíduos poderiam declarar falsamente residência naquela região para obter vantagem.

5 **Definição de interpretabilidade:** não há um consenso universal sobre como medir a interpretabilidade de um modelo. Ao contrário de métricas de performance (como acurácia ou AUC), que têm valores numéricos bem definidos, a interpretabilidade é subjetiva e pode depender do perfil do usuário final (Lipton, 2018).

- Exemplo em saúde: para um analista de dados, um gráfico de dependência pode ser considerado altamente interpretável; já para um gestor municipal, uma árvore de decisão simples pode ser mais clara.

6 **Alto custo computacional:** Métodos como SHAP exigem grande poder de processamento, especialmente em bases volumosas, como o SIM ou o SINAN nacionais. Isso pode limitar seu uso em secretarias de saúde com infraestrutura tecnológica mais restrita.

Considerações práticas

Esses desafios mostram que a interpretabilidade deve ser usada como ferramenta auxiliar, e não como resposta definitiva. O ideal é sempre combinar explicações geradas por IA com o conhecimento dos especialistas de saúde e com a análise epidemiológica tradicional. Assim, evitamos tanto o risco de confiar cegamente em modelos, quanto o de rejeitar tecnologias úteis por receio de sua complexidade.

Conclusões e Perspectivas Futuras

A interpretabilidade em modelos de inteligência artificial e *machine learning* não é apenas um detalhe técnico: ela é um componente central para que essas tecnologias possam ser incorporadas de forma ética, eficaz e sustentável na saúde pública.

Ao longo deste capítulo, vimos que:

- **Predição não é causalidade:** modelos preditivos identificam padrões úteis para antecipar riscos, mas não explicam mecanismos causais. Para análises causais, devem-se utilizar métodos específicos de inferência.
- **Interpretabilidade é estratégica:** seja por meio de modelos intrínsecos (como regressões e árvores simples) ou extrínsecos (como SHAP e LIME), compreender as variáveis que influenciam uma predição é essencial para a aceitação e o uso prático dos modelos.
- **SHAP se destaca:** por oferecer explicações locais (individuais) e globais (populacionais), com propriedades matemáticas consistentes, o SHAP é hoje uma das ferramentas mais sólidas para apoiar decisões em saúde.
- **Desafios persistem:** problemas como correlação entre variáveis, incerteza estatística, alto custo computacional e risco de manipulação precisam ser continuamente enfrentados para que os algoritmos sejam usados com segurança.

Implicações para a saúde pública

No contexto do SUS e das secretarias de saúde, a interpretabilidade pode apoiar:

- **Vigilância epidemiológica:** identificar fatores preditores de agravos como violência de repetição, suicídio ou complicações por doenças crônicas.
- **Gestão de recursos:** priorizar grupos populacionais de maior risco de forma transparente e auditável.
- **Políticas equitativas:** detectar e corrigir vieses que poderiam reforçar desigualdades sociais e raciais.

Perspectivas futuras

O campo da interpretabilidade em *machine learning* ainda está em evolução. Alguns caminhos promissores incluem:

- **Integração com ética e regulação:** organismos nacionais e internacionais começam a exigir que algoritmos de saúde sejam explicáveis e auditáveis, como forma de proteger os cidadãos.
- **Interdisciplinaridade:** a interpretabilidade precisa dialogar com outras áreas do conhecimento, como psicologia e sociologia, para desenvolver formas de comunicação mais adequadas a diferentes públicos.
- **Tecnologias mais acessíveis:** com o avanço das ferramentas abertas e das infraestruturas em nuvem, é esperado que métodos como SHAP e LIME se tornem mais viáveis mesmo para municípios com menor capacidade computacional.

Referências

Bishop, C. M. (1995). **Neural Networks for Pattern Recognition**. Oxford University Press.

Doshi-Velez, F., & Kim, B. (2017). **Towards A Rigorous Science of Interpretable Machine Learning**. arXiv preprint arXiv:1702.08608.

Lipton, Z. C. (2018). **The mythos of model interpretability**. Communications of the ACM, 61(10), 36–43.

Lundberg, S. M., & Lee, S.-I. (2017). **A unified approach to interpreting model predictions**. Advances in Neural Information Processing Systems (NeurIPS), 30.

Molnar, C. (2022). **Interpretable Machine Learning**. 2nd edition.

Obermeyer, Z., Powers, B., Vogeli, C., & Mullainathan, S. (2019). **Dissecting racial bias in an algorithm used to manage the health of populations**. Science, 366(6464), 447–453.

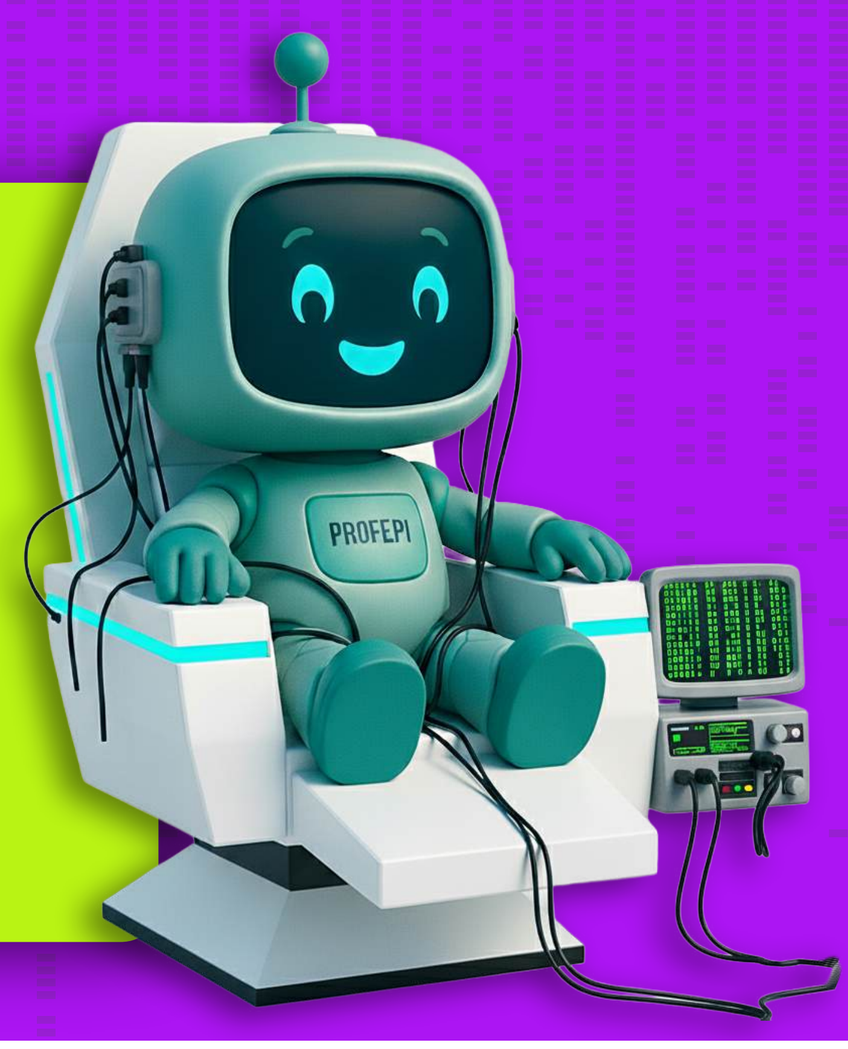
Pearl, J. (2009). Causality: **Models, Reasoning and Inference**. Cambridge University Press.

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": **Explaining the predictions of any classifier**. Proceedings of the 22nd ACM SIGKDD, 1135–1144.



capítulo 10

MÓDULO 4



Perspectivas do uso de Machine Learning na Saúde: aprendizado de transferência, federado e contínuo

Além do Machine Learning Tradicional

O paradigma clássico da área de *machine learning* (ML) opera sob a premissa de que os dados de treinamento e teste são extraídos da mesma distribuição e que cada tarefa é resolvida de forma isolada.

No entanto, o cenário da saúde é frequentemente caracterizado pela escassez de dados em contextos específicos e pela necessidade de proteger a privacidade do paciente. Para superar esses desafios, abordagens avançadas, como o aprendizado de transferência, o aprendizado federado e o aprendizado contínuo, estão ganhando destaque, prometendo revolucionar a forma como os modelos preditivos são desenvolvidos e implementados.

Aprendizado de Transferência (*Transfer Learning*)

Transfer Learning (TL) é uma solução poderosa para o problema da insuficiência de dados. A essência do TL é aproveitar o conhecimento adquirido em um domínio de origem, onde há abundância de dados, e transferi-lo para um domínio de destino, no qual os dados são escassos.

O que se transfere são os pesos de um modelo pré-treinado, e, historicamente, as redes neurais profundas (*Deep Learning*) têm sido o veículo mais comum para essa transferência, especialmente em tarefas envolvendo imagens e processamento de linguagem natural (NLP).

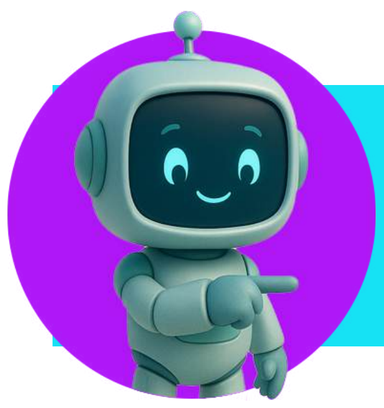
A decisão de quando aplicar o TL depende da disponibilidade de dados, da similaridade entre os domínios e da constatação de que a transferência efetivamente melhora a performance do modelo no novo contexto.

Estudo de Caso: o Projeto IACOV-BR e a Predição na COVID-19

Um exemplo prático da aplicação desses conceitos no Brasil é o estudo multicêntrico IACOV-BR, que visou a predição de desfechos relacionados à COVID-19, como óbito, necessidade de UTI e ventilação mecânica. O estudo envolveu 18 hospitais de diversas regiões do país, refletindo uma grande heterogeneidade demográfica, de recursos e de práticas clínicas, com amostras que variavam de 47 a 1.776 pacientes.

O objetivo central era investigar se agregar dados de múltiplos hospitais para o treinamento de algoritmos poderia melhorar a capacidade preditiva em cada hospital individualmente. Foram testadas oito estratégias de treinamento distintas, desde o treino puramente local (usando apenas 70% dos dados do próprio hospital) até diferentes formas de agregar dados de outros hospitais, seja por região ou de todo o país.

Os resultados, no entanto, foram claros. Para a maioria dos hospitais e desfechos, o treinamento local demonstrou ser a melhor estratégia. Isso sugere que a heterogeneidade entre as regiões brasileiras é tão grande que a agregação de dados, ao invés de ajudar, introduz ruído e prejudica a performance.



Essa constatação reforça que não existe uma solução única e que é necessário testar múltiplas estratégias para encontrar a abordagem ideal em cada contexto específico do mundo real.

Aprendizado de Transferência em Ação: Ajuste Fino de Modelos

Explorando uma abordagem mais sofisticada, o estudo IACOV-BR utilizou aprendizado de transferência em modelos baseados em árvores, como o XGBoost. A técnica consiste em utilizar um modelo pré-treinado no hospital de origem (aquele com o melhor desempenho local) como base e, em seguida, realizar um "ajuste fino", adicionando novas árvores treinadas especificamente com os dados do hospital de destino para capturar seus padrões particulares.

As análises buscaram avaliar se essa transferência superaria a simples validação externa do melhor modelo. Os resultados mostraram que o aprendizado de transferência conseguiu melhorar o desempenho em vários hospitais, mas o grau de melhoria variou consideravelmente.

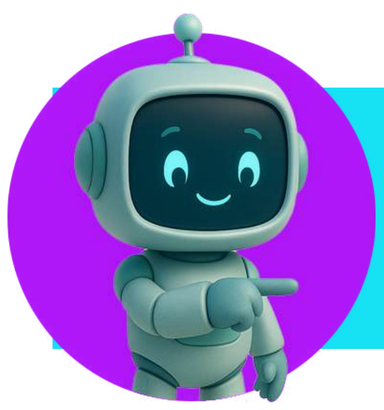
Uma análise de sensibilidade revelou que a maior parte da melhoria na performance ocorria com a adição das primeiras árvores, com o desempenho se estabilizando em seguida, indicando que um ajuste fino direcionado pode ser mais eficiente.

Esses resultados apontam para o potencial do aprendizado de transferência como uma estratégia para disseminar a inteligência artificial na saúde. Em vez de exigir que cada hospital desenvolva um modelo complexo do zero, o que seria inviável para centros com poucos dados, essa abordagem permite a adaptação de um modelo robusto e já validado às realidades locais.

Isso não apenas acelera o desenvolvimento e a implementação de ferramentas preditivas, mas também abre caminho para um ecossistema de saúde mais colaborativo, no qual o conhecimento gerado em grandes centros pode ser transferido e refinado para beneficiar uma rede mais ampla de hospitais e pacientes.

Aprendizado Federado (Federated Learning)

Paralelamente aos desafios de volume e diversidade de dados, a privacidade é uma preocupação primordial. O Aprendizado Federado (*Federated Learning*) surge como resposta a essa necessidade.



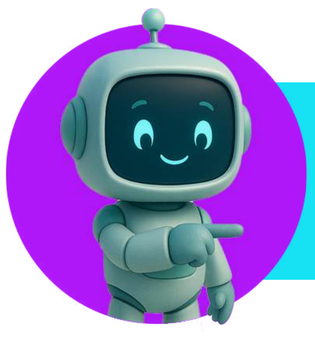
Sua principal proposta é treinar um modelo global de *machine learning* de forma colaborativa sem que os dados brutos precisem sair de seus locais de origem, como hospitais ou dispositivos móveis.

Nesse sistema, cada dispositivo ou instituição treina um modelo localmente. Em seguida, apenas as atualizações desse modelo são enviadas a um servidor central, que as agrega para criar um modelo global aprimorado.

Essa abordagem respeita as rigorosas práticas de privacidade e as regulamentações como a LGPD, permitindo a colaboração entre instituições que, de outra forma, não poderiam compartilhar dados de pacientes. O aprendizado federado, no entanto, enfrenta seus próprios desafios, como a heterogeneidade dos dados entre os diferentes locais e a necessidade de otimizar a comunicação para ser eficiente.

Aprendizado Contínuo (Online Learning)

Por fim, o campo do aprendizado de máquina está evoluindo do modelo tradicional *offline*, em que o treinamento ocorre em lotes (*batch*), para o Aprendizado Contínuo ou *Online Learning*. Nessa modalidade, os dados chegam de forma sequencial, em um fluxo contínuo, e o modelo é atualizado a cada nova observação.



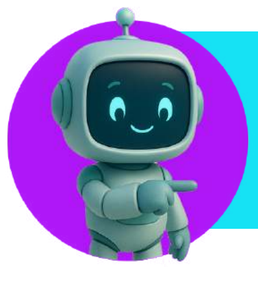
Isso permite que o algoritmo se adapte em tempo real a novas tendências e padrões, com um custo computacional menor para cada atualização.

Essa capacidade de adaptação instantânea é importante para aplicações de grande escala e dinâmicas, como as encontradas no setor de saúde, em que novas informações sobre pacientes e tratamentos surgem constantemente.

Considerações finais

A integração desses três aprendizados – transferência, federado e contínuo – representa a próxima fronteira para a inteligência artificial na saúde. Juntas, elas abordam os principais gargalos do setor: a escassez de dados, as barreiras de privacidade e a natureza estática dos modelos tradicionais.

Pode-se imaginar um futuro em que um modelo de base, desenvolvido em um grande centro de pesquisa, é compartilhado via aprendizado de transferência. Em seguida, ele é aprimorado de forma colaborativa e segura por uma rede de hospitais por meio do aprendizado federado, sem que nenhum dado sensível seja exposto. Finalmente, esse modelo global é continuamente atualizado em cada local com novos dados de pacientes em tempo real, por meio do aprendizado contínuo.



Essa colaboração de diferentes técnicas emergentes em IA tem o potencial de criar um ecossistema de saúde verdadeiramente inteligente, dinâmico e ético, acelerando a inovação e melhorando os desfechos clínicos em uma escala sem precedentes.

capítulo 11

MÓDULO 4



Desafios Éticos em Inteligência Artificial na Saúde

Introdução

A incorporação da inteligência artificial (IA) na saúde já não é uma promessa distante: ela está presente em sistemas de vigilância epidemiológica, diagnósticos por imagem, gestão hospitalar e até na alocação de recursos em saúde. Entretanto, o uso dessas ferramentas traz desafios éticos complexos, que precisam ser considerados desde a concepção até a supervisão contínua dos modelos (Char, Abràmoff e Feudtner, 2020).

Entre esses desafios, destacam-se a representatividade dos dados e o risco de preconceito algorítmico, que podem impactar diretamente a equidade em saúde.

1. Vieses e Desigualdades nos Dados

Os dados de saúde não são neutros. Eles refletem desigualdades históricas e estruturais. Quando modelos de ML são treinados com essas bases, podem ampliar disparidades, em vez de reduzi-las.

- Exemplo internacional: nos EUA, um algoritmo amplamente utilizado para priorizar pacientes com maior risco de complicações subestimava pacientes negros, porque usava custos hospitalares como proxy de gravidade – e historicamente esses custos eram menores em populações negras devido a menor acesso a serviços (Obermeyer *et al.*, 2019).

- Exemplo em vigilância: bases como SINAN ou SIH podem subnotificar populações indígenas, tornando os algoritmos menos sensíveis para esse grupo.

2. Categorias de Vieses em IA Aplicada à Saúde

O desenvolvimento e a aplicação de algoritmos de *machine learning* em saúde estão sujeitos a diferentes formas de viés. Esses vieses podem se originar tanto da forma como os dados são coletados e representados quanto da maneira como os algoritmos são interpretados e utilizados pelos profissionais.

1

Viés de minoria: Ocorre quando há baixa representatividade de determinados grupos populacionais nos dados de treinamento.

- Exemplo: se um algoritmo de predição de risco cardiovascular for treinado majoritariamente com dados de adultos brancos urbanos, ele poderá ter desempenho muito pior para populações indígenas ou quilombolas, que são sub-representadas nos registros.
- Consequência: modelos tornam-se menos confiáveis para grupos vulneráveis, aumentando o risco de decisões clínicas ou de gestão inadequadas.
- Implicação para o SUS: se políticas de prevenção forem orientadas por esses modelos, populações já invisibilizadas podem continuar sem receber atenção proporcional às suas necessidades.

2

Viés de automação: refere-se à tendência dos profissionais em aceitar de forma acrítica as decisões de um sistema automatizado, mesmo quando ele está errado (*automation bias*).

- Exemplo: se um algoritmo indicar que um paciente tem baixo risco de complicação, médicos podem relaxar a vigilância, mesmo diante de sinais clínicos que indicariam o contrário.
- Consequência: erros do algoritmo passam despercebidos e podem comprometer o cuidado.
- Implicação para a saúde pública: gestores podem aceitar previsões algorítmicas sobre surtos ou riscos sem considerar limitações metodológicas, levando a decisões equivocadas na alocação de recursos.

3

Viés de rejeição: é o oposto do anterior: ocorre quando profissionais desconfiam excessivamente do algoritmo, mesmo quando a previsão é correta (*rejection bias*).

- Exemplo: um modelo pode identificar corretamente que determinado bairro apresenta risco elevado de dengue. Porém, se os gestores desacreditarem do resultado por considerarem a ferramenta “inacessível” ou “caixa-preta”, podem deixar de agir preventivamente.
- Consequência: perde-se a oportunidade de agir de forma antecipada, desperdiçando o potencial da IA.
- Implicação para o SUS: dificulta a adoção de tecnologias inovadoras, perpetuando desigualdades no acesso a ferramentas modernas de vigilância.

4

Viés de privilégio: refere-se à priorização de condições de saúde que afetam principalmente grupos mais privilegiados, em detrimento das doenças que impactam populações vulneráveis.

- Exemplo: grandes investimentos em IA para câncer de pele em países de alta renda, enquanto doenças negligenciadas, como leishmaniose, doença de Chagas ou hanseníase, recebem pouca atenção tecnológica.
- Consequência: reforça a "lacuna 10/90" já descrita na literatura em saúde global, em que 90% dos recursos de pesquisa são destinados a apenas 10% dos problemas que afetam a população mundial.
- Implicação para a saúde pública: algoritmos desenvolvidos sem foco nas Doenças Crônicas Não Transmissíveis (DCNTs) mais prevalentes em países de baixa e média renda podem não responder às reais demandas do SUS.

Esses quatro tipos de vieses mostram que a IA em saúde não é neutra. Ela pode tanto corrigir desigualdades quanto agravá-las, dependendo de como for concebida, validada e aplicada.

3. Equidade e Justiça Distributiva em IA Aplicada à saúde

O conceito de justiça distributiva refere-se à forma como bens, serviços e oportunidades são distribuídos entre diferentes grupos da sociedade. No contexto da saúde, isso significa garantir que todos os indivíduos, independentemente de raça, gênero, classe social ou local de moradia, tenham acesso justo aos benefícios proporcionados pela inteligência artificial (IA).

A preocupação é que, sem medidas proativas, algoritmos de *machine learning* podem reproduzir ou ampliar desigualdades históricas, dado que:

1. Os dados de saúde são coletados de forma desigual;
2. As doenças que afetam populações privilegiadas tendem a receber mais atenção científica e tecnológica;
3. Os modelos podem apresentar performance inferior em populações vulneráveis, prejudicando justamente aqueles que mais necessitam de cuidados.

Três dimensões de equidade em IA em saúde:

1. Resultados iguais

O objetivo é que os benefícios clínicos e de saúde pública sejam igualmente distribuídos entre grupos protegidos (negros, indígenas, mulheres, populações de baixa renda) e grupos majoritários.

- **Exemplo:** um algoritmo para triagem de risco cardiovascular deve ser capaz de prever infartos com a mesma eficácia em homens e mulheres. Se ele acerta mais em homens, isso gera desigualdade de resultados.
- **Desafio:** garantir que o modelo não apenas seja preciso globalmente, mas que não agrave disparidades entre grupos.

2. Performance igual

A equidade também depende de que o modelo apresente sensibilidade, especificidade e valor preditivo positivo semelhantes em diferentes grupos.

- **Exemplo:** se um modelo de predição de câncer de mama for menos sensível em mulheres negras por falta de dados representativos, isso pode atrasar diagnósticos e aumentar mortalidade nesse grupo.
- **Métricas utilizadas:**
  - igualdade de oportunidade: mesma sensibilidade entre grupos;
  - probabilidades equalizadas: sensibilidade e especificidade iguais;
  - paridade preditiva: mesmo valor preditivo positivo.
- **Desafio:** muitas vezes, não é possível atingir todas as métricas de justiça simultaneamente. É necessário priorizar de acordo com o contexto epidemiológico e social.

3. Alocação justa de recursos

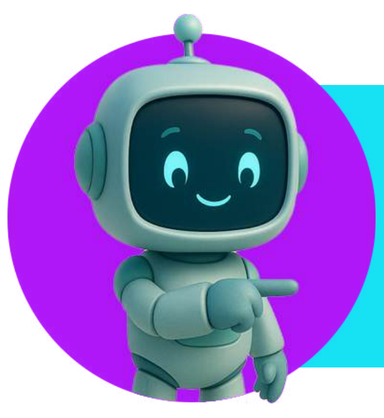
Decisões algorítmicas muitas vezes orientam a distribuição de recursos em saúde, como leitos de UTI, exames de imagem ou equipes de atenção básica.

- **Exemplo:** se um algoritmo de alocação de leitos priorizar pacientes de regiões mais ricas (onde há mais dados disponíveis e melhor registro de saúde), isso pode concentrar recursos em populações já privilegiadas.
- **Implicação no SUS:** é essencial que algoritmos sejam auditados para garantir que populações vulneráveis recebam recursos de forma proporcional às suas necessidades de saúde.

O dilema dos trade-offs

Um ponto crítico é que diferentes métricas de justiça podem ser incompatíveis. Por exemplo:

- um modelo pode atingir paridade no valor preditivo positivo (PPV), mas ainda assim não apresentar sensibilidades iguais entre grupos;
- ajustar o limiar de decisão para aumentar a sensibilidade em um grupo pode elevar a taxa de falsos positivos nesse mesmo grupo.



Isso significa que não existe um modelo “perfeitamente justo”. A decisão sobre qual métrica priorizar deve ser tomada de forma transparente, envolvendo profissionais de saúde, gestores, cientistas de dados e, idealmente, representantes da sociedade civil.

Por que a justiça distributiva importa no SUS?

No Brasil, a equidade é um dos princípios organizativos do SUS. Isso significa que as tecnologias em saúde, incluindo IA, precisam ser desenhadas para reduzir desigualdades, e não reforçá-las.

- Exemplo prático: se um modelo identifica áreas com maior risco de surtos de dengue, mas não inclui favelas por subnotificação, o resultado será a priorização de bairros formais, ampliando a vulnerabilidade das populações mais pobres.
- Ação necessária: garantir representatividade adequada nos bancos de dados, avaliar desempenho estratificado e adotar políticas públicas que incluam critérios de justiça social no uso da IA.

4. Pipeline ético: do Desenho à Supervisão

A ética em inteligência artificial aplicada à saúde não pode ser pensada apenas no final do processo. Ela deve ser considerada em todas as fases do ciclo de vida de um modelo de *machine learning* (Char, Abràmoff e Feudtner, 2020).

Esse percurso é chamado de *pipeline* ético, que envolve desde a concepção inicial até a supervisão contínua da tecnologia em uso.

Concepção do modelo

- Definição clara de objetivos: o modelo deve ser construído para responder a um problema real de saúde pública ou clínica, e não apenas por interesse tecnológico.
- Participação de partes interessadas: médicos, gestores, cientistas de dados e representantes de populações vulneráveis devem participar da definição da meta.
- Transparência e auditabilidade: o sistema precisa ser explicável o suficiente para permitir auditorias independentes.
- Exemplo aplicado: ao propor um modelo para prever risco de repetição de violência, é essencial discutir desde o início como os dados serão usados, quais variáveis serão incluídas e quais impactos sociais podem surgir.

Coleta e preparação de dados

- Representatividade: é crucial garantir que grupos protegidos (ex.: negros, indígenas, populações rurais) estejam devidamente representados.
- Documentação rigorosa: registrar como os dados foram coletados, quais variáveis estão disponíveis e quais lacunas existem.
- Privacidade: balancear a necessidade de identificar desigualdades com a proteção de informações sensíveis.
- Exemplo aplicado: em um modelo para predição de hipertensão infantil, certificar-se de que crianças de diferentes níveis socioeconômicos e regiões do país estejam incluídas nos dados.

Desenvolvimento do modelo

- Escolhas técnicas conscientes: algoritmos e funções de perda não são neutros; eles podem favorecer determinados grupos.
- Controle de vieses: aplicar técnicas de balanceamento de dados, inclusão de restrições éticas ou penalizações que minimizem desigualdades preditivas.
- Exemplo aplicado: ao treinar um modelo para alocação de leitos de UTI, é possível ajustar parâmetros para evitar que bairros mais ricos sejam favorecidos apenas por terem mais registros.

Calibração e validação

- Validação estratificada: avaliar métricas de sensibilidade, especificidade e valor preditivo por subgrupos (raça, gênero, território).
- Equidade como critério: não basta atingir alta acurácia global; é preciso verificar se o modelo funciona igualmente bem em diferentes grupos populacionais.
- Exemplo aplicado: validar se um algoritmo para triagem de risco de suicídio tem o mesmo desempenho em homens jovens e em mulheres idosas.

Implementação prática

- Integração ao fluxo de trabalho: o algoritmo deve ser incorporado aos processos clínicos ou de vigilância de forma que faça sentido para os profissionais.
- Treinamento dos usuários: médicos, enfermeiros e gestores precisam compreender como interpretar os resultados e suas limitações.
- Exemplo aplicado: em secretarias municipais, um sistema de predição de surtos só será útil se os técnicos entenderem as variáveis usadas e como agir diante dos alertas.

Supervisão contínua

- Monitoramento ativo: acompanhar periodicamente o desempenho do modelo em diferentes populações e contextos.
- Auditoria ética: avaliar se o algoritmo está produzindo resultados justos ou se está reforçando desigualdades.
- Feedback constante: incorporar opiniões de profissionais de saúde e dos próprios pacientes.
- Exemplo aplicado: um algoritmo de priorização de cirurgias deve ser monitorado para garantir que não esteja discriminando, por exemplo, moradores de regiões periféricas.

5. Estudos de caso

Predição de alta hospitalar: modelos podem prever permanência mais longa para pacientes de bairros pobres, o que leva gestores a alocar menos recursos nesses locais – reforçando desigualdades.

Monitoramento intensivo: sistemas de predição para deterioração clínica podem falhar em grupos minoritários, por falta de exemplos no treinamento.

Diagnóstico automatizado por imagem: algoritmos treinados com radiografias de um único país podem ter performance inferior em outros contextos, prejudicando populações sub-representadas.

Recomendações para Prática em Saúde Pública

- Envolver múltiplos atores (gestores, profissionais de saúde, representantes de populações vulneráveis) no desenho e implantação.
- Transparência e comunicação: explicar de forma clara como o modelo foi desenvolvido e quais são suas limitações.
- Auditoria independente: permitir que modelos utilizados em políticas públicas sejam revisados por instituições externas.
- Regulação e governança: acompanhar iniciativas internacionais de regulação de IA em saúde, como as diretrizes da Organização Mundial da Saúde (WHO, 2021).

Conclusão

A inteligência artificial aplicada à saúde tem enorme potencial para melhorar diagnósticos, otimizar recursos, apoiar a vigilância epidemiológica e promover um cuidado mais eficiente e personalizado. No entanto, esse potencial só se realiza de maneira justa quando os desafios éticos são reconhecidos e tratados de forma sistemática.

Ao longo deste capítulo, vimos que:

- os dados não são neutros: eles carregam marcas de desigualdades históricas e estruturais. Se utilizados de forma acrítica, os algoritmos de *machine learning* podem perpetuar preconceitos já existentes.
- existem diferentes formas de viés: desde a falta de representatividade de grupos minoritários até a priorização de doenças que afetam populações mais privilegiadas. Esses vieses podem distorcer diagnósticos, atrasar tratamentos ou restringir o acesso de populações vulneráveis a recursos escassos.
- a justiça distributiva é essencial: a equidade precisa ser medida em termos de resultados iguais, performance igual e alocação justa de recursos. Isso reforça os princípios constitutivos do SUS, que preveem universalidade e equidade como pilares da política de saúde brasileira.
- a ética deve estar presente em todo o ciclo de vida da IA: da concepção e coleta de dados à validação, implementação e supervisão contínua. O *pipeline* ético funciona como um guia prático para garantir que algoritmos sejam não apenas eficazes, mas também socialmente responsáveis.

Em outras palavras, a IA não é apenas uma ferramenta técnica, mas também uma tecnologia social, cujo impacto depende das escolhas humanas que guiam sua criação e uso.

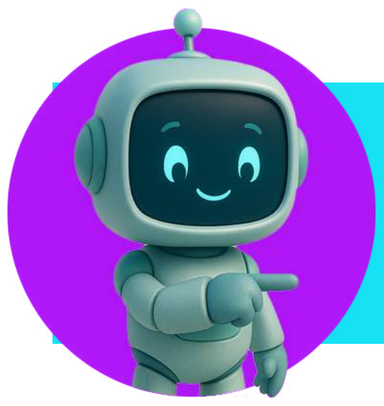
Perspectivas para o SUS

No contexto brasileiro, incorporar a ética à IA em saúde significa:

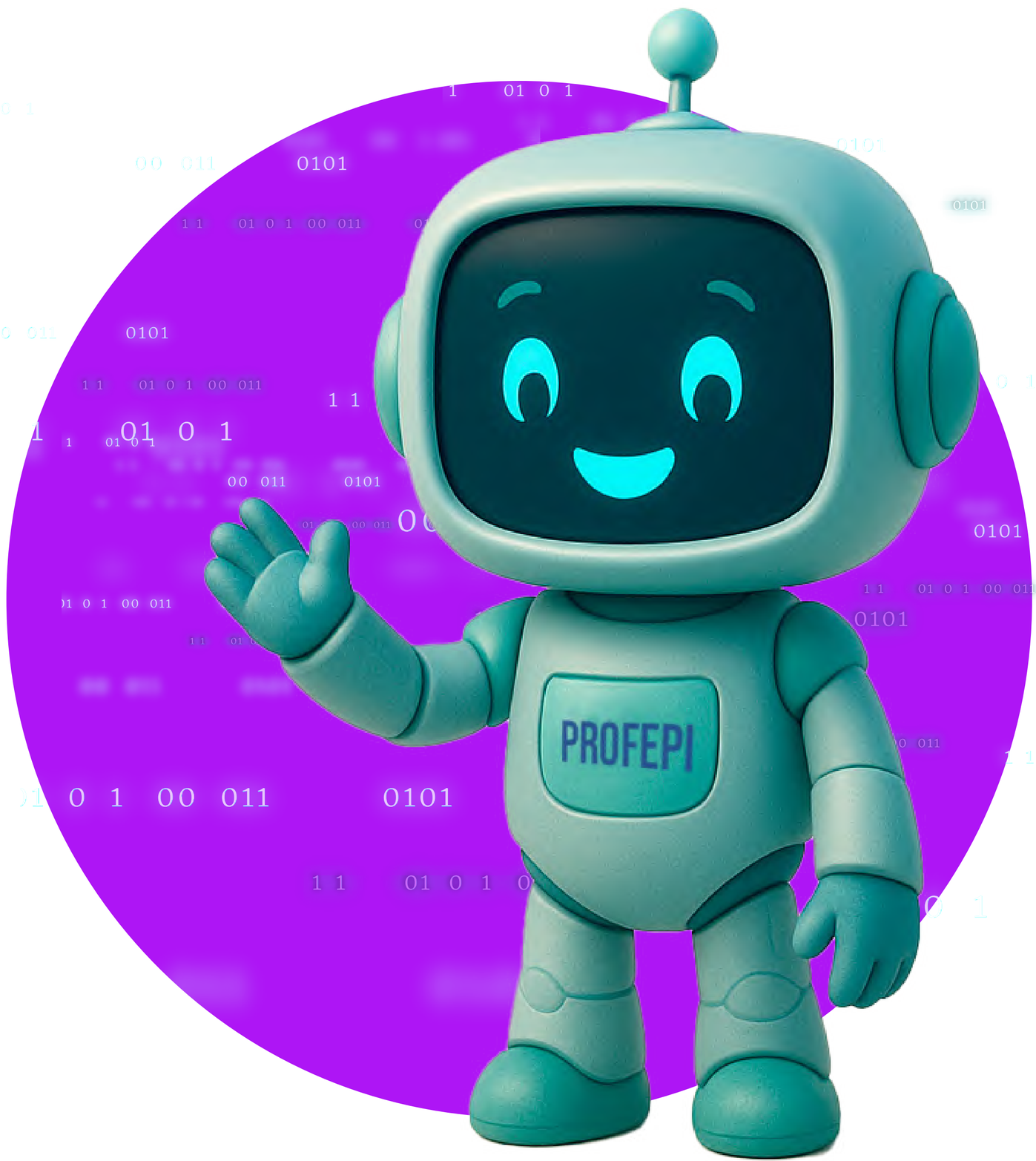
- fortalecer a representatividade dos grupos mais vulneráveis nas bases de dados (indígenas, quilombolas, populações periféricas);
- adotar práticas de auditoria e transparência, permitindo que modelos usados em políticas públicas possam ser revisados e compreendidos pela sociedade;
- formar equipes interdisciplinares que integrem epidemiologistas, cientistas de dados, profissionais de saúde e representantes sociais no desenvolvimento de algoritmos;
- garantir regulação e governança, alinhando-se a diretrizes internacionais, como as recomendações da Organização Mundial da Saúde (WHO, 2021), mas adaptadas ao contexto do SUS.

Reflexão final

Se mal conduzida, a inteligência artificial pode se tornar uma “máquina de desigualdades”, reproduzindo preconceitos já enraizados. Por outro lado, se projetada com justiça e equidade desde o início, ela pode ser uma aliada estratégica na redução das iniquidades em saúde.



Assim, a questão central não é apenas se devemos usar IA na saúde, mas como iremos usá-la: com responsabilidade ética, rigor técnico e compromisso com os princípios do SUS.



OPAS
Organização Pan-Americana da Saúde
Organização Mundial da Saúde
ESCRITÓRIO REGIONAL PARA AS Américas

SUS

MINISTÉRIO DA SAÚDE

GOVERNO DO BRASIL
DO LADO DO POVO BRASILEIRO



PROFEPI

Programa de Fortalecimento da Epidemiologia nos Serviços de Saúde